

МЕТОДИЧЕСКОЕ ПОСОБИЕ · ПРОЕКТ MORSE-LIDAR

Топология и геометрия 3D-поверхностей для дескриптора Морса–Смейла

Теория Морса, триангуляции, персистентные гомологии, кривизна и граничные эффекты — ровно то, что нужно, чтобы понимать и менять код проекта

Для команды проекта: Чупров Д. А., Евстратов И. М.
РТУ МИРЭА, ИПТИП · 2026

Содержание

0. Введение: что делает проект и как читать методичку

- 0.1 Зачем эта методичка
- 0.2 Конвейер: от облака точек до числа
- 0.3 Порядок чтения
- 0.4 Обозначения
- 0.5 Как запускать примеры

1. Линейная алгебра для 3D-геометрии

- 1.1 Векторы и координаты
- 1.2 Скалярное произведение и угол
- 1.3 Векторное произведение, площадь, нормаль, ориентация
- 1.4 Матрицы, повороты, ортогональные преобразования
- 1.5 Ковариационная матрица и PCA
- 1.6 Линейная функция на треугольнике и её градиент
- 1.7 Котангенс угла треугольника
- 1.8 Вопросы для самопроверки
- 1.9 Что почитать

2. Симплициальные комплексы и триангуляции

- 2.1 Симплексы
- 2.2 Симплициальный комплекс
- 2.3 Звезда и линк вершины
- 2.4 Когда комплекс является поверхностью
- 2.5 Граничные рёбра и вершины
- 2.6 Ориентируемость
- 2.7 Триангуляция регулярной сетки
- 2.8 Периодическая сетка как тор
- 2.9 2.5D-триангуляция Делоне облака
- 2.10 Реконструкция Пуассона
- 2.11 Неизменяемость `TriMesh` и кэширование
- 2.12 Вопросы для самопроверки
- 2.13 Что почитать

3. Эйлерова характеристика

- 3.1 Определение и вычисление
- 3.2 Инвариантность
- 3.3 Классификация компактных поверхностей
- 3.4 χ сеток проекта
- 3.5 Связь с критическими точками
- 3.6 Почему на открытой поверхности проверки нет
- 3.7 Персистентность сохраняет баланс
- 3.8 Пример: тор и $\cos x + \cos y$
- 3.9 Вопросы для самопроверки
- 3.10 Что почитать

4. Гладкая теория Морса

- 4.1 Функция на поверхности
- 4.2 Градиент и критические точки
- 4.3 Лемма Морса
- 4.4 Вырожденные точки и обезьянье седло
- 4.5 Функции Морса типичны
- 4.6 Подуровневые множества и главная теорема
- 4.7 Неравенства Морса и эйлерова характеристика
- 4.8 Градиентный поток, интегральные линии, устойчивые многообразия
- 4.9 Вопросы для самопроверки
- 4.10 Что почитать

5. Критические точки на триангулированных поверхностях

- 5.1 Кусочно-линейная функция
- 5.2 Звезда, линк, нижний и верхний линк
- 5.3 Пример: вершина с шестью соседями
- 5.4 Код: `link_components`, `lower_upper_links`, `analyze_morse`
- 5.5 Симуляция простоты
- 5.6 Плато и персистентность
- 5.7 Реальный прогон: шум
- 5.8 Граница
- 5.9 Вопросы для самопроверки
- 5.10 Что почитать

6. Комплекс Морса–Смейла и сепаратрисы

- 6.1 Интегральные линии градиента
- 6.2 Устойчивые и неустойчивые многообразия. Сепаратрисы
- 6.3 Условие Морса–Смейла
- 6.4 Ячейки комплекса Морса–Смейла
- 6.5 Дискретный вариант: теория Морса Формана
- 6.6 Эвристический бэкэнд `separatrices.py`
- 6.7 Чем эвристика отличается от настоящего комплекса
- 6.8 ТТК-бэкэнд
- 6.9 Упрощение по персистентности: ТТК и наш код
- 6.10 Вопросы для самопроверки
- 6.11 Что почитать

7. Графы и двойственные структуры

- 7.1 Графы: основные понятия
- 7.2 Обход графа: DFS и BFS на примере `link_components`
- 7.3 Мультиграфы, петли, кратные рёбра
- 7.4 Двойственный граф триангуляции
- 7.5 От ячеек Морса–Смейла к раскрашенному трёхвалентному графу
- 7.6 Проверки `validate_colored_trivalent_graph` и `validate_colored_graph`
- 7.7 Почему граф — полный комбинаторный инвариант
- 7.8 Пример руками: сфера и тор
- 7.9 Вопросы для самопроверки
- 7.10 Что почитать

8. Персистентные гомологии и сравнение сканов

- 8.1 Гомологии без алгебры
- 8.2 Фильтрация и нижняя звезда
- 8.3 Рождение, смерть и правило старшего
- 8.4 Диаграмма персистентности и порог
- 8.5 Пары в вершинах: `critical_pairs()`
- 8.6 Расстояние bottleneck и теорема устойчивости
- 8.7 Эксперимент повторных сканов
- 8.8 Вопросы для самопроверки
- 8.9 Что почитать

9. Геометрия поверхности, кривизна и скалярные поля

- 9.1 Метрика и топология
- 9.2 Скалярное поле: от его выбора зависит весь дескриптор
- 9.3 Нормаль, главные кривизны, H и K
- 9.4 Theorema Egregium и теорема Гаусса–Бонне
- 9.5 Дискретная гауссова кривизна: угловой дефект
- 9.6 Дискретная средняя кривизна: котангенсный оператор Лапласа–Бельтрами
- 9.7 Shape index Кендеринка
- 9.8 Инвариантность к повороту и чувствительность к шуму
- 9.9 Проверка на сфере
- 9.10 Вопросы для самопроверки
- 9.11 Что почитать

10. Граничные эффекты и открытые поверхности

- 10.1 Почему скан — открытая поверхность
- 10.2 Граничные вершины и рёбра
- 10.3 Что ломается на границе
- 10.4 Маскирование в коде
- 10.5 Сепаратрисы, уходящие в край
- 10.6 Кривизна на краю
- 10.7 Персистентность на открытой поверхности
- 10.8 Как закрыть поверхность
- 10.9 Типичные ошибки и симптомы
- 10.10 Вопросы для самопроверки
- 10.11 Что почитать

11. Практикум, учебный маршрут, глоссарий, литература

- 11.1 Учебный маршрут на 2–4 недели
- 11.2 Сквозные упражнения
- 11.3 Ответы и подсказки
- 11.4 Почему Python, а не C++
- 11.5 Глоссарий
- 11.6 Литература

0. Введение: что делает проект и как читать методичку

0.1 Зачем эта методичка

Проект `morse-lidar` строит по 3D-скану компактный **топологический дескриптор** поверхности. В него входят критические точки (critical points), сепаратрисы (separatrices) и диаграмма персистентности (persistence diagram). Дескрипторы двух сканов можно сравнить числом: расстоянием bottleneck (бутылочного горлышка, bottleneck distance). Код на 2,4 тысячи строк небольшой, но плотный: почти каждая строка опирается на конкретный факт из теории Морса, вычислительной топологии или дифференциальной геометрии. Если этих фактов не знать, код читается как набор странных проверок. Если знать, становится видно, *почему* сделано именно так и что сломается при изменении.

Методичка собрана под этот репозиторий, а не как курс «топологии вообще». Каждая глава устроена одинаково: интуиция, определение, пример с числами и раздел «Как это в коде» со ссылками на модули `src/morse_lidar/`. Почти все рисунки посчитаны самой библиотекой. То, что изображено на картинке, и есть результат работы функций из репозитория.

0.2 Конвейер: от облака точек до числа

Команда `morse-lidar (cli.py, функция main)` выполняет цепочку шагов. Ниже указано, в какой главе разобран каждый шаг.

№	Шаг	Где в коде	Глава
1	Загрузка облака точек (PLY, PCD, LAS, XYZ) или глубинной карты .npy	<code>pointcloud.load_point_cloud</code> , <code>preprocess.load_depth</code>	—
2	Ось «вверх», удаление пола или стола (RANSAC), выравнивание PCA, вокселизация	<code>set_up_axis</code> , <code>remove_dominant_plane</code> , <code>align_pca</code> , <code>voxel_downsample</code>	1, 9
3	Поверхность: растр → регулярная сетка, Делоне 2.5D или Пуассон	<code>rasterize_depth</code> , <code>mesh.grid_mesh</code> , <code>surface.reconstruct_surface</code>	2
4	Скалярное поле на вершинах: высота, гауссова или средняя кривизна, shape index	<code>intrinsic.intrinsic_scalar</code>	9
5	Критические точки по нижнему и верхнему линку (lower/upper link) с симуляцией простоты (simulation of simplicity, SoS)	<code>critical.analyze_morse</code> , <code>TriMesh.order</code>	4, 5
6	Сепаратрисы: эвристический монотонный спуск по соседям или ТТК	<code>separatrices.trace_separatrices</code> , <code>ttk_backend.run_ttk_msc</code>	6, 7
7	Проверка инвариантов: эйлерова характеристика (Euler characteristic), трёхвалентный граф	<code>descriptor.validate_euler</code> , <code>validate_colored_graph</code>	3, 7
8	Персистентность: пары критических точек, диаграмма, порог	<code>persistence.critical_pairs</code> , <code>persistent_counts</code>	8
9	JSON-отчёт (контракт <code>docs/DATA_CONTRACT.md</code>)	<code>cli._descriptor_json</code>	10
10	Сравнение двух отчётов и эксперимент повторных сканов	<code>compare.compare_reports</code> , <code>experiment.run_experiment</code>	8

Граница поверхности проходит через все шаги. Реальный скан почти всегда открыт, а на краю ломаются и классификация критических точек, и кривизна. Поэтому граничным эффектам отведена отдельная глава 10.

0.3 Порядок чтения

Главы расположены в учебном порядке: каждая опирается только на предыдущие. По важности для проекта они ранжируются иначе. Если времени мало, читайте в таком порядке:

Приоритет	Тема	Главы
1	Теория Морса и критические точки на сетке	4, 5
2	Персистентность и сравнение сканов	8
3	Триангуляции и эйлерова характеристика	2, 3
4	Комплекс Морса–Смейла (Morse–Smale complex), сепаратрисы, двойственный граф	6, 7
5	Кривизна и выбор скалярного поля	9
6	Граница и открытые поверхности	10
7	Линейная алгебра (справочник по мере надобности)	1

Минимальный набор, чтобы понимать код и менять его, — главы 2–5 и 8. Остальные нужны для диагностики: почему на краю много экстремумов, почему отчёты не сравниваются, почему ТТК отказывается работать с открытым сканом.

Учебный маршрут на 2–4 недели, упражнения, глоссарий и список литературы собраны в главе 11.

0.4 Обозначения

Обозначение	Смысл
M	поверхность (2-многообразие), возможно с границей ∂M
K	симплициальный комплекс (триангуляция), V, E, F — числа вершин, рёбер, треугольников
$f: M \rightarrow \mathbb{R}$	скалярное поле (функция высоты, кривизна)
$\text{Lk}(v), \text{St}(v)$	линк и звезда вершины v
$\text{Lk}^-(v), \text{Lk}^+(v)$	нижний и верхний линк
$\chi(M)$	эйлерова характеристика
$\beta_0, \beta_1, \beta_2$	числа Бетти
$\text{Dgm}(f)$	диаграмма персистентности функции f
d_B	расстояние bottleneck между диаграммами
H, K	средняя и гауссова кривизна (буква K для кривизны и для комплекса различается по контексту)

0.5 Как запускать примеры

Все примеры в тексте проверены в виртуальном окружении репозитория:

```
cd morse-lidar
python -m venv .venv && .venv/bin/pip install -e '[topology,signal]'
.venv/bin/python -c "import morse_lidar, gudhi; print('ok')"
```

Пример полного прогона на синтетике и сравнения двух отчётов:

```
.venv/bin/morse-lidar --input scan.ply --output a.json \  
  --geometry delaunay --voxel-size 1 --persistence-threshold 0.5  
.venv/bin/morse-lidar-compare a.json b.json
```

1. Линейная алгебра для 3D-геометрии

Весь конвейер morse-lidar начинается с массива точек формы $(n, 3)$ и заканчивается треугольной сеткой со скаляром в каждой вершине. Между этими этапами работают несколько операций линейной алгебры: скалярное и векторное произведения (углы, площади, нормали), ортогональные преобразования (смена вертикальной оси, выравнивание по главным осям) и собственные векторы ковариационной матрицы. В этой главе разобраны именно они, с привязкой к функциям `mesh.py`, `pointcloud.py`, `surface.py` и `intrinsic.py`.

1.1 Векторы и координаты

Точка сканера записывается тройкой $\mathbf{p} = (x, y, z) \in \mathbb{R}^3$. Разность двух точек $\mathbf{b} - \mathbf{a}$ есть вектор (vector): направленный отрезок от $\mathbf{a}$ к $\mathbf{b}$. Векторы складываются по координатам и умножаются на числа; длина (норма) вектора $|\mathbf{v}| = \sqrt{v_x^2 + v_y^2 + v_z^2}$.

Система координат правая (right-handed), если поворот от оси x к оси y на кратчайший угол, видимый с конца оси z , идёт против часовой стрелки. Все формулы ниже, включая знак векторного произведения, предполагают правую систему.

В коде. Облако хранится в `PointCloud.points` (массив `float` формы $(n, 3)$), вершины сетки в `TriMesh.vertices` той же формы. Конструкторы проверяют форму и конечность значений (`np.isfinite`).

1.2 Скалярное произведение и угол

Скалярное произведение (dot product)

$$\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y + a_z b_z = |\mathbf{a}| |\mathbf{b}| \cos \theta,$$

где $\theta \in [0, \pi]$ — угол между векторами. Отсюда стандартная формула $\theta = \arccos \frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}| |\mathbf{b}|}$. Ортогональность означает $\mathbf{a} \cdot \mathbf{b} = 0$, проекция $\mathbf{b}$ на единичный вектор $\mathbf{u}$ равна $\mathbf{u} \cdot \mathbf{b}$.

Формула через `arccos` плохо работает для малых (и близких к π) углов. Производная `arccos` в точке 1 бесконечна: $\cos \theta \approx 1 - \theta^2/2$, и при $\theta \sim 10^{-8}$ величина $\theta^2/2 \sim 10^{-16}$ теряется в округлении `double`. Проверка:

```
a, b = np.array([1., 0, 0]), np.array([1., 1e-8, 0])
np.arccos(a @ b / np.linalg.norm(a) / np.linalg.norm(b)) # 0.0
np.arctan2(np.linalg.norm(np.cross(a, b)), a @ b) # 1e-08
```

Устойчивый вариант использует пару синус-косинус: $|\mathbf{a} \times \mathbf{b}| = |\mathbf{a}| |\mathbf{b}| \sin \theta$ (см. 1.3), поэтому

$$\theta = \arctan2(|\mathbf{a} \times \mathbf{b}|, \mathbf{a} \cdot \mathbf{b}).$$

Нормировать векторы не нужно: общий множитель $|\mathbf{a}| |\mathbf{b}|$ сокращается, а `arctan2` сохраняет полную относительную точность и при малых, и при тупых углах.

В коде. `intrinsic.py`, функция `_corner_geometry()`: для каждого угла треугольника берутся рёбра `first`, `second`, считаются `sine = |first × second|` и `cosine = first · second`, затем `angles = np.arctan2(sine, cosine)`. Сумма углов при вершине нужна для гауссовой кривизны (гл. 9), а на тонких «иглообразных» треугольниках реальных сканов углы бывают очень малы.

1.3 Векторное произведение, площадь, нормаль, ориентация

Векторное произведение (cross product)

$$\mathbf{a} \times \mathbf{b} = (a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x)$$

перпендикулярно обоим сомножителям, его длина $|a||b|\sin\theta$ равна площади параллелограмма, а направление задаётся правилом правой руки. Оно антикоммутативно: $b \times a = -a \times b$.

Для треугольника с вершинами a, b, c вектор

$$n = (b - a) \times (c - a), \quad S = \frac{1}{2}|n|$$

задаёт нормаль грани (face normal) и удвоенную площадь. Направление n зависит от порядка обхода вершин: обход $a \rightarrow b \rightarrow c$ против часовой стрелки, если смотреть с конца n . Перестановка двух вершин (a, c, b) меняет знак нормали. Порядок обхода называют ориентацией треугольника (orientation, winding).

Пример. $a = (0, 0, 0)$, $b = (3, 0.4, 0)$, $c = (1.2, 2.4, 0)$. Тогда $n = (0, 0, 3 \cdot 2.4 - 0.4 \cdot 1.2) = (0, 0, 6.72)$, $S = 3.36$. Нормаль смотрит в $+z$, обход в плоскости xy идёт против часовой.

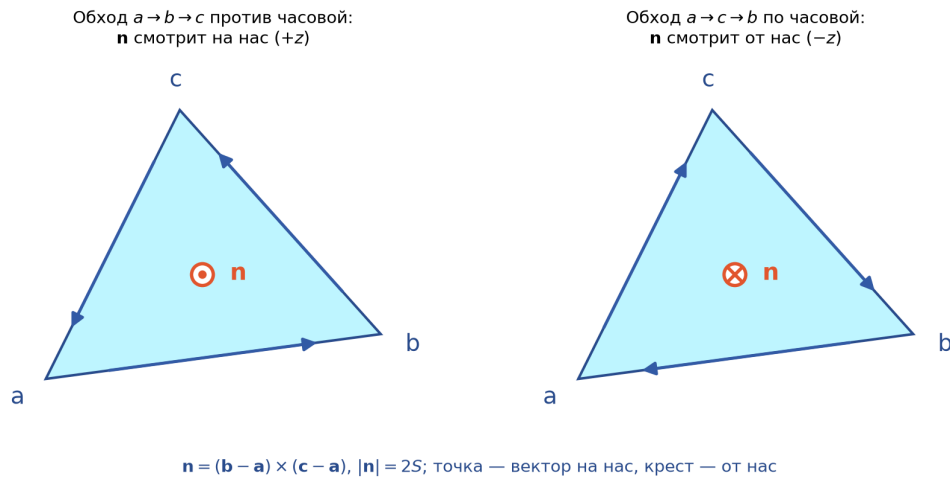


Рис. 1.1. Ориентация треугольника и направление нормали $n = (b - a) \times (c - a)$.

В коде. `mesh.py`, `TriMesh.vertex_areas`: площадь каждой грани $0.5 * |\text{cross}(b - a, c - a)|$, вершине достаётся треть площади каждой инцидентной грани (барицентрическая площадь, barycentric area). Для того же треугольника из примера каждая вершина получает $3.36/3 = 1.12$.

`TriMesh.vertex_normals`: ненормированные нормали граней `cross(b - a, c - a)` суммируются в каждую из трёх вершин через `np.add.at`, сумма нормируется. Так как $|n| = 2S$, это нормаль, взвешенная по площади (area-weighted): крупные грани влияют сильнее, вырожденные почти не влияют. Деление на `max(length, eps)` защищает от нулевой суммы.

Ориентация граней важна не только для красоты. Знак средней кривизны в `intrinsic.py` определяется скалярным произведением с `vertex_normals` (гл. 9); если половина граней обходится по часовой стрелке, а половина против, нормали в вершинах частично гасят друг друга и знак кривизны становится случайным.

В коде. `surface.py`, `reconstruct_delaunay()`: SciPy не гарантирует согласованный обход треугольников Делоне, поэтому после триангуляции вычисляются нормали всех граней, и у треугольников с `normals[:, scalar_axis] < 0` меняются местами две вершины: `faces[flip][:, [0, 2, 1]]`. После этого все нормали смотрят в сторону $+z$ (точнее, $+ \text{scalar_axis}$). Знак z -компоненты n зависит только от x, y вершин, то есть совпадает с ориентацией проекции треугольника на плоскость xy .

1.4 Матрицы, повороты, ортогональные преобразования

Линейное преобразование $\mathbb{R}^3 \rightarrow \mathbb{R}^3$ задаётся матрицей 3×3 : $p' = Mp$. В numpy точки лежат строками, поэтому то же самое записывается как `points @ M.T`. Матрица Q ортогональна, если $Q^T Q = I$; её столбцы образуют ортонормированный базис, она сохраняет длины и углы, и $\det Q = \pm 1$. При $\det Q = +1$ это поворот (rotation), при $\det Q = -1$ — поворот с зеркальным отражением. Отражение меняет правую систему на левую и, как следствие, переворачивает все нормали из 1.3.

В коде. `pointcloud.py`, `set_up_axis(cloud, up)`. Экспорт iPhone/ARKit использует систему «Y вверх», а пакет считает высотой Z . Для `up="y"` столбцы переставляются как `points[:, [2, 0, 1]]`, то есть $(x', y', z') = (z, x, y)$. Это циклическая перестановка, её матрица

$$P = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}, \quad \det P = +1,$$

то есть настоящий поворот. Простой обмен $y \leftrightarrow z$ имел бы $\det = -1$ и превратил бы купол в зеркальный купол с вывернутыми нормальными.

1.5 Ковариационная матрица и PCA

Пусть точки облака центрированы: $\mathbf{q}_i = \mathbf{p}_i - \bar{\mathbf{p}}$. Ковариационная матрица

$$C = \frac{1}{n-1} \sum_{i=1}^n \mathbf{q}_i \mathbf{q}_i^T$$

симметрична и неотрицательно определена. Для единичного вектора $\mathbf{u}$ величина $\mathbf{u}^T C \mathbf{u}$ есть дисперсия проекций точек на направление $\mathbf{u}$. По спектральной теореме $C = V \Lambda V^T$, где столбцы V — ортонормированные собственные векторы, $\Lambda = \text{diag}(\lambda_1 \geq \lambda_2 \geq \lambda_3 \geq 0)$. Максимум дисперсии достигается на $\mathbf{e}_1$, минимум на $\mathbf{e}_3$. Это метод главных компонент (principal component analysis, PCA). Для почти плоского скана $\mathbf{e}_3$ — нормаль к «средней плоскости» объекта, то есть естественное направление высоты.

Собственный вектор определён только с точностью до знака: если $C\mathbf{e} = \lambda\mathbf{e}$, то и $C(-\mathbf{e}) = \lambda(-\mathbf{e})$. Какой знак вернёт `np.linalg.eigh`, не гарантируется, а от знака оси Z зависит, станет объект куполом или чашей, то есть поменяются местами минимумы и максимумы.

В коде. `pointcloud.py`, `align_pca()`:

```
centred = cloud.points - cloud.points.mean(axis=0)
_, eigenvectors = np.linalg.eigh(np.cov(centred.T))
basis = eigenvectors[:, ::-1].copy() # largest variance first, smallest -> Z
if basis[2, 2] < 0: # new Z must not point against the input "up"
    basis[:, 2] *= -1.0
if ((centred @ basis[:, 0]) ** 3).mean() < 0:
    basis[:, 0] *= -1.0
basis[:, 1] = np.cross(basis[:, 2], basis[:, 0])
return PointCloud(centred @ basis)
```

`eigh` возвращает собственные значения по возрастанию, поэтому порядок столбцов разворачивается. Знак новой оси Z выбирается так, чтобы её z -компонента была неотрицательной (не смотреть против исходного «верха»; поэтому `set_up_axis` вызывается раньше, см. `cli.py`, `_build_mesh()`). Знак X фиксируется третьим моментом (асимметрией) проекций. Y не берётся из `eigh`, а вычисляется как $\mathbf{z} \times \mathbf{x}$, что гарантирует правую тройку и $\det = +1$.

Пример. Облако из 1500 точек с дисперсиями 9, 1, 0.01 вдоль локальных осей наклонено на 30° вокруг x . После `align_pca` дисперсии по X, Y, Z равны 9.01, 1.01, 0.010; проверка `M.T @ M` даёт единичную матрицу с погрешностью $2 \cdot 10^{-15}$ и $\det M = 1$.

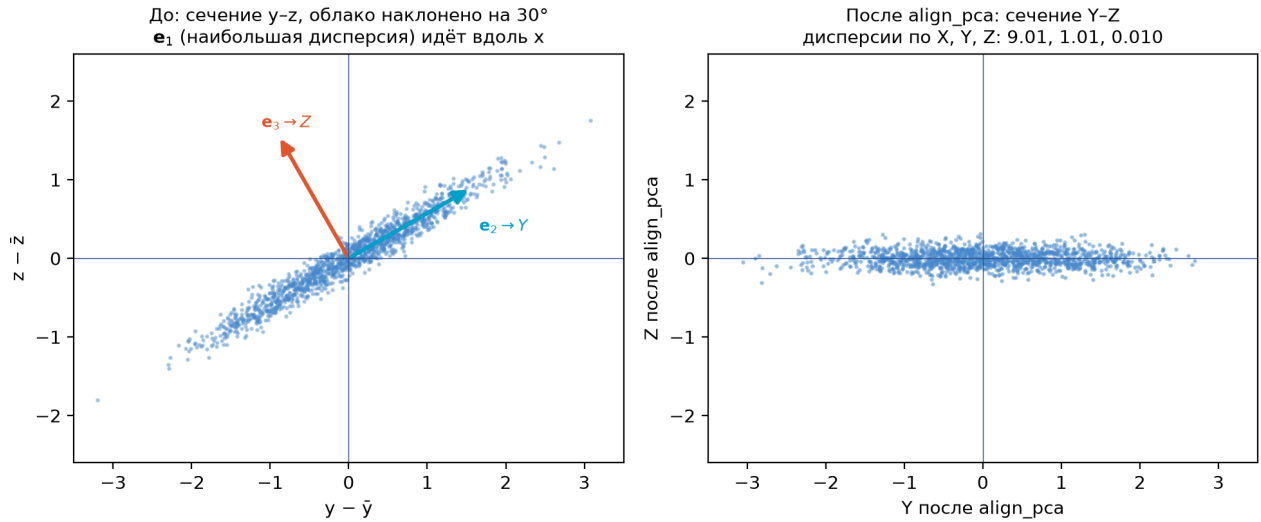


Рис. 1.2. PCA: слева главные оси исходного облака, справа то же облако после `align_pca()` (данные посчитаны этой функцией).

Типичная ошибка. Ожидать от `align_pca` устойчивости на любых объектах. Если $\lambda_2 \approx \lambda_3$ (вытянутый цилиндр) или $\lambda_1 \approx \lambda_2$ (круглая чашка), собственные векторы внутри почти равной пары определены плохо и прыгают от скана к скану. Если третий момент близок к нулю (симметричный объект), знак X решает шум. Если $\mathbf{e}_3$ почти горизонтален (скан стены при `ur="z"`), z -компонента близка к нулю и знак новой Z тоже неустойчив. Для сравнения сканов (гл. 8) это означает, что поза должна быть сопоставимой и до PCA.

1.6 Линейная функция на треугольнике и её градиент

Значения скаляра заданы только в вершинах. Внутри треугольника функцию продолжают линейно через барицентрические координаты (barycentric coordinates): точка $\mathbf{p} = \lambda_0 \mathbf{p}_0 + \lambda_1 \mathbf{p}_1 + \lambda_2 \mathbf{p}_2$, $\lambda_i \geq 0$, $\sum \lambda_i = 1$, получает значение

$$f(\mathbf{p}) = \lambda_0 f_0 + \lambda_1 f_1 + \lambda_2 f_2.$$

Координата λ_i равна отношению площади треугольника, противолежащего вершине i (с вершиной $\mathbf{p}$ вместо $\mathbf{p}_i$), к площади S . Такая функция линейна на грани, поэтому её градиент постоянен:

$$\nabla f = \frac{1}{2S} \sum_{i=0}^2 f_i (\hat{\mathbf{n}} \times \mathbf{e}_i),$$

где $\hat{\mathbf{n}}$ — единичная нормаль, $\mathbf{e}_i$ — ребро, противолежащее вершине i , взятое по обходу ($\mathbf{e}_0 = \mathbf{p}_2 - \mathbf{p}_1$ и далее циклически). Вектор $\hat{\mathbf{n}} \times \mathbf{e}_i$ лежит в плоскости грани и перпендикулярен ребру $\mathbf{e}_i$, то есть указывает к вершине i .

Пример. Треугольник из 1.3, значения $f = (1, 2.5, 4)$. Формула даёт $\nabla f = (0.357, 1.071, 0)$. Проверка: $\nabla f \cdot (\mathbf{b} - \mathbf{a}) = 1.5 = f_b - f_a$ и $\nabla f \cdot (\mathbf{c} - \mathbf{a}) = 3 = f_c - f_a$.

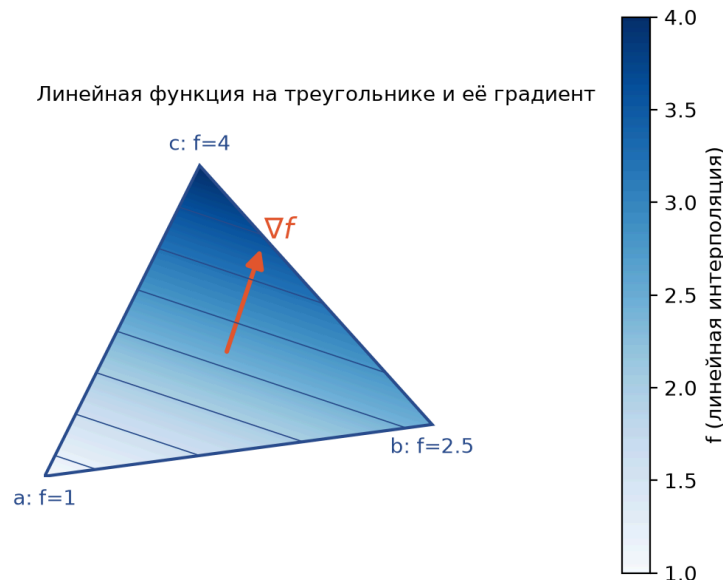


Рис. 1.3. Линейная интерполяция по барицентрическим координатам: линии уровня параллельны, градиент постоянен и перпендикулярен им.

Отсюда вывод, важный для гл. 5–6: у кусочно-линейной функции внутри грани и на ребре нет критических точек (градиент там ненулевой, если значения в вершинах различны), критическими могут быть только вершины.

В коде. Градиент по граням в репозитории явно не вычисляется. Классификатор `critical.py` работает только с порядком значений в вершинах, а эвристические сепаратрисы `separatrices.py` идут к соседней вершине, ближайшей по рангу среди более низких (при подъёме — среди более высоких), а не по ∇f (подробнее в гл. 6). Формула выше нужна, чтобы понимать, насколько грубо такой монотонный путь приближает настоящий градиентный поток.

1.7 Котангенс угла треугольника

Для угла θ между рёбрами $\mathbf{u}, \mathbf{v}$, выходящими из одной вершины,

$$\cot\theta = \frac{\cos\theta}{\sin\theta} = \frac{\mathbf{u} \cdot \mathbf{v}}{|\mathbf{u} \times \mathbf{v}|}.$$

Котангенсы естественно возникают из градиентов барицентрических координат: для вершин $i \neq j$ с третьей вершиной k

$$S \nabla\lambda_i \cdot \nabla\lambda_j = -\frac{1}{2}\cot\theta_k.$$

В примере из 1.3 угол при c равен 68.55° , $\cot = 0.3929$, и обе части равенства дают -0.1964 . Суммируя такие слагаемые по граням, получают котангенсный лапласиан (cotangent Laplacian), из которого в гл. 9 вычисляется средняя кривизна.

В коде. `intrinsic.py, _corner_geometry():` `cotangents = cosine / max(sine, eps)` с теми же ненормированными `sine` и `cosine`, что и для углов. Защита `eps` спасает от деления на ноль, но у вырожденного треугольника котангенс всё равно огромен, поэтому тонкие треугольники Делоне заметно портят кривизну.

1.8 Вопросы для самопроверки

1. Почему угол в `_corner_geometry()` вычисляется через `arctan2`, а не через `arccos`? Для каких углов разница существенна?

2. Как изменится `vertex_normals`, если у одной грани поменять местами две вершины?
3. Почему в `vertex_normals` не нормируют нормали граней перед суммированием?
4. Почему `set_up_axis` для `up="y"` использует перестановку `[2, 0, 1]`, а не `[0, 2, 1]`?
5. Какие две неоднозначности знака решает `align_pca` и как? Почему ось Y вычисляется векторным произведением?
6. Где у кусочно-линейной функции на треугольной сетке могут находиться критические точки?

Ответы

1. Около $\theta = 0$ и $\theta = \pi$ производная `arccos` бесконечна, и потеря точности в косинусе даёт большую ошибку угла (для 10^{-8} рад `arccos` возвращает 0). Пара `(sin, cos)` в `arctan2` точна во всём диапазоне.
2. Нормаль этой грани сменит знак, и её вклад в нормаль каждой из трёх вершин станет противоположным; соседние нормали частично погасятся.
3. Длина ненормированной нормали равна $2S$, поэтому сумма автоматически взвешена по площади граней.
4. `[2, 0, 1]` — циклическая перестановка с $\det = +1$ (поворот). `[0, 2, 1]` — обмен двух осей с $\det = -1$ (отражение), он меняет ориентацию и знак кривизны.
5. Знак Z : z -компонента новой оси неотрицательна. Знак X : третий момент проекций неотрицателен. $Y = Z \times X$ делает базис правым, $\det = +1$.
6. Только в вершинах: внутри грани градиент постоянен и при различных значениях в вершинах не равен нулю.

1.9 Что почитать

- Botsch M., Kobbelt L., Pauly M., Alliez P., Lévy B. *Polygon Mesh Processing*. A K Peters, 2010 — гл. 1–3 (нормали, барицентрические координаты, котангенсная формула).
- Strang G. *Linear Algebra and Its Applications*. — разделы про ортогональные матрицы и спектральное разложение симметричной матрицы.

2. Симплициальные комплексы и триангуляции

Теория Морса в проекте применяется не к гладкой поверхности, а к треугольной сетке. Чтобы классификатор критических точек (гл. 5) и формула Эйлера (гл. 3) работали, сетка должна быть не произвольным набором треугольников, а симплициальным комплексом, который к тому же является поверхностью. В этой главе вводятся нужные определения и разбирается, как сетки строятся в `mesh.py` и `surface.py`.

2.1 Симплексы

Интуиция: вершина, отрезок и треугольник — простейшие «кирпичи» размерностей 0, 1, 2, из которых склеивается поверхность.

Определение. k -мерный симплекс (simplex) $\sigma = [v_0, \dots, v_k]$ — выпуклая оболочка $k + 1$ аффинно независимых точек. В проекте используются 0-симплексы (вершины), 1-симплексы (рёбра) и 2-симплексы (треугольники, грани). Грань (face) симплекса σ — симплекс, натянутый на любое непустое подмножество его вершин. У треугольника $[a, b, c]$ семь граней: три вершины, три ребра $[a, b]$, $[b, c]$, $[a, c]$ и сам треугольник.

Комбинаторно симплекс задаётся просто множеством номеров вершин. Именно так он хранится в коде: строка `faces[i]` массива формы `(m, 3)` — это три номера вершин треугольника.

2.2 Симплициальный комплекс

Определение. Симплициальный комплекс (simplicial complex) K — конечный набор симплексов, такой что (1) любая грань симплекса из K тоже лежит в K ; (2) два симплекса из K пересекаются либо по пустому множеству, либо по общей грани.

Условие (2) запрещает, например, вершину, лежащую посередине чужого ребра, или два треугольника с одинаковым набором вершин. В абстрактной (комбинаторной) версии геометрии нет вовсе, остаётся только условие (1).

В коде. `TriMesh` хранит только треугольники; рёбра выводятся из них. Вершины хранятся массивом `vertices`, и вершина, не входящая ни в один треугольник, тоже считается частью комплекса (изолированный 0-симплекс). Проверок условия (2) нет: `__post_init__` проверяет формы массивов, конечность и диапазон индексов, а повтор вершины внутри треугольника обнаруживается только при первом вызове `neighbors_and_link()` (`ValueError("faces must not repeat a vertex")`).

2.3 Звезда и линк вершины

Эти два понятия — главный инструмент гл. 5: тип вершины (минимум, седло, максимум) определяется тем, как устроен её линк.

Интуиция: звезда — «зонтик» из всех треугольников вокруг вершины; линк — край этого зонтика, кольцо соседей.

Определение. Звезда (star) вершины v : $St(v) = \{\sigma \in K: v \in \sigma\}$ — все симплексы, содержащие v . Замкнутая звезда $\bar{St}(v)$ — звезда вместе со всеми гранями её симплексов. Линк (link): $Lk(v) = \{\tau \in \bar{St}(v): v \notin \tau\}$ — те симплексы замкнутой звезды, которые не содержат v . Для треугольной сетки линк — граф: его вершины — соседи v , его рёбра — рёбра, противолежащие v в треугольниках звезды.

Пример. Сетка `grid_mesh` размера 5×5 (рис. ниже). Внутренняя вершина 12 входит в 6 треугольников, её соседи $\{6, 7, 11, 13, 17, 18\}$, рёбра линка $(6, 7), (7, 13), (13, 18), (17, 18), (11, 17), (6, 11)$ образуют цикл. Граничная вершина 2 входит в 3 треугольника, соседи $\{1, 3, 7, 8\}$, рёбра линка $(1, 7), (7, 8), (3, 8)$ образуют путь от граничного соседа 1 к граничному соседу 3.

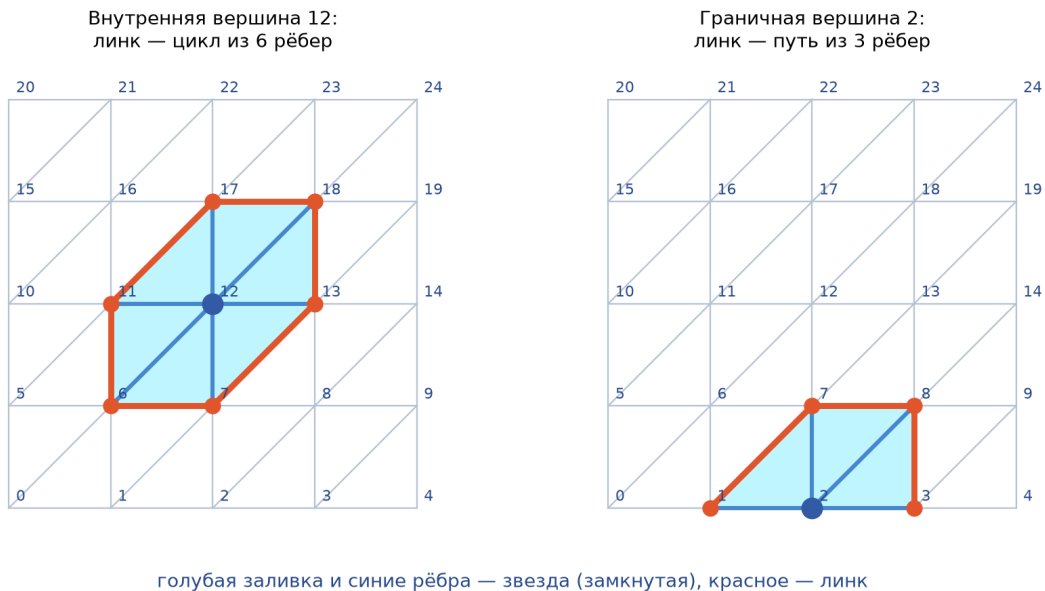


Рис. 2.1. Звезда и линк вершины во внутренней и граничной позиции. Рисунок построен по данным `TriMesh.neighbors_and_link()`.

В коде. `mesh.py`, `TriMesh._neighbors_and_link`:

```
for face in self.faces:
    for vertex in face:
        others = [int(item) for item in face if item != vertex]
        ...
        first, second = others
        neighbors[int(vertex)].update((first, second))
        link_edges[int(vertex)].add(tuple(sorted((first, second))))
```

Каждый треугольник $[a, b, c]$ добавляет в линк вершины a противолежащее ребро (b, c) и т. д. Результат кэшируется и возвращается кортежами `frozenset`. В `critical.py` функция `lower_upper_links()` делит соседей на тех, кто ниже и выше v , и считает компоненты связности нижнего и верхнего линка (lower/upper link) именно по этим рёбрам.

2.4 Когда комплекс является поверхностью

Определение. Комплекс из треугольников — поверхность (2-многообразие, manifold) с краем, если (а) каждое ребро лежит ровно в одном или двух треугольниках; (б) линк каждой вершины — либо цикл (внутренняя вершина), либо простой путь (граничная вершина).

Условие (а) запрещает «книжку» из трёх страниц на одном ребре. Условие (б) запрещает «бабочку» (bowtie): два зонтика, соприкасающиеся в одной вершине; её линк состоит из двух компонент. Окрестность внутренней вершины поверхности гомеоморфна диску, граничной — полудиску; цикл и путь в линке — комбинаторное выражение этого факта.

Типичная ошибка. Считать, что `TriMesh` гарантирует многообразие. Это не проверяется. Если у сетки 9×9 удалить треугольники неаккуратным квадратным окном, у края дыры появляются две вершины-бабочки, и `euler_characteristic` даёт -2 вместо 0 для кольца; с аккуратным окном получается $V = 80$, $E = 200$, $F = 120$, $\chi = 0$. Ребро в трёх треугольниках вообще выпадает из классификации: оно не граничное (`count == 1`) и не внутреннее, а `closed` просто вернёт `False`. Классификатор `analyze_morse` на таких вершинах выдаёт бессмыслицу, потому что его логика рассчитана на линк-цикл или линк-путь.

2.5 Граничные рёбра и вершины

Из условия (а) следует простой критерий: ребро граничное (boundary edge), если оно принадлежит ровно одному треугольнику, и внутреннее, если двум. Граничная вершина — конец хотя бы одного граничного ребра. Поверхность замкнута (closed), если граничных рёбер нет.

В коде. mesh.py :

```
@cached_property
def edge_counts(self) -> Mapping[tuple[int, int], int]:
    edges = np.sort(self.faces[:, [0, 1, 1, 2, 2, 0]].reshape(-1, 2), axis=1)
    unique, counts = np.unique(edges, axis=0, return_counts=True)
    return MappingProxyType({...})

@cached_property
def closed(self) -> bool:
    return bool(self.edge_counts) and all(count == 2 for count in self.edge_counts.values())

@cached_property
def boundary_edges(self) -> frozenset[tuple[int, int]]:
    return frozenset(edge for edge, count in self.edge_counts.items() if count == 1)
```

Каждый треугольник порождает три ребра; после сортировки концов ребро (i, j) и (j, i) становятся одним ключом, и `np.unique` считает, сколько треугольников его содержат. `boundary_vertices` — объединение концов граничных рёбер.

Граничные вершины важны далее: `analyze_morse()` по умолчанию их пропускает (`include_boundary=False`), а `intrinsic.py` заменяет на них кривизну средним по внутренним соседям (гл. 9, 10).

2.6 Ориентируемость

Ориентация треугольника — циклический порядок обхода его вершин (гл. 1.3). Две смежные грани ориентированы согласованно (consistently), если общее ребро они проходят в противоположных направлениях: $[a, b, c]$ и $[b, a, d]$ согласованы ($a \rightarrow b$ и $b \rightarrow a$), а $[a, b, c]$ и $[a, b, d]$ — нет. Поверхность ориентируема (orientable), если ориентации всех граней можно согласовать. Сфера, тор, диск, кольцо ориентируемы; лента Мёбиуса нет.

Практический тест: в согласованно ориентированной поверхности каждое ориентированное ребро $(i \rightarrow j)$ встречается не более одного раза. Для всех сеток, которые строит пакет (`grid_mesh`, `periodic_grid_mesh`, `reconstruct_delaunay`), этот тест проходит; сам `TriMesh` ориентацию не проверяет и называет себя `oriented` только в docstring.

2.7 Триангуляция регулярной сетки

Карта глубины — массив `rows × cols`. Вершина с индексом (r, c) получает номер $r \cdot \text{cols} + c$ и координаты $(x_0 + s_x c, y_0 + s_y r, h_{rc})$ (`_grid_vertices`). Каждая клетка сетки — квадрат с углами $a = (r, c)$, $b = (r, c + 1)$, $c = (r + 1, c)$, $d = (r + 1, c + 1)$, и он режется одной диагональю $a-d$ на два треугольника.

В коде. `mesh.py, _grid_faces()`:

```
a, b, c, d = index(y, x), index(y, x + 1), index(y + 1, x), index(y + 1, x + 1)
faces = np.empty((2 * len(a), 3), dtype=np.int64)
faces[0::2] = np.column_stack((a, b, d))
faces[1::2] = np.column_stack((a, d, c))
```

Оба треугольника обходятся против часовой стрелки в плоскости xy ($a \rightarrow b \rightarrow d$ и $a \rightarrow d \rightarrow c$), нормали смотрят в $+z$, общее ребро $a-d$ проходится в противоположных направлениях.



Рис. 2.2. Триангуляция `grid_mesh 4x4`: граничные рёбра и вершины выделены красным, в левой нижней клетке подписаны углы a, b, c, d . Рисунок построен по `TriMesh.edge_counts` и `boundary_edges`.

Почему диагональ одна и та же во всех клетках. Если выбирать диагональ произвольно, триангуляция остаётся корректной, но число соседей вершины начинает меняться от 4 до 8, и классификация критических точек зависит от случайного выбора. С одинаковой диагональю у каждой внутренней вершины ровно 6 соседей: 4 по осям и 2 по диагонали ($(r \pm 1, c \pm 1)$ с одинаковыми знаками). Шесть — минимальное число соседей, при котором в линке-цикле помещаются чередующиеся «ниже-выше-ниже-выше» для простого седла и даже «ниже-выше» трижды для мульти-седла (multi-saddle) кратности 2, например обезьяньего седла (monkey saddle) (гл. 5). Платой является анизотропия: направление $a-d$ выделено.

Подсчёт для сетки $n \times m$ (n строк, m столбцов): $V = nm$, $F = 2(n-1)(m-1)$, рёбра — горизонтальные $n(m-1)$, вертикальные $m(n-1)$ и диагональные $(n-1)(m-1)$. Граничных рёбер $2(n-1) + 2(m-1)$. Для 4×4 : $V = 16$, $E = 12 + 12 + 9 = 33$, $F = 18$, граничных рёбер и вершин по 12. Угловые вершины неравноправны: у вершины 0 три соседа (её клетка разрезана диагональю из неё), у вершины 3 только два (один треугольник).

2.8 Периодическая сетка как тор

Если склеить левый край прямоугольника с правым, а нижний с верхним, получится тор. Комбинаторно это делается взятием индексов по модулю: в `_grid_faces(..., periodic=True)` клеток не $(n-1)(m-1)$, а nm , и `index(row, col) = (row % rows) * cols + (col % cols)`. Клетки последнего столбца и последней строки «замыкаются» на первые. Получается замкнутая поверхность: $V = nm$, $E = 3nm$, $F = 2nm$, каждое ребро ровно в двух треугольниках, у каждой вершины 6 соседей. Функция `_validated_field` требует минимум 3×3 : при двух строках вершины $(r+1)$ и $(r-1)$ совпали бы, и линк перестал бы быть циклом из шести различных вершин.

Важно. Склейка только комбинаторная. Координаты вершин остаются плоскими, поэтому «шовные» треугольники геометрически растянуты через весь прямоугольник: для 8×8 у 28 из 128 граней нормаль смотрит в $-z$. Для высоты как скаляра это не важно (классификация использует только порядок значений), а для кривизны критично, поэтому `cli.py` запрещает `--scalar` кривизны вместе с `--surface periodic`.

2.9 2.5D-триангуляция Делоне облака

Облако LiDAR можно превратить в сетку, не растеризуя его: спроецировать точки на плоскость xy , построить там триангуляцию Делоне (Delaunay triangulation) и поднять треугольники обратно в 3D. Триангуляция Делоне — та, в которой описанная окружность каждого треугольника не содержит других точек; среди всех триангуляций набора точек она максимизирует минимальный угол. Сетка получается графиком функции $z = h(x, y)$ — поэтому «2.5D».

В коде. `surface.py`, `reconstruct_delaunay()`: 1. `_unique_projected_points()` ищет точки с одинаковой проекцией (`np.unique` по столбцам `horizontal_axes`) и заменяет каждую группу одной точкой — по координатной медианой (`np.median(..., axis=0)`). Без этого Delaunay получил бы совпадающие точки, а график не может иметь двух значений в одной точке. 2. `scipy.spatial.Delaunay(points[:, horizontal_axes])` даёт треугольники. 3. Треугольники переориентируются так, чтобы нормали смотрели в $+z$ (гл. 1.3).

Скаляр в вершинах — координата `scalar_axis` (высота).

Граница такой сетки — выпуклая оболочка проекции, поэтому по краю часто возникают длинные тонкие треугольники, соединяющие далёкие точки; $\chi = 1$ (топологический диск), если облако не вырождено.

Чего 2.5D-сетка не может представить: всё, что не является графиком над xy , — нависания (overhangs), вертикальные стенки, обратные стороны объекта. Если над полом висит полка, точки пола и полки чередуются в проекции, и Делоне соединяет их треугольниками, образуя «пилу» из почти вертикальных граней.

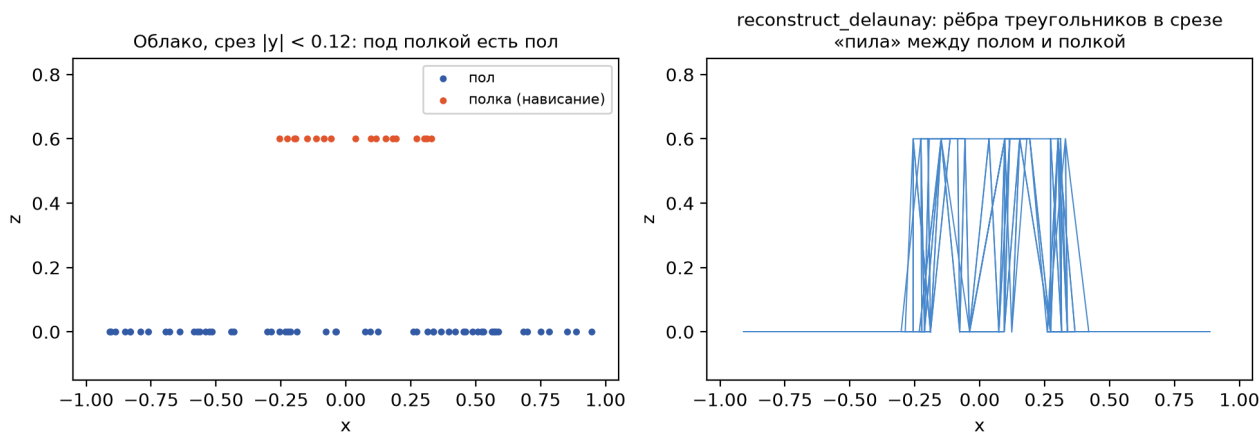


Рис. 2.3. Нависание в 2.5D-триангуляции: слева срез облака (пол и полка над ним), справа рёбра треугольников `reconstruct_delaunay` в том же срезе.

Типичная ошибка. Сливаются только точки с точно равными x, y . Почти совпадающие точки (в эксперименте — на расстоянии 10^{-13}) Qhull молча исключает из триангуляции; вершина остаётся в `vertices`, но не входит ни в один треугольник. Тогда `euler_characteristic` увеличивается на 1 (было 1, стало 2), а `analyze_morse` классифицирует изолированную вершину как минимум (у неё пустой нижний линк). Проверка, что каждая вершина используется, в коде отсутствует.

2.10 Реконструкция Пуассона

Реконструкция Пуассона (Poisson surface reconstruction) решает другую задачу: по точкам с нормальными векторами строит индикаторную функцию «внутри/снаружи» объекта и извлекает её поверхность уровня. Результат по построению замкнут (watertight) и может описывать любые 3D-формы, в том числе нависания.

В коде. `surface.py`, `reconstruct_poisson()` вызывает Open3D: оценивает нормали (`estimate_normals`, `orient_normals_consistent_tangent_plane`), строит сетку `create_from_point_cloud_poisson(depth=depth)` и получает плотность точек у каждой вершины. Вдали от данных алгоритм «досочиняет» поверхность (замыкает её), поэтому при `density_quantile > 0` вершины с плотностью ниже заданного квантиля удаляются (`remove_vertices_by_mask`). Это убирает выдуманную поверхность, но открывает сетку: появляются граничные рёбра, `closed` становится `False`. Значение `0` сохраняет замкнутость, которая нужна ТТК-бэкенду (`cli.py` отказывается запускать ТТК на незамкнутой сетке). Скаляр — координата z вершин.

В окружении `.venv` проекта Open3D не установлен, поэтому численных примеров для Пуассона в методичке нет; важно помнить, что после обрезки результат может содержать и неманифолдные вершины, о которых говорилось в 2.4.

2.11 Неизменяемость TriMesh и кэширование

Все топологические запросы (`edge_counts`, `boundary_vertices`, `neighbors_and_link`, `order`) вызываются внутри циклов по вершинам, поэтому вычисляются один раз и кэшируются через `functools.cached_property`. Кэш корректен, только если данные не меняются. `@dataclass(frozen=True)` запрещает переписывать поля, но не защищает содержимое numpy-массивов, поэтому `__post_init__` копирует их в собственные буферы только для чтения:

```
vertices = np.frombuffer(vertices.tobytes(), dtype=vertices.dtype).reshape(vertices.shape)
...
object.__setattr__(self, "vertices", vertices)
```

Массив, созданный `np.frombuffer` поверх объекта `bytes`, не может быть изменён: `mesh.vertices[0, 0] = 5` даёт `ValueError: assignment destination is read-only`. Словарь рёбер отдаётся как `MappingProxyType`, множества как `frozenset`. Чтобы получить сетку с другим скаляром, создают новый объект: `TriMesh(mesh.vertices, mesh.faces, new_values)` (так делает `cli.py` для кривизны).

2.12 Вопросы для самопроверки

1. Чем отличаются звезда, замкнутая звезда и линк вершины? Что из этого хранит `neighbors_and_link()`?
2. Как по линку отличить внутреннюю вершину поверхности, граничную и неманифолдную?
3. Почему `grid_mesh` режет все клетки одной и той же диагональю? Сколько соседей у внутренней вершины?
4. Сколько рёбер и граничных рёбер у `grid_mesh 5 × 7`? У `periodic_grid_mesh 5 × 7`?
5. Чего не может представить `reconstruct_delaunay` и почему? Что делается с точками с одинаковой проекцией?
6. Почему обрезка по плотности в `reconstruct_poisson` делает сетку незамкнутой и чем это важно для ТТК?

Ответы

1. Звезда — симплексы, содержащие v ; замкнутая звезда — они вместе с гранями; линк — симплексы замкнутой звезды без v . `neighbors_and_link()` хранит вершины линка (соседей) и рёбра линка.

2. Внутренняя — линк связный цикл; граничная — простой путь; неманифолдная — несколько компонент или вершина степени больше 2 в графе линка.
3. Чтобы у всех внутренних вершин было одинаково 6 соседей и классификация не зависела от случайного выбора диагонали.
4. $E = 5 \cdot 6 + 7 \cdot 4 + 4 \cdot 6 = 82$, граничных $2 \cdot 4 + 2 \cdot 6 = 20$ (проверено: `len(edge_counts) == 82`, `len(boundary_edges) == 20`). Периодическая: $E = 3 \cdot 35 = 105$, граничных 0.
5. Всё, что не является графиком над плоскостью xy : нависания, вертикальные стенки, обратные стороны. Точки с точно равной проекцией сливаются в одну по покоординатной медиане.
6. Удаление вершин вместе с инцидентными треугольниками создаёт рёбра, лежащие в одном треугольнике. ТТК-бэкенд требует замкнутую сетку, поэтому для него нужен `--poisson-density-quantile 0`.

2.13 Что почитать

- Edelsbrunner H., Harer J. *Computational Topology: An Introduction*. AMS, 2010 — гл. I–III (комплексы, звёзды и линки, триангуляции).
- de Berg M., Cheong O., van Kreveld M., Overmars M. *Computational Geometry: Algorithms and Applications*. Springer, 2008 — гл. 9 (триангуляция Делоне).
- Kazhdan M., Bolitho M., Hoppe H. *Poisson Surface Reconstruction*. Eurographics SGP, 2006.

3. Эйлерова характеристика

Эйлерова характеристика — одно число, которое связывает комбинаторику сетки (сколько вершин, рёбер, граней), её топологию (сколько «ручек» и краёв) и количество критических точек любой функции на ней. В проекте это главный встроенный тест корректности: если найденные минимумы, седла и максимумы не сходятся с χ , дескриптор не строится.

3.1 Определение и вычисление

Определение. Для конечного симплициального комплекса K с V вершинами, E рёбрами и F треугольниками эйлерова характеристика (Euler characteristic)

$$\chi(K) = V - E + F.$$

Примеры. Тетраэдр: $4 - 6 + 4 = 2$. Октаэдр: $6 - 12 + 8 = 2$ (проверено построением `TriMesh` из 6 вершин и 8 граней: `euler_characteristic == 2, closed == True`). Один треугольник: $3 - 3 + 1 = 1$.

В коде. `mesh.py`:

```
@cached_property
def euler_characteristic(self) -> int:
    return len(self.vertices) - len(self.edge_counts) + len(self.faces)
```

V — все строки `vertices`, включая вершины, не входящие ни в один треугольник; E — число различных неупорядоченных рёбер (ключей `edge_counts`, гл. 2.5); F — число строк `faces`. Повторяющиеся треугольники и неиспользуемые вершины искажают результат (см. «Типичная ошибка» в гл. 2.9).

3.2 Инвариантность

Главное свойство: χ не зависит от того, как поверхность разбита на треугольники, — только от самой поверхности. Доказательство требует гомологий (гл. 8), но интуицию дают элементарные операции подразбиения:

Операция	ΔV	ΔE	ΔF	$\Delta \chi$
новая вершина внутри треугольника, соединённая с тремя углами	+1	+3	+2	0
новая вершина на внутреннем ребре, соединённая с двумя противоположными вершинами	+1	+3	+2	0
перекидывание (flip) диагонали в паре треугольников	0	0	0	0

Любые две триангуляции одной поверхности связаны последовательностью таких операций и обратных к ним, поэтому χ сохраняется. Практически это значит, что сетки 4×4 и 256×256 одного и того же скана дают одно и то же χ , хотя V, E, F отличаются в тысячи раз.

3.3 Классификация компактных поверхностей

Связная компактная ориентируемая поверхность определяется с точностью до гомеоморфизма двумя числами: родом g (genus, число «ручек») и числом b граничных компонент (краёв). Эйлера характеристика

$$\chi = 2 - 2g - b.$$

Поверхность	g	b	χ	Как получается в проекте
сфера	0	0	2	Пуассон с <code>density_quantile=0</code> у замкнутого объекта без ручек
тор	1	0	0	<code>periodic_grid_mesh</code>
диск	0	1	1	<code>grid_mesh</code> , <code>reconstruct_delaunay</code>
кольцо (annulus)	0	2	0	скан с одной дырой внутри
крендель (двойной тор)	2	0	-2	—

Для несвязной поверхности χ складывается по компонентам. Пример кольца проверен кодом: из `grid_mesh` 9×9 удалены 8 треугольников двух центральных клеток по каждой оси, получилось $V = 80$, $E = 200$, $F = 120$, $\chi = 0$ и 40 граничных рёбер (внешний край 32 и внутренний 8).

3.4 χ сеток проекта

Из подсчёта в гл. 2.7 для `grid_mesh` $n \times m$:

$$\chi = nm - [n(m-1) + m(n-1) + (n-1)(m-1)] + 2(n-1)(m-1) = 1,$$

а для `periodic_grid_mesh`: $\chi = nm - 3nm + 2nm = 0$. Проверка запуском:

Сетка	V	E	F	χ	closed
<code>grid_mesh</code> 3×3	9	16	8	1	False
<code>grid_mesh</code> 5×7	35	82	48	1	False
<code>grid_mesh</code> 10×10	100	261	162	1	False
<code>periodic_grid_mesh</code> 3×3	9	27	18	0	True
<code>periodic_grid_mesh</code> 5×7	35	105	70	0	True
<code>periodic_grid_mesh</code> 10×10	100	300	200	0	True

```
for n, m in [(3, 3), (5, 7), (10, 10)]:
    g, p = grid_mesh(np.zeros((n, m))), periodic_grid_mesh(np.zeros((n, m)))
    print(g.euler_characteristic, p.euler_characteristic)  # 1 0
```

3.5 Связь с критическими точками

Для функции Морса f на замкнутой поверхности M выполняется

$$\# \min - \# \text{saddle} + \# \max = \chi(M).$$

Здесь эта формула только формулируется; гладкое обоснование (каждая критическая точка индекса k приклеивает k -мерную клетку) дано в гл. 4, кусочно-линейная версия — в гл. 5. Интуиция: на сфере у вы-

соты один минимум и один максимум ($1 - 0 + 1 = 2$); на «бублике», стоящем на ребре, добавляются два седла ($1 - 2 + 1 = 0$).

Вершина, у которой нижний линк имеет $k + 1$ компонент, — мульти-седло (multi-saddle) кратности k ; в формуле она считается k раз. Обезьянье седло $x^3 - 3xy^2$ имеет три «долины» и три «гребня», кратность 2: в `grid_mesh 5×5` центральная вершина получает `lower_components = 3`, `upper_components = 3`, `saddle_multiplicity = 2`. Сдвиг функции относительно узлов сетки (например, $x \rightarrow x - 0.3$, то есть конечное изменение значений в вершинах) разваливает его на два простых седла (проверено: те же 2 в сумме). Бесконечно малое шевеление значений этого не делает (гл. 5.5).

В коде. `critical.py`, `analyze_morse()` записывает `saddle_multiplicity = max(lower_count, upper_count) - 1`. `descriptor.py`:

```
@property
def saddle_index_sum(self) -> int:
    return sum(max(point.saddle_multiplicity, 1) for point in self.critical_points
               if point.kind == CriticalType.SADDLE)

def validate_euler(descriptor: Descriptor) -> None:
    counts = descriptor.counts
    actual = counts["max"] - descriptor.saddle_index_sum + counts["min"]
    if actual != descriptor.euler_characteristic:
        raise ValueError(f"Morse Euler invariant failed: {actual} != {descriptor.euler_characteristic}")
```

`build_descriptor()` вызывает проверку, только если передан `euler_characteristic`; `build_ttk_descriptor()` вызывает её всегда. Заметим, что `counts["saddle"]` считает седла без кратности, а в проверке используется взвешенная сумма.

3.6 Почему на открытой поверхности проверки нет

В коде. `cli.py`, `main()`: `expected_euler = mesh.euler_characteristic if mesh.closed else None`.

Для поверхности с краем формула 3.5 в простом виде неверна: функция может «уходить» через край, не создавая внутренних критических точек, а граничные вершины по умолчанию маскируются (`include_boundary=False`). Примеры на `grid_mesh 9×9` ($\chi = 1$), только внутренние вершины:

Функция	min	saddle	max	min – saddle + max
наклонная плоскость $x + 0.3y$	0	0	0	$0 \neq 1$
купол $e^{-3(x^2 + y^2)}$	0	0	1	$1 = 1$ (случайно)
седло $x^2 - y^2$	0	1	0	$-1 \neq 1$

`build_descriptor(mesh, euler_characteristic=1)` на наклонной плоскости падает с `Morse Euler invariant failed: 0 != 1`. Включение границы (`include_boundary=True`) не спасает: для плоскости появляются минимум и максимум в углах, сумма $2 \neq 1$, потому что граничную вершину с линком-путём нужно классифицировать по другим правилам. Правильная граничная версия теории разобрана в гл. 10.

3.7 Персистентность сохраняет баланс

Шум порождает лишние критические точки, но всегда парами соседних индексов: минимум с седлом или седло с максимумом. Пара вносит в $\#min - \#saddle + \#max$ вклад $(+1) + (-1) = 0$, поэтому удаление любой пары сохраняет χ .

В коде. `persistence.py`, `persistent_counts()`: для каждой пары из `critical_pairs()` с персистентностью не ниже порога обе вершины получают «попадание»; седло учитывается столько раз, сколько у него значимых пар (мульти-седло кратности k участвует в k парах), экстремум — один раз ($\min(\text{hits}, 1)$). Бесконечные (существенные) классы на торе — глобальный минимум (размерность 0), два седла циклов тора (размерность 1) и глобальный максимум (размерность 2) — проходят любой порог и дают $1 - 2 + 1 = 0$. Функция только фильтрует счётчики: сепаратрисы и граф не перестраиваются, это не настоящее сокращение комплекса Морса–Смейла (гл. 6, 8).

3.8 Пример: тор и $\cos x + \cos y$

Функция $f(x, y) = \cos x + \cos y$ периодична с периодом 2π по обеим осям, то есть задана на торе. Её критические точки: максимум $(0, 0)$, $f = 2$; минимум (π, π) , $f = -2$; седла $(\pi, 0)$ и $(0, \pi)$, $f = 0$.

```
N = 32
x = np.linspace(0, 2 * np.pi, N, endpoint=False)
X, Y = np.meshgrid(x, x)
torus = periodic_grid_mesh(np.cos(X) + np.cos(Y), spacing=(2 * np.pi / N,) * 2)
points = analyze_morse(torus)
descriptor = build_descriptor(torus, euler_characteristic=torus.euler_characteristic)
print(descriptor.counts, descriptor.saddle_index_sum, torus.euler_characteristic)
# {'min': 1, 'regular': 1020, 'saddle': 2, 'max': 1} 2 0
```

Вершины: максимум 0 в $(0, 0)$, седла 16 в $(\pi, 0)$ и 512 в $(0, \pi)$, минимум 528 в (π, π) ; $1 - 2 + 1 = 0$, `validate_euler` проходит. Значения f вдоль линий симметрии многократно совпадают, и ничьи разрешает симуляция простоты (SoS) в `TriMesh.order` (гл. 5).

С шумом $\sigma = 0.05$ классификатор находит 7 минимумов, 13 сёдел и 6 максимумов: $7 - 13 + 6 = 0$, инвариант по-прежнему выполняется. После `persistent_counts` с порогом 0.1 остаётся 1, 2, 1.

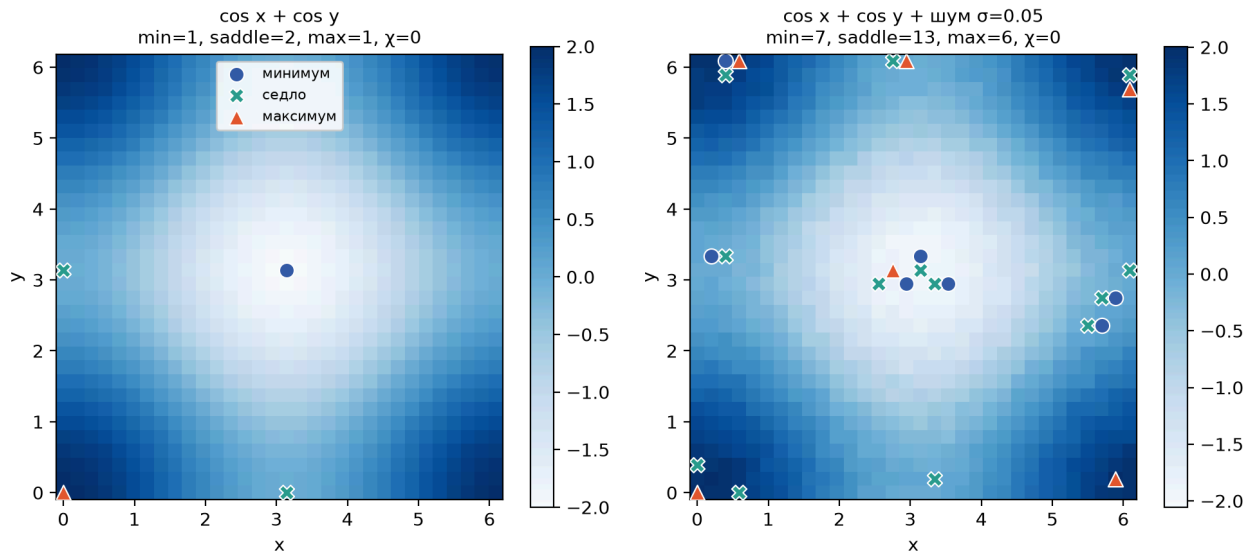


Рис. 3.1. Тор `periodic_grid_mesh` 32×32 с функцией $\cos x + \cos y$: критические точки из `analyze_morse()` без шума (слева) и с шумом (справа). Синие — минимумы, зелёные — седла, красные — максимумы.

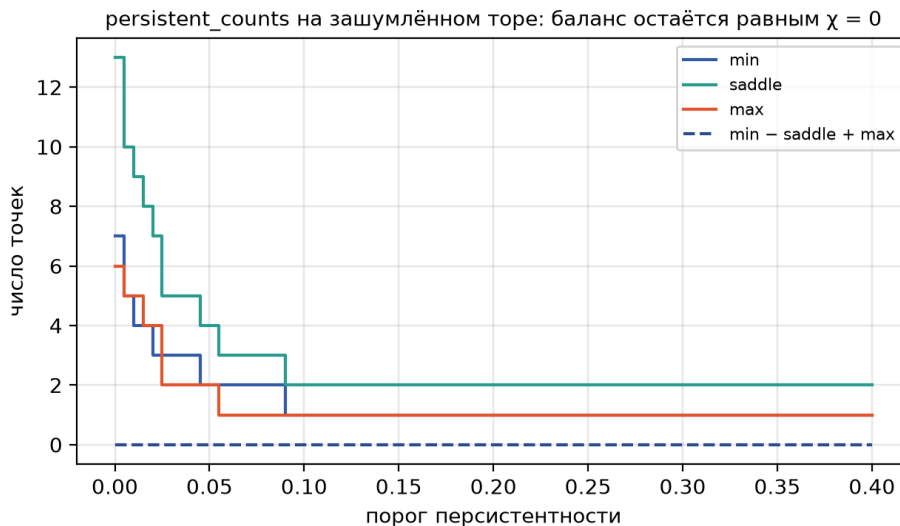


Рис. 3.2. `persistent_counts()` на зашумлённом торе в зависимости от порога: счётчики падают до 1, 2, 1, а знакопеременная сумма всё время равна $\chi = 0$.

3.9 Вопросы для самопроверки

1. Чему равна χ у `grid_mesh` 100×200 и у `periodic_grid_mesh` того же размера? Почему не нужно считать?
2. Почему вставка вершины в треугольник не меняет χ ?
3. Как `validate_euler` учитывает обезьянье седло? Чем `counts["saddle"]` отличается от `saddle_index_sum`?
4. Почему в `cli.py` для открытой поверхности `expected_euler = None`? Приведите функцию, на которой проверка упала бы.
5. Почему фильтрация по персистентности сохраняет $\#min - \#saddle + \#max$?
6. Чему равна χ сетки с изолированной вершиной и что при этом покажет `analyze_morse`?

Ответы

1. 1 и 0: это диск и тор, χ не зависит от разбиения (формулы 3.4).
2. $\Delta V = 1$, $\Delta E = 3$, $\Delta F = 3 - 1 = 2$, итого $1 - 3 + 2 = 0$.
3. Кратность $k = \max(\text{lower}, \text{upper}) - 1 = 2$, седло входит в `saddle_index_sum` дважды; `counts` считает его один раз.
4. Формула верна только для замкнутых поверхностей, а граница маскируется. Наклонная плоскость: $0 \neq 1$.
5. Значимые пары содержат точки соседних индексов, вклад пары $+1 - 1 = 0$; существенные классы дают ровно χ .
6. На 1 больше, чем у той же сетки без неё; изолированная вершина будет классифицирована как минимум (пустой нижний линк), и сумма останется согласованной с искажённой χ , скрывая ошибку.

3.10 Что почитать

- Edelsbrunner H., Harer J. *Computational Topology: An Introduction*. AMS, 2010 — гл. II (классификация поверхностей, эйлерова характеристика).
- Милнор Дж. *Теория Морса*. М.: Мир, 1965 — §3 (гомотопический тип через критические значения) и §5 (неравенства Морса).

4. Гладкая теория Морса

Эта глава даёт гладкую (непрерывную) теорию, из которой выросли все алгоритмы проекта. В коде нет ни производных, ни гессианов: там только вершины, рёбра и треугольники (гл. 5). Но каждое дискретное правило в `critical.py`, `persistence.py` и `separatrices.py` повторяет какую-то гладкую теорему. Если знать эту теорему, легко понять, что код обязан выдавать, и заметить, когда он выдаёт не то.

4.1 Функция на поверхности

Представим рельеф местности. Для каждой точки поверхности M известна высота $f(p)$. В проекте M — это поверхность, восстановленная по LiDAR-скану, а f — глубина, высота или другое скалярное поле (гл. 9). Формально рассматривается гладкая функция $f: M \rightarrow \mathbb{R}$ на двумерной поверхности M . Поверхность может быть замкнутой (сфера, тор) или иметь край (кусок рельефа; гл. 10).

Простейший пример — график $z = f(x, y)$ над прямоугольником. Второй пример понадобится нам постоянно: функция на торе. Тор можно поставить «на ребро» и взять высоту $f = z$. Можно поступить и иначе: взять периодическую по обеим координатам функцию, например $f(x, y) = \sin(2\pi x/W) + 0.4\cos(2\pi y/H)$, и отождествить противоположные стороны прямоугольника $W \times H$. Именно так устроены тестовые сетки `periodic_grid_mesh` в `tests/test_critical.py`.

4.2 Градиент и критические точки

Интуиция такая. В обычной точке склона есть направление наибольшего подъёма, а линия уровня проходит поперёк этого направления. На дне ямы, на вершине холма и на перевале такого направления нет: в первом приближении поверхность там горизонтальна.

В локальных координатах (x, y) **градиент (gradient)** функции — это вектор $\nabla f = (\partial f/\partial x, \partial f/\partial y)$. Точка p называется **критической (critical point)**, если $\nabla f(p) = \mathbf{0}$. Число $f(p)$ в такой точке называется **критическим значением (critical value)**. Остальные точки и значения называются регулярными.

Чтобы различать критические точки между собой, используют вторые производные, собранные в **гессиан (Hessian)**:

$$H_f(p) = \begin{pmatrix} f_{xx} & f_{xy} \\ f_{xy} & f_{yy} \end{pmatrix}.$$

Критическая точка **невырождена (non-degenerate)**, если $\det H_f(p) \neq 0$. Матрица H симметрична, поэтому у неё два вещественных собственных значения (гл. 1). **Индекс (index)** невырожденной критической точки равен числу отрицательных собственных значений, то есть числу независимых направлений, по которым функция убывает.

Модель	$H_f(0)$	Собственные значения	Индекс	Тип
$x^2 + y^2$	$\text{diag}(2, 2)$	2, 2	0	минимум
$x^2 - y^2$	$\text{diag}(2, -2)$	2, -2	1	седло
$-x^2 - y^2$	$\text{diag}(-2, -2)$	-2, -2	2	максимум

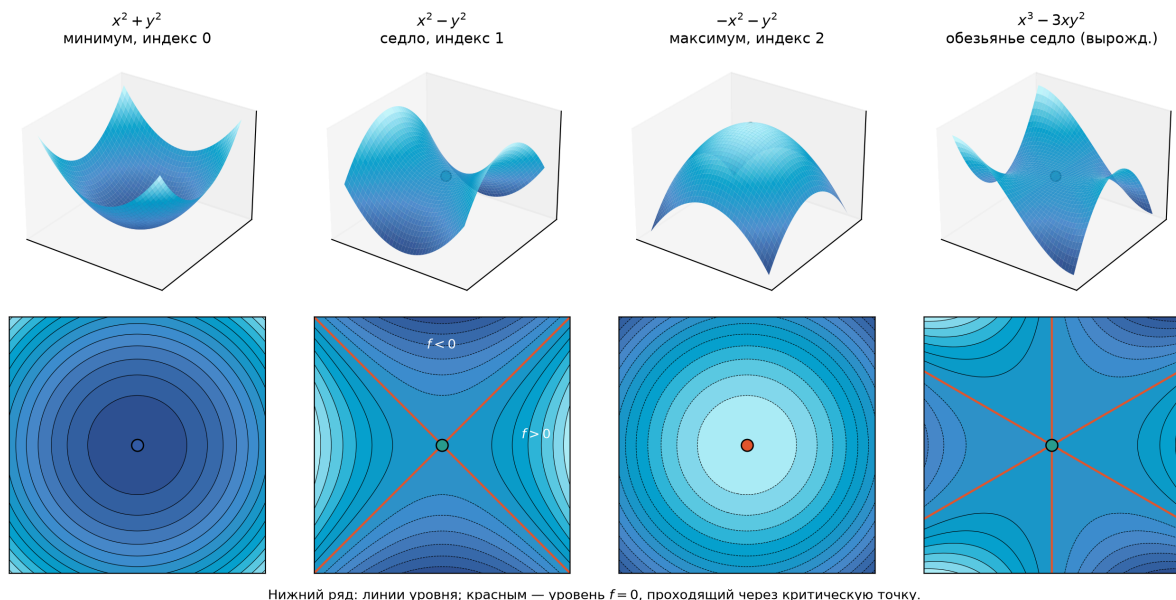


Рис. 4.1. Три модели невырожденных критических точек и вырожденное обезьянье седло. Верхний ряд — графики, нижний — линии уровня. Через седло проходят две линии уровня $f = 0$, через обезьянье седло — три.

Пример с числами. Пусть $f(x, y) = x^2 + 3xy + y^2$. Тогда $\nabla f = (2x + 3y, 3x + 2y)$ обращается в нуль только в начале координат, а $H = \begin{pmatrix} 2 & 3 \\ 3 & 2 \end{pmatrix}$ имеет собственные значения 5 и -1 . Значит, это седло индекса 1, хотя по формуле это заметить нелегко. Знак определителя даёт быстрый тест: $\det H < 0$ означает седло, $\det H > 0$ — экстремум (минимум, если $f_{xx} > 0$).

В коде. Индекс в коде нигде не хранится явно, вместо него используется тип: `critical.py`, перечисление `CriticalType` со значениями `MINIMUM = "min"`, `SADDLE = "saddle"`, `MAXIMUM = "max"` и `REGULAR`. Индексы 0, 1, 2 соответствуют min, saddle, max.

4.3 Лемма Морса

Сначала смысл. Возле невырожденной критической точки функцию можно «распрямить» гладкой заменой координат: при этом не важно, как функция выглядит в исходных координатах. В результате останется одна из трёх моделей таблицы. Других вариантов локального поведения у невырожденной точки нет.

Лемма Морса (Morse lemma). Пусть p — невырожденная критическая точка индекса λ . Тогда в некоторой окрестности p существуют локальные координаты (u, v) с $u(p) = v(p) = 0$, в которых

$$f = f(p) \pm u^2 \pm v^2,$$

причём минусов ровно λ .

Отсюда два следствия. Во-первых, невырожденные критические точки изолированы: у модели $\pm u^2 \pm v^2$ других критических точек рядом нет. Если невырождены все критические точки, то на компактной поверхности их конечное число: множество критических точек замкнуто (значит, компактно) и дискретно. Во-вторых, тип точки виден по её окрестности: у минимума все соседние точки выше, у максимума все ниже, а у седла при обходе маленькой окружности вокруг p знак разности $f - f(p)$ меняется четыре раза $(+, -, +, -)$. Именно последний признак, а не гессиан, перенесён в дискретный классификатор гл. 5.

4.4 Вырожденные точки и обезьянье седло

Рассмотрим $f(x, y) = x^3 - 3xy^2$, **обезьянье седло (monkey saddle)**; название объясняется тем, что обезьяне нужно место для двух ног и хвоста. Здесь $\nabla f = (3x^2 - 3y^2, -6xy)$ обращается в нуль только в начале координат, а все вторые производные там равны нулю, поэтому $H_f(0) = 0$ и точка вырождена. Линия

уровня $f = 0$ состоит из трёх прямых $x = 0$ и $y = \pm x/\sqrt{3}$. При обходе окружности знак f меняется шесть раз (правый столбец рисунка выше): вокруг точки три «долины» и три «ребра».

Почему такое седло «стоит двух сёдел»? Слегка пошевелим функцию: $f_\varepsilon = x^3 - 3xy^2 + \varepsilon x$, $\varepsilon > 0$. Условие $\partial f_\varepsilon / \partial y = -6xy = 0$ даёт $x = 0$ или $y = 0$. При $y = 0$ уравнение $3x^2 + \varepsilon = 0$ решений не имеет, а при $x = 0$ получаем $y = \pm\sqrt{\varepsilon/3}$. Итак, критических точек две, и в каждой $\det H = \det \begin{pmatrix} 0 & -6y \\ -6y & 0 \end{pmatrix} = -36y^2 < 0$, то есть это два обычных седла. Одна вырожденная точка распалась на два невырожденных седла. В сумме Эйлера из п. 4.7 она поэтому даёт вклад $2 \cdot (-1)^1 = -2$. Обобщённо обезьянье седло называют **мультиседлом (multi-saddle)** кратности 2.

В коде. Кратность хранится в поле `CriticalPoint.saddle_multiplicity` (`critical.py`). Свойство `Descriptor.saddle_index_sum` (`descriptor.py`) суммирует седла с весом кратности: в docstring сказано «a monkey saddle counts twice». Как кратность вычисляется на сетке, разобрано в гл. 5.

4.5 Функции Морса типичны

Функция называется **функцией Морса (Morse function)**, если все её критические точки невырождены. Можно дополнительно потребовать, чтобы все критические значения были различны. Оба требования выполняются «почти всегда»: функции Морса образуют открытое и всюду плотное множество среди гладких функций на компактной поверхности. На практике это значит, что сколь угодно малое шевеление (как $+\varepsilon x$ выше) делает функцию морсовской, а морсовская функция остаётся морсовской при достаточно малом шевелении.

Для данных отсюда следуют два вывода. Первый: функции с вырождениями или равными критическими значениями образуют нигде не плотное множество, и из него можно выйти сколь угодно малым возмущением. Второй: в реальном LiDAR-скане такие ситуации возникают постоянно из-за квантования глубины (плато из одинаковых миллиметров). Бесконечно малое возмущение, которое выполняется символически, без изменения чисел, называется **симуляцией простоты (simulation of simplicity, SoS)**; ей посвящён п. 5.5.

4.6 Подуровневые множества и главная теорема

Представим, что рельеф заливают вода, и её уровень a медленно растёт. Затопленная часть поверхности — это **подуровневое множество (sublevel set)**

$$M_a = \{p \in M: f(p) \leq a\}.$$

Пока уровень поднимается по склонам, затопленная область только расширяется, и её форма (число озёр и число островов внутри них) не меняется. Перемены происходят ровно в моменты, когда вода доходит до критической точки.

Теорема А. Если на отрезке $[a, b]$ нет критических значений (и $f^{-1}[a, b]$ компактно), то M_a и M_b диффеоморфны; более того, M_b стягивается на M_a вдоль градиента.

Теорема В. Пусть на отрезке $[c - \varepsilon, c + \varepsilon]$ лежит одно критическое значение c с одной невырожденной точкой индекса λ (и $f^{-1}[c - \varepsilon, c + \varepsilon]$ компактно). Тогда $M_{c+\varepsilon}$ получается из $M_{c-\varepsilon}$ приклеиванием **ручки (handle)** индекса λ , то есть с точностью до гомотопии — клетки размерности λ .

На поверхности это даёт три случая.

- **Минимум ($\lambda = 0$):** приклеивается диск отдельно от всего остального, и рождается новая компонента связности (новое озеро).
- **Седло ($\lambda = 1$):** приклеивается полоска, два конца которой лежат на краю $M_{c-\varepsilon}$. Если концы принадлежат разным компонентам, они сливаются (два озера соединились через перевал). Если одной и той же, появляется новая петля, то есть одномерная дыра.
- **Максимум ($\lambda = 2$):** приклеивается диск по всей граничной окружности. Он закрывает дыру (последний остров уходит под воду), а на замкнутой поверхности последний максимум закрывает её целиком.

Подуровневые множества $M_a = \{f \leq a\}$ для $f = (x^2 - 1)^2 + y^2 + 0.3x$; минимумы $f = -0.31$ и 0.29 , седло $f = 1.01$

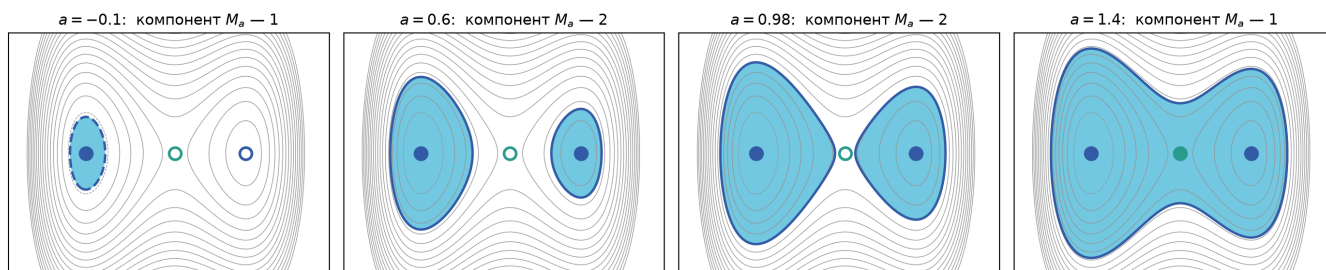


Рис. 4.2. Подуровневые множества рельефа $f = (x^2 - 1)^2 + y^2 + 0.3x$ (два минимума разной глубины и седло между ними). Закрашенные кружки — критические точки, уже оказавшиеся под водой. Число компонент посчитано по сетке (`scipy.ndimage.label`).

Пример с числами (рисунок выше). Критические точки лежат на оси $y = 0$, где $\partial f / \partial x = 4x(x^2 - 1) + 0.3 = 0$. Это минимумы $x \approx -1.036$ ($f \approx -0.305$) и $x \approx 0.960$ ($f \approx 0.294$) и седло $x \approx 0.075$ ($f \approx 1.011$). При $a = -0.1$ затоплен только левый минимум, и компонента одна. При $a = 0.6$ родилась вторая компонента. При $a = 0.98$ компонент всё ещё две, хотя они почти касаются возле седла. При $a = 1.4$ уровень прошёл седло, и компоненты слились. Число компонент изменилось ровно в трёх критических значениях и нигде больше.

Тот же процесс на торе с высотой $f = z$ (большой радиус $R = 2$, малый $r = 0.8$) проходит через четыре критические точки: минимум $z = -2.8$, седло $z = -1.2$ (низ внутренней дыры), седло $z = 1.2$ (верх дыры) и максимум $z = 2.8$. Последовательность превращений: пусто → диск → цилиндр (первое седло приклеило полосу к диску) → тор без диска (второе седло дало вторую петлю) → тор (максимум заклеил дыру).

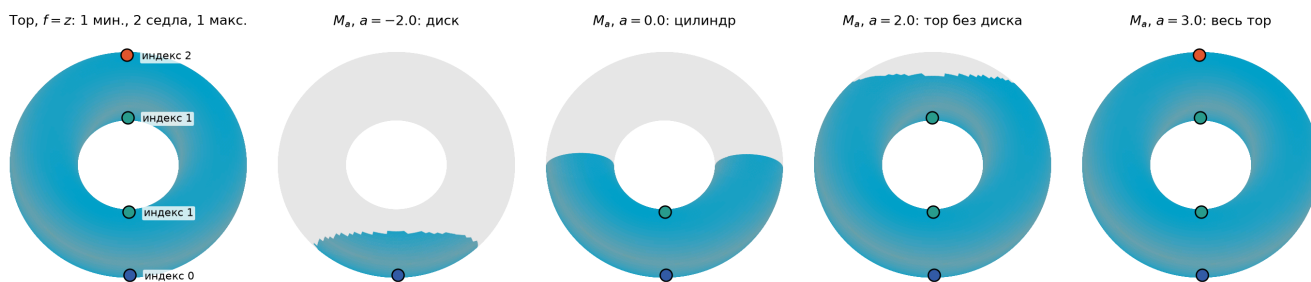


Рис. 4.3. Тор с функцией высоты: критические точки найдены функцией `analyze_morse` на триангуляции тора из 10 240 вершин, найдено ровно 1 минимум, 2 седла и 1 максимум. Справа — подуровневые множества M_a (бирюзовым).

Важно. Это фундамент персистентности (гл. 8). Каждый минимум рождает компоненту, и каждое седло либо убивает компоненту (сливает две), либо рождает петлю; каждый максимум, кроме последнего, убивает петлю; на замкнутой поверхности последний максимум рождает класс размерности 2 (бесконечная пара). Если записать пары «кто родил — кто убил», получится диаграмма персистентности. В коде это `persistence.py`, `critical_pairs()`: в нулевой размерности пара — это минимум и седло, в первой — седло и максимум (см. `docstring` класса `CriticalPair`).

4.7 Неравенства Морса и эйлерова характеристика

Обозначим через c_λ число критических точек индекса λ , а через b_λ — **числа Бетти (Betti numbers)** поверхности: b_0 — число компонент, b_1 — число независимых петель, b_2 — число замкнутых компонент (для ориентируемых поверхностей; у сферы 1, 0, 1, у тора 1, 2, 1; подробнее в гл. 8). Из теоремы В следует, что поверхность гомотопически эквивалентна клеточному комплексу из c_0 0-клеток, c_1 1-клеток и c_2 2-клеток. Отсюда получаются следующие соотношения.

Слабые неравенства Морса: $c_\lambda \geq b_\lambda$. Функция на торе не может иметь меньше двух сёдел, потому что каждая из двух независимых петель тора должна где-то родиться.

Формула Эйлера:

$$\sum_{p \text{ критич.}} (-1)^{\text{ind} p} = c_0 - c_1 + c_2 = \chi(M).$$

Интуиция: эйлерова характеристика клеточного комплекса равна #вершин – #рёбер + #граней (гл. 3), а клетки здесь как раз и соответствуют критическим точкам. Сфера: $1 - 0 + 1 = 2$. Тор с высотой: $1 - 2 + 1 = 0$. Если на торе 40 минимумов и 40 максимумов, то седел обязательно ровно 80. Вырожденные точки подставляются со своим вкладом: обезьянье седло даёт -2 .

В коде. `descriptor.py`, функция `validate_euler()`: проверяется `counts["max"] - descriptor.saddle_index_sum + counts["min"] == euler_characteristic`, и при нарушении бросается `ValueError`. Это «предохранитель» на ошибки классификатора: на замкнутой сетке равенство обязано выполняться всегда. Функция `build_descriptor` вызывает проверку, только если ей передан `euler_characteristic` (по умолчанию `None`). `CLI (cli.py)` передаёт его лишь для замкнутых сеток: `expected_euler = mesh.euler_characteristic if mesh.closed else None`. Почему так — в гл. 10.

4.8 Градиентный поток, интегральные линии, устойчивые многообразия

Вернёмся к картинке с водой. Капля, пущенная по склону, катится по направлению $-\nabla f$, пока не остановится в минимуме. Её траектория — **интегральная линия (integral line)**, то есть кривая $\gamma(t)$, для которой

$$\frac{d\gamma}{dt} = -\nabla f(\gamma(t)).$$

Свойства (для функции Морса на замкнутой поверхности): через каждую регулярную точку проходит ровно одна интегральная линия; интегральные линии не пересекаются; каждая начинается и заканчивается в критических точках и вдоль неё f строго убывает.

Устойчивое многообразие (stable manifold) точки p — множество $W^s(p)$ всех точек, капля из которых приходит в p при $t \rightarrow +\infty$. **Неустойчивое многообразие (unstable manifold)** $W^u(p)$ — точки, приходящие в p при $t \rightarrow -\infty$ (капля из них «вытекла» из p). Размерности: $\dim W^u(p) = \text{ind} p$, $\dim W^s(p) = 2 - \text{ind} p$.

Точка	$W^s(p)$	$W^u(p)$
минимум	двумерный «водосборный бассейн»	сама точка
седло	две дуги, приходящие сверху	две дуги, уходящие вниз к минимумам
максимум	сама точка	двумерная «шапка» вокруг вершины

Четыре дуги, выходящие из седла, — это **сепаратрисы (separatrix)**. Они делят поверхность на ячейки: бассейны минимумов разделены сепаратрисами, ведущими к максимумам, и наоборот. Если устойчивые и неустойчивые многообразия пересекаются трансверсально (на поверхности это означает, что ни одна интегральная линия не идёт из седла в седло), функция называется функцией Морса–Смейла, а разбиение — комплексом Морса–Смейла (гл. 6). Каждая его ячейка — четырёхугольник «минимум — седло — максимум — седло».

В коде. `separatrices.py`, функция `trace_separatrices()`: для каждого седла из каждой компоненты нижнего линка проводится дуга вида "u" вниз до минимума, а из каждой компоненты верхнего — дуга "s" вверх до максимума. Обозначения согласуются с таблицей: при потоке $-\nabla f$ дуги к минимумам лежат в W^u седла, а дуги к максимумам — в W^s . У простого седла получается ровно по две дуги каждого вида (тест `test_simple_saddles_emit_exactly_two_arcs_each_way`). Дуги строятся монотонным шагом по соседям, а не интегрированием настоящего потока. Вопреки слову «steepest» в docstring, на каждом шаге выбирается сосед, ближайший по рангу (самый пологий монотонный шаг, а не крутейший), поэтому настоящим комплексом Морса–Смейла эти дуги не являются (гл. 6).

4.9 Вопросы для самопроверки

1. У функции $f = x^2 + 4xy + y^2$ в начале координат критическая точка. Какого она индекса?
2. Почему у функции Морса на компактной поверхности конечное число критических точек?
3. Сколько раз меняется знак $f - f(p)$ на маленькой окружности вокруг минимума, простого седла, обезьяньего седла?
4. Функция на торе имеет 5 минимумов и 3 максимума, все точки невырождены. Сколько у неё седел?
5. Что происходит с M_a при прохождении седла, и чем отличаются два возможных варианта?
6. Какую размерность имеют W^s и W^u у седла и почему из седла выходят ровно четыре сепаратрисы?

Ответы

1. $H = \begin{pmatrix} 2 & 4 \\ 4 & 2 \end{pmatrix}$, $\det H = -12 < 0$, собственные значения 6 и -2 : седло, индекс 1.
2. По лемме Морса каждая критическая точка изолирована. Множество критических точек $\{\nabla f = 0\}$ замкнуто в компакте, значит компактно; компактное множество из изолированных точек конечно (из его покрытия такими окрестностями выбирается конечное подпокрытие, а в каждой окрестности одна точка).
3. 0, 4 и 6 раз. Число «долин» при этом 0, 2, 3, а кратность седла на единицу меньше числа долин.
4. $c_0 - c_1 + c_2 = 0$, поэтому $c_1 = 5 + 3 = 8$.
5. Приклеивается полоска (1-ручка). Если её концы на разных компонентах, компоненты сливаются (b_0 падает на 1). Если на одной, рождается новая петля (b_1 растёт на 1).
6. Обе одномерны ($2 - 1 = 1$ и 1). Каждое состоит из двух дуг (по обе стороны от седла в модели $u^2 - v^2$): две вниз и две вверх, итого четыре.

4.10 Что почитать

- Милнор Дж. *Теория Морса*. — М.: Мир, 1965 (J. Milnor, *Morse Theory*, Princeton, 1963). Часть I: лемма Морса, теоремы А и В, неравенства Морса.
- Н. Edelsbrunner, J. Harer. *Computational Topology: An Introduction*. AMS, 2010. Глава VI (Morse functions) связывает гладкую теорию с алгоритмами.

5. Критические точки на триангулированных поверхностях

В гл. 4 критическая точка определялась через производные. У сетки из LiDAR-скана производных нет: есть вершины с числами и треугольники между ними. В этой главе гладкие определения переводятся на язык сетки так, чтобы сохранились лемма Морса (п. 4.3) и формула Эйлера (п. 4.7). Затем разбирается код `critical.py` и `mesh.py` и показывается, что происходит на шумных и квантованных данных.

5.1 Кусочно-линейная функция

Сетка задаётся классом `TriMesh` (`mesh.py`): массив вершин `vertices` формы $(n, 3)$, треугольники `faces` формы $(m, 3)$ и одно число на вершину `values`. Значение внутри треугольника получается **линейной интерполяцией**: если точка p имеет барицентрические координаты $(\lambda_0, \lambda_1, \lambda_2)$ относительно вершин v_0, v_1, v_2 (гл. 2), то

$$f(p) = \lambda_0 f(v_0) + \lambda_1 f(v_1) + \lambda_2 f(v_2), \quad \lambda_i \geq 0, \quad \sum \lambda_i = 1.$$

Такая функция называется **кусочно-линейной (piecewise linear, PL)**. Её график — многогранник, склеенный из плоских треугольников. Внутри треугольника градиент постоянен; если значения в вершинах различны, он ненулевой, так что критических точек внутри треугольников нет. На рёбрах и в вершинах градиент скачет и не определён. Однако точка внутри ребра с разными значениями на концах регулярна: функция растёт вдоль ребра, и вокруг такой точки ровно одна «нижняя» и одна «верхняя» сторона. Поэтому искать критические точки имеет смысл только в вершинах, и нужен признак, не использующий производных.

Такой признак уже встречался в п. 4.3: тип точки определяется тем, как меняется знак $f - f(p)$ на маленькой окружности вокруг неё. На сетке роль этой окружности играет линк вершины.

5.2 Звезда, линк, нижний и верхний линк

Звезда (star) вершины v — все треугольники, содержащие v . **Линк (link)** $Lk(v)$ — рёбра этих треугольников, противоположные v , вместе с их концами (соседями v). Для внутренней вершины поверхности линк — замкнутый цикл, «окружность» вокруг v . Для граничной вершины это незамкнутый путь.

Нижний линк (lower link) $Lk^-(v)$ — часть линка, натянутая на соседей с меньшим значением: соседи u с $f(u) < f(v)$ и рёбра линка, оба конца которых такие. **Верхний линк (upper link)** $Lk^+(v)$ определяется так же для соседей со значением больше. Рёбра линка, соединяющие нижнего соседа с верхним, не входят ни в один из них.

Классификация (так её реализует `analyze_morse()`). Пусть L и U — числа компонент связности нижнего и верхнего линка. Тогда

L	U	Тип	Гладкий аналог
0	любое	минимум	все соседи выше
≥ 1	0	максимум	все соседи ниже
1	1	регулярная	склон: одна «нижняя» дуга, одна «верхняя»
≥ 1	≥ 1 , причём $L \geq 2$ или $U \geq 2$	седло	несколько долин и гребней

Кратность седла в коде равна `max(lower_count, upper_count) - 1`.

Эта идея восходит к Т. Банчоффу (Т. Banchoff, 1967). Для поверхностей его определение индекса вершины можно записать как $i(v) = 1 - \frac{1}{2}$ (число перемен знака $f - f(v)$ при обходе линка); теорема Банчоффа

утверждает, что $\sum_v i(v) = \chi(M)$. В виде «компонент нижнего линка» этот признак используется в работе Edelsbrunner, Harer, Zomorodian (2003); тот же признак применяется в ТТК.

Почему у внутренней вершины $L = U$. Обойдём цикл линка и пометим каждого соседа буквой «н» (ниже) или «в» (выше). Если встречаются обе буквы, цикл распадается на максимальные дуги из одинаковых букв, и эти дуги чередуются: н-дуга, в-дуга, н-дуга, ... Цикл замкнут, поэтому после последней в-дуги снова идёт первая н-дуга, и дуг каждого вида поровну: $L = U = k$. Перемен знака при этом $2k$, и индекс Банчоффа равен $i(v) = 1 - k$.

- Минимум и максимум: перемен нет, $i = 1$ (одна буква на весь цикл; в коде $L = 0, U = 1$ или $L = 1, U = 0$).
- Регулярная вершина: $k = 1, i = 0$.
- Простое седло: $k = 2, i = -1$.
- k -кратное седло: $k + 1$ нижних и $k + 1$ верхних компонент, $i = -k$. Обезьянье седло: 3 и 3, $i = -2$.

Значит, формула Банчоффа и проверка `validate_euler()` (п. 4.7) — одно и то же: минимумы и максимумы дают по $+1$, седло — $-(\text{кратность})$. У вершины с d соседями $k \leq d/2$. У сеток `grid_mesh` и `periodic_grid_mesh` каждая внутренняя вершина имеет 6 соседей, поэтому кратность там не больше 2.

Типичная ошибка. Для граничной вершины линк — путь, и равенство $L = U$ не обязано выполняться. Например, на рельефе из п. 5.7 при `include_boundary=True` находятся граничные «сёдла» с $L = 1, U = 2$. Формула $\max(L, U) - 1$ для них даёт кратность, не согласованную с формулой Эйлера. Поэтому граница по умолчанию маскируется (п. 5.8).

5.3 Пример: вершина с шестью соседями

Возьмём внутреннюю вершину сетки `grid_mesh` со значением 5. Её соседи в порядке обхода по кругу: восток (В), северо-восток (СВ), север (С), запад (З), юго-запад (ЮЗ), юг (Ю). Такой порядок задан диагональю, которой `_grid_faces()` режет каждую клетку: треугольники (a, b, d) и (a, d, c) , диагональ идёт с юго-запада на северо-восток. Рёбра линка соединяют соседних по кругу: В–СВ, СВ–С, С–З, З–ЮЗ, ЮЗ–Ю, Ю–В.

Случай	В	СВ	С	З	ЮЗ	Ю	Буквы по кругу	L	U	Тип
а	7	8	6	4	3	2	в в в н н н	1	1	регулярная
б	6	9	7	8	6	7	в в в в в в	0	1	минимум
в	7	6	3	8	2	4	в в н в н н	2	2	седло, кратность 1
г	7	3	8	2	6	4	в н в н в н	3	3	седло, кратность 2

В случае «в» верхний линк — это {В, СВ} и {З} (З не соединён с СВ, их разделяет нижний С), а нижний линк — {С} и {ЮЗ, Ю}. Таблица и рисунок получены запуском кода на «веере» из шести треугольников (функция `fan_mesh()` в `figures/ch05_figures.py`):

```
from ch05_figures import fan_mesh          # запускать из каталога figures/
from morse_lidar import analyze_morse
from morse_lidar.critical import lower_upper_links
mesh = fan_mesh(5, [7, 6, 3, 8, 2, 4])    # центр — вершина 0, соседи 1..6
p = analyze_morse(mesh)[0]                # соседи граничные и замаскированы
print(p.kind.value, p.lower_components, p.upper_components, p.saddle_multiplicity)
# saddle 2 2 1
print(lower_upper_links(mesh, 0))
# ([{3}, {5, 6}], [{1, 2}, {4}])
```

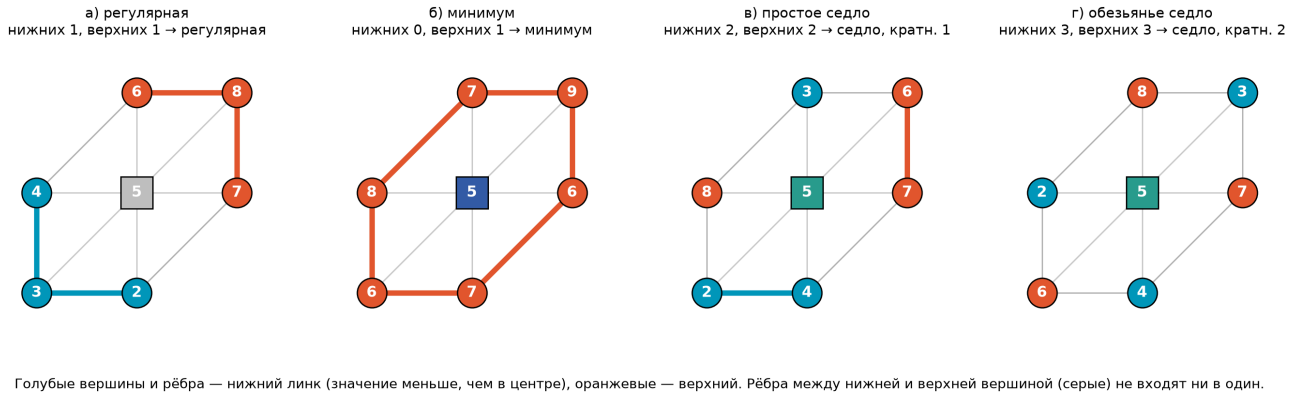


Рис. 5.1. Линк вершины со значением 5 в четырёх случаях. Квадрат в центре окрашен по типу, найденному `analyze_morse()`.

Проверка на гладких моделях. Сэмплируем модели гл. 4 на сетке 5×5 с узлами $x, y \in \{-2, \dots, 2\}$ и построим `grid_mesh`. В центре `analyze_morse` находит: для $x^2 + y^2$ минимум ($L = 0, U = 1$), для $x^2 - y^2$ седло ($2, 2$), для $-x^2 - y^2$ максимум ($1, 0$), для $x^3 - 3xy^2$ седло ($3, 3$) кратности 2. С обезьяньим седлом всё не так просто, см. п. 5.5.

5.4 Код: `link_components`, `lower_upper_links`, `analyze_morse`

Сначала `mesh.py`, метод `neighbors_and_link()`. Для каждого треугольника и каждой его вершины две другие вершины добавляются в соседи, а противоположное ребро — в линк. Результат кэшируется (`cached_property`) и возвращается как кортежи `frozenset`.

Компоненты подмножества линка ищет `critical.py`, `link_components()`:

```
def link_components(vertices, link_edges):
    graph = {vertex: set() for vertex in vertices}
    for left, right in link_edges:
        if left in vertices and right in vertices: # ребро внутри подмножества
            graph[left].add(right)
            graph[right].add(left)
    components, unseen = [], set(vertices)
    while unseen:
        start = unseen.pop()
        component, stack = {start}, [start]
        while stack: # обход в глубину стеком
            current = stack.pop()
            for neighbor in graph[current] & unseen:
                unseen.remove(neighbor)
                component.add(neighbor)
                stack.append(neighbor)
            components.append(component)
    return components
```

Это обычный поиск компонент связности графа обходом в глубину (DFS), только граф — линк, *ограниченный* подмножеством: ребро попадает в граф, лишь если оба его конца в `vertices`. Именно так серое ребро «нижний — верхний» на рисунке выпадает из обоих линков. `unseen.pop()` берёт произвольный элемент множества, но число компонент и их состав от этого не зависят. Работа линейна по степени вершины.

`lower_upper_links()` делит соседей не по `values`, а по рангам `mesh.order`:

```
lower = {item for item in neighbors[vertex] if order[item] < order[vertex]}
upper = neighbors[vertex] - lower
return link_components(lower, link_edges[vertex]), link_components(upper, link_edges[vertex])
```

Ранги различны у всех вершин, поэтому «ниже» и «выше» дополняют друг друга, и равенств не бывает.

`analyze_morse(mesh, include_boundary=False)` проходит по всем вершинам, пропускает граничные (если `include_boundary` ложно), вычисляет `lower_count`, `upper_count` и выбирает тип по таблице п. 5.2: проверки идут в порядке `lower_count == 0` → минимум, `upper_count == 0` → максимум, `>= 2` у одного из чисел → седло, иначе регулярная. Возвращается список `CriticalPoint` для **всех** незамаскированных вершин, включая регулярные. Поэтому при подсчёте статистики регулярные вершины нужно отделять (у `Descriptor.counts` для них есть отдельный ключ `"regular"`).

5.5 Симуляция простоты

Проблема. Таблица п. 5.2 предполагает, что у соседей значения строго больше или строго меньше. В данных это не так. Глубинная карта LiDAR квантована (типично 1 мм), и на ровном полу или стене получаются плато из десятков одинаковых чисел. Если сосед равен центру, непонятно, куда его отнести. Если отнести «никуда», ломается признак $L = U$. Если решать случайно, результат будет меняться от запуска к запуску.

Идея SoS (Edelsbrunner, Mücke, 1990). Заменим $f(v_i)$ на $f(v_i) + \varepsilon \cdot i$, где i — номер вершины, а $\varepsilon > 0$ бесконечно мало. Тогда все значения различны, а неравенства, которые были строгими, не изменились. Вычислять с ε не нужно: сравнение возмущённых значений сводится к лексикографическому сравнению пар

$$u < v \Leftrightarrow f(u) < f(v) \text{ или } (f(u) = f(v) \text{ и } u < v).$$

То есть сначала значение, при равенстве — индекс вершины. Это и есть «бесконечно малое шевеление» из п. 4.5, выполненное символически.

В коде. `mesh.py`, свойство `TriMesh.order`:

```
ranks = np.empty(len(self.values), dtype=np.int64)
ranks[np.lexsort((np.arange(len(self.values)), self.values))] = np.arange(len(self.values))
```

`np.lexsort` сортирует по **последнему** ключу как главному, то есть по `values`, а при равенстве — по индексу. Результат — перестановка вершин по возрастанию; присваивание `ranks[perm] = arange` превращает её в ранг каждой вершины. Пример: `values = [2, 1, 2, 0]` даёт перестановку `[3, 1, 0, 2]` и ранги `[2, 1, 3, 0]`; две двойки различены индексами 0 и 2.

Что это даёт.

1. **Детерминизм.** Классификация зависит только от чисел и номеров вершин, а не от порядка обхода множеств или случайности. Повторный запуск на тех же данных даёт тот же результат.
2. **Комбинаторная «морсовость».** После SoS все значения различны, каждая вершина однозначно имеет тип, и на замкнутой поверхности формула Эйлера выполняется точно. Тест `test_plateaus_are_handled_by_the_same_tie_break_everywhere` берёт *полностью плоский* тор 8×8 из нулей. Получаются минимум в вершине 0 (наименьший индекс), максимум в вершине 63 и два седла (вершины 7 и 56): $1 - 2 + 1 = 0$.
3. **Но мульти-седла остаются.** В гладком случае малое шевеление разбивает обезьянье седло на два простых (п. 4.4). На фиксированной сетке это не так: узор «в н в н в н» сохраняется при любом достаточно малом изменении значений. SoS устраняет равенства, но не вырождения типа мульти-седла. Поэтому в коде есть `saddle_multiplicity` и взвешенный счёт `saddle_index_sum`.

SoS решает, где окажется седло. Вернёмся к $x^3 - 3xy^2$ на сетке 5×5 . У центра (вершина 12, значение 0) соседи С (вершина 17) и Ю (вершина 7) тоже равны 0. По правилу индекса Ю ($7 < 12$) попадает в нижний линк, а С ($17 > 12$) — в верхний. Получается узор В в, СВ н, С в, З н, ЮЗ в, Ю н, то есть обезьянье седло 3/3. Если перенумеровать вершины в обратном порядке, не трогая ни геометрии, ни значений, то С и Ю поменяются ролями. Центр становится регулярным, а седло распадается на два простых 2/2 в соседних вершинах. Суммарная кратность сёдел в обоих случаях равна 2, как у гладкого обезьяньего седла (п. 4.4): сумма индексов Банхоффа по внутренним вершинам не изменилась.

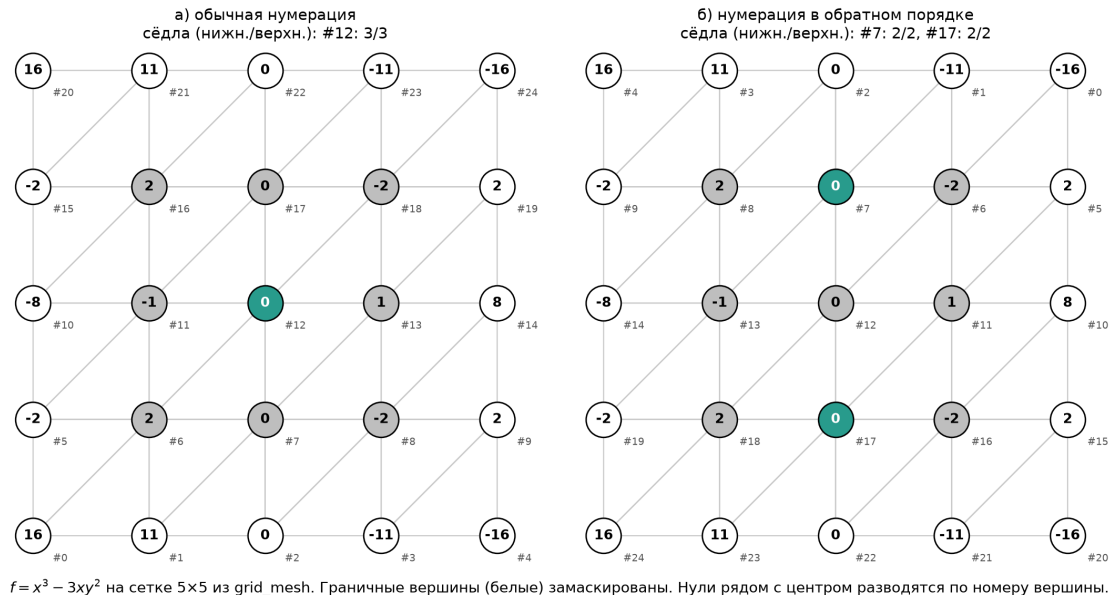


Рис. 5.2. Одна и та же функция $x^3 - 3xy^2$ при двух нумерациях вершин. Серые вершины регулярные, зелёные — сёдла.

Важно. На плато результат зависит от нумерации вершин. Два скана одной сцены с разным порядком точек (другой проход реконструкции, другой `voxel_downsample`) дадут разные *искусственные* критические точки на плато. Значимые точки (п. 5.6) от этого не зависят.

5.6 Плато и персистентность

Плато из m одинаковых значений SoS превращает в «лесенку» по индексам. Внутри неё возникают цепочки критических точек: локальные минимумы и сёдла, разница значений между которыми равна нулю. Это нулевая персистентность (гл. 8), и порогом персистентности такие точки отсекаются.

Чтобы отсечение работало, пары «рождение — смерть» должны строиться по **тем же** рангам. `persistence.py`, функция `critical_pairs()`:

```
order = mesh.order
tree = _lower_star_tree(mesh, order.astype(float)) # фильтрация по рангам SoS
tree.persistence(homology_coeff_field=2, min_persistence=0, persistence_dim_max=True)
for birth_simplex, death_simplex in tree.persistence_pairs():
    birth_vertex = max(birth_simplex, key=lambda vertex: order[vertex])
    ...
    death = float(mesh.values[death_vertex]) # персистентность — в исходных единицах
```

Каждый симплекс нижней звезды (lower star) получает ранг своей старшей вершины, а персистентность затем считается по сырым `values`. Почему не взять сырые значения сразу? Проверим на квантованном торе (поле из `tests/test_topology_fixes.py`, округлённое с шагом 0.1):

Фильтрация	Пар с <code>min_persistence=0</code>	Совпадение с классификатором
ранги <code>order</code> (как в коде)	46	каждая критическая вершина входит ровно в <code>max(saddle_multiplicity, 1)</code> пар
сырые <code>values</code>	4	только бесконечные классы, плато-пары потеряны

GUDHI отбрасывает пары нулевой длины (порог `min_persistence` строгий), а на плато в сырых значениях все пары имеют длину 0. Даже при `min_persistence=-1` (тогда пар 5762, включая регулярные) GUDHI разводит равенства по-своему, и вершины пар не обязаны совпадать с теми, что нашёл `analyze_morse`. В рангах длина пары равна нулю, только если рождающий и убивающий симплексы имеют одну старшую вершину. Это происходит внутри нижней звезды одной вершины, такие пары код дополнительно пропускает (`if death_vertex == birth_vertex: continue`). Мы проверили, что на шумном и на квантованном торе, а также на открытой сетке каждая внутренняя критическая вершина встречается ровно в `max(saddle_multiplicity, 1)` парах, а регулярная — ни в одной. Именно на это рассчитана функция `persistent_counts()`: седло считается столько раз, сколько у него значимых пар.

Квантование рельефа п. 5.7 (без шума):

Шаг квантования	Критических точек	Из них с персистентностью 0	После порога 10^{-12} (мин/сёдел/макс)
0.001	3	0	2 / 1 / 0
0.01	3	0	2 / 1 / 0
0.05	29	26	2 / 1 / 0
0.1	123	120	2 / 1 / 0

Пока шаг мал по сравнению с перепадом высот между соседними узлами, плато не возникают. Когда шаг его превосходит (у дна ям, где рельеф пологий), появляется лесенка искусственных точек, и все они уходят при любом положительном пороге.

5.7 Реальный прогон: шум

Возьмём рельеф гл. 4, $f = (x^2 - 1)^2 + y^2 + 0.3x$, на сетке 60×80 узлов (шаг около 0.05) и добавим гауссов шум с СКО σ :

```
y, x = np.meshgrid(np.linspace(-1.5, 1.5, 60), np.linspace(-2, 2, 80), indexing="ij")
height = (x**2 - 1) ** 2 + y**2 + 0.3 * x
mesh = grid_mesh(height + sigma * np.random.default_rng(seed).normal(size=height.shape))
points = analyze_morse(mesh) # граница замаскирована
pairs = critical_pairs(mesh)
kept = persistent_counts(annotate_persistence(points, pairs), pairs, 0.3)
```

Среднее по пяти seed (0–4), граница замаскирована:

σ	мин	сёдел	макс	мульти-сёдел	после порога 0.3 (мин/сёдел/макс)
0	2	1	0	0	2 / 1 / 0
0.001	2.2	1.2	0	0	2 / 1 / 0
0.003	2.8	2.2	0.4	0	2 / 1 / 0
0.01	7.2	9.0	2.8	0	2 / 1 / 0
0.03	47.6	77.6	32.2	1.2	2 / 1 / 0
0.1	274.6	495.8	235.8	15.4	2.4 / 2.0 / 0.6

Наблюдения. Шум уже при $\sigma = 0.003$ (меньше 0.1 % перепада высот) рождает лишние точки, прежде всего у дна ям и у седла, где рельеф пологий. Число точек растёт быстро, и сёдел примерно столько же, сколько минимумов и максимумов вместе: каждая лишняя яма рождается в паре с лишним перевалом. При $\sigma = 0.03$ появляются мульти-сёдла, хотя исходная функция их не имеет. Порог персистентности 0.3

возвращает истинную картину до $\sigma = 0.03$. При $\sigma = 0.1$ шум сравним с глубиной правой ямы (около 0.72 от седла), и появляются устойчивые ложные точки.

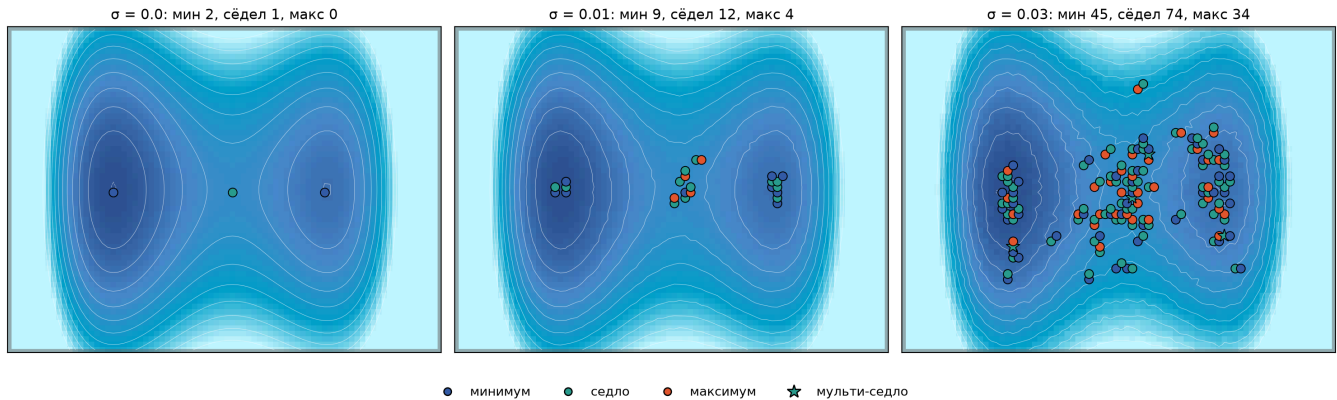


Рис. 5.3. Карта высот с критическими точками, найденными `analyze_morse`, при $\sigma = 0, 0.01, 0.03$ (seed 0). Серая рамка — замаскированная граница. Звёздочка — мульти-седло.

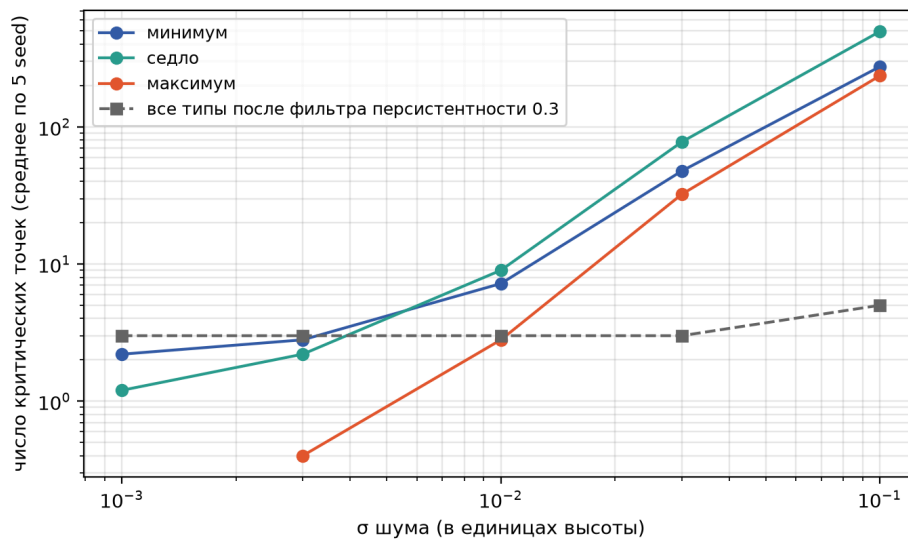


Рис. 5.4. Рост числа критических точек с шумом (логарифмические шкалы) и число точек, переживших порог персистентности 0.3.

Важно. На открытой сетке с замаскированной границей равенство $c_0 - c_1 + c_2 = \chi(\text{диска}) = 1$ не гарантировано. Например, при $\sigma = 0.1$, seed 0 получается $270 - 515 + 245 = 0$ (с весом кратности). Поэтому для открытых сеток CLI не передаёт в `build_descriptor` эйлерову характеристику, и `validate_euler` не вызывается.

5.8 Граница

По умолчанию `analyze_morse(mesh, include_boundary=False)` пропускает вершины из `mesh.boundary_vertices`, то есть концы рёбер, принадлежащих одному треугольнику (`edge_counts == 1`). Причина: линк граничной вершины — путь, а не цикл, и классификация «по компонентам» теряет связь с формулой Эйлера (п. 5.2). Внутренние вершины, соседние с границей, классифицируются как обычно, а их линк может содержать граничных соседей. Подробно о краевых эффектах и о дугах, упирающихся в край (`Separatrix.ends_on_boundary`), — гл. 10.

5.9 Вопросы для самопроверки

1. Почему у PL-функции с различными значениями в вершинах нет критических точек внутри треугольников?

2. У внутренней вершины 8 соседей со знаками «в н н в н в н» по кругу. Каковы L , U , тип и кратность?
3. Почему у граничной вершины может быть $L \neq U$, а у внутренней — нет?
4. Вычислите `order` для `values = [1, 0, 1, 0]`.
5. Чем опасно строить пары персистентности по сырым значениям, если классификатор работает по рангам?
6. Почему SoS не избавляет от мульти-сёдел, хотя в гладком случае малое шевеление их разбивает?

Ответы

1. Внутри треугольника функция линейна, её градиент постоянен и равен нулю, только если все три значения равны.
2. Дуги по кругу: в | нн | в | н | вв | н, причём последнее «н» примыкает к первому «в» — это разные буквы, дуги не сливаются. Получаем н-дуги {нн}, {н}, {н} и в-дуги {в}, {в}, {вв}: $L = U = 3$, обезьянье седло кратности 2.
3. У граничной вершины линк — путь, и дуги на его концах ничем не «замыкаются», поэтому н-дуг может быть на одну больше или меньше, чем в-дуг. На цикле дуги чередуются по кругу, и их число чётно.
4. Порядок по (значение, индекс): 1(0), 3(0), 0(1), 2(1), ранги [2, 0, 3, 1].
5. Пары на плато будут иметь нулевую длину и отбросятся GUDHI (или будут разведены по другому правилу), и критические вершины классификатора не найдут себе пар. `annotate_persistence` присвоит им персистентность 0, а `persistent_counts` неправильно посчитает сёдла.
6. Узор чередования «в н в н в н» — открытое условие: при малом изменении чисел знаки разностей с соседями не меняются. На фиксированной сетке разбить такое седло можно только сменой триангуляции или значений на конечную величину.

5.10 Что почитать

- T. Banchoff. Critical points and curvature for embedded polyhedra. *J. Differential Geometry* 1 (1967), 245–256.
- H. Edelsbrunner, E. P. Mücke. Simulation of Simplicity: a technique to cope with degenerate cases in geometric algorithms. *ACM Transactions on Graphics* 9(1) (1990), 66–104.
- H. Edelsbrunner, J. Harer, A. Zomorodian. Hierarchical Morse–Smale complexes for piecewise linear 2-manifolds. *Discrete & Computational Geometry* 30 (2003), 87–107.

6. Комплекс Морса–Смейла и сепаратрисы

Критические точки (главы 4–5) отвечают на вопрос «сколько на поверхности ям, перевалов и вершин». Комплекс Морса–Смейла отвечает на следующий вопрос: «как они соединены». Это граф на поверхности, вершины которого — критические точки, а рёбра — особые линии градиента, сепаратрисы. Он разрезает поверхность на простые ячейки и служит основой дескриптора в `descriptor.py`.

6.1 Интегральные линии градиента

Представим каплю воды на рельефе: она течёт вниз по самому крутому направлению, то есть по антиградиенту. Траектория капли и есть интегральная линия.

Определение. Пусть M — замкнутая гладкая поверхность, $f: M \rightarrow \mathbb{R}$ — функция Морса (гл. 4). *Интегральная линия* (integral line) — максимальная кривая $\gamma: \mathbb{R} \rightarrow M$, для которой

$$\frac{d\gamma}{dt}(t) = \nabla f(\gamma(t)), \quad \nabla f(\gamma(t)) \neq 0.$$

Вдоль неё f строго возрастает: $\frac{d}{dt}f(\gamma(t)) = \|\nabla f\|^2 > 0$. На компактной поверхности обе половины линии сходятся к критическим точкам:

$$\text{org}(\gamma) = \lim_{t \rightarrow -\infty} \gamma(t), \quad \text{dest}(\gamma) = \lim_{t \rightarrow +\infty} \gamma(t), \quad f(\text{org}) < f(\text{dest}).$$

Через каждую регулярную точку проходит ровно одна интегральная линия (теорема существования и единственности для ОДУ), поэтому линии не пересекаются и покрывают всю поверхность без критических точек.

6.2 Устойчивые и неустойчивые многообразия. Сепаратрисы

Соберём все линии, которые «втекают» в критическую точку p и «вытекают» из неё:

$$D(p) = \{p\} \cup \{x: \text{dest}(\gamma_x) = p\}, \quad A(p) = \{p\} \cup \{x: \text{org}(\gamma_x) = p\}.$$

$D(p)$ — нисходящее (descending) многообразие: всё, куда можно попасть, спускаясь из p по линиям градиента. $A(p)$ — восходящее (ascending) многообразие: всё, куда можно подняться из p . Для потока по $+\nabla f$ это соответственно устойчивое и неустойчивое многообразия. Это соглашение литературы и ТТК: двумерно нисходящее многообразие максимума и восходящее многообразие минимума. Размерность определяется индексом $\lambda(p)$ (гл. 4):

Точка	λ	$\dim D(p)$	$\dim A(p)$	Смысл
минимум	0	0	2	из ямы всё поднимается вверх
седло	1	1	1	две линии вниз, две вверх
максимум	2	2	0	с вершины всё спускается вниз

У седла $D(p)$ — две линии, по которым спускаются к минимумам, $A(p)$ — две линии, по которым поднимаются к максимумам. Эти четыре линии называются **сепаратрисами (separatrices)**: они разделяют области, стекающие в разные ямы, и области, поднимающиеся к разным вершинам. В коде у сепаратрисы есть вид `kind: u` — дуга от седла вниз к минимуму (descending), `s` — дуга от седла вверх к максимуму (ascending). Так их определяет `docs/DATA_CONTRACT.md`: «`kind is u for descent and s for ascent`». Если смотреть на поток по $-\nabla f$, то нисходящие дуги образуют неустойчивое (unstable), а восходящие — устойчивое (stable) многообразие седла; по-видимому, отсюда и буквы.

Важно. Направление «вниз» у дуги `u` — это направление трассировки в коде (от седла к минимуму), а не направление градиентного потока. Поле `source` у любой сепаратрисы — седло, `target` — экстремум.

6.3 Условие Морса–Смейла

Классический плохой пример — тор, поставленный вертикально, с функцией высоты. На нём минимум внизу, максимум вверху и два седла: нижняя и верхняя точки отверстия. Восходящие линии нижнего седла идут по внутренней стороне бублика и упираются точно в верхнее седло. Получается линия «седло → седло», и сепаратрисы двух седел совпадают.

Определение. Функция Морса f удовлетворяет *условию Морса–Смейла*, если для любых критических p, q многообразия $D(p)$ и $A(q)$ пересекаются трансверсально. На поверхности это равносильно тому, что *нет интегральных линий из седла в седло*. Действительно, для двух седел $D(p)$ и $A(q)$ — кривые, и их трансверсальное пересечение в двумерной M состоит из изолированных точек. Но пересечение состоит из целых интегральных линий (оно инвариантно относительно потока), поэтому, если оно непусто, его размерность не меньше 1. Остаётся только пустое пересечение.

Почему «общая» функция удовлетворяет условию. Связь седло–седло — событие коразмерности 1: стоит немного наклонить тор, и восходящая линия нижнего седла пройдёт мимо верхнего седла, слева или справа от него. Значит, сколь угодно малое возмущение f разрушает связь, а малое возмущение функции Морса–Смейла новой связи не создаёт. Градиенты Морса–Смейла образуют открытое плотное множество (теорема Купки–Смейла), так что на реальных данных с шумом условие выполняется «почти всегда», хотя на симметричных синтетических полях его легко нарушить.

6.4 Ячейки комплекса Морса–Смейла

Определение. Комплекс Морса–Смейла (Morse–Smale complex, MSC) — разбиение M на связные компоненты множеств $D(p) \cap A(q)$. Нульмерные клетки — критические точки, одномерные — сепаратрисы, двумерные — ячейки. Граф из критических точек и сепаратрис называется 1-скелетом комплекса.

Возьмём точку x внутри ячейки. Её линия начинается в минимуме и кончается в максимуме, а соседние линии идут туда же, пока не наткнутся на сепаратрису. Поэтому каждая ячейка — «четырёхугольник» с углами min – saddle – max – saddle и сторонами u, s, s, u (рис. 6.1). Два седла ячейки могут совпадать: седло может прилегать к одной ячейке двумя секторами.

Ячейка комплекса Морса–Смейла: min – saddle_a – max – saddle_b

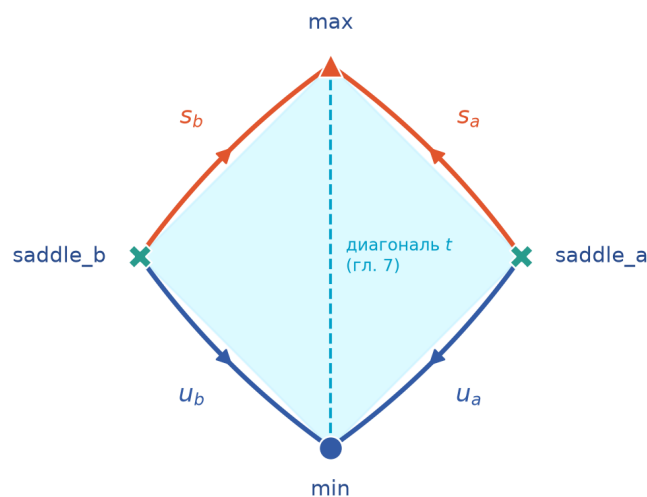


Рис. 6.1. Одна ячейка комплекса Морса–Смейла. Стрелки показывают направление трассировки дуг от седла. Штриховая диагональ t понадобится в главе 7.

Подсчёт. Пусть у функции n_0 минимумов, n_1 простых седел и n_2 максимумов. У каждого седла 4 сепаратрисы, так что рёбер $E = 4n_1$. Каждая ячейка ограничена 4 сторонами, каждая сторона граничит с 2 ячейками, поэтому $4F = 2E$ и $F = 2n_1$. Проверка через эйлерову характеристику (гл. 3):

$$V - E + F = (n_0 + n_1 + n_2) - 4n_1 + 2n_1 = n_0 - n_1 + n_2 = \chi(M).$$

Получилось ровно соотношение Морса из главы 4. Пример: на торе с одним минимумом, двумя седлами и одним максимумом $E = 8$ дуг и $F = 4$ ячейки. На сфере с одним минимумом и одним максимумом седел нет, $E = 0$, и единственная «ячейка» — сфера без двух точек, то есть цилиндр, а не четырёхугольник. Такой случай код ТТК-ветки не принимает (гл. 7).

6.5 Дискретный вариант: теория Морса Формана

На триангуляции градиента нет, и интегрировать ОДУ нельзя. Теория Морса Формана (Forman's discrete Morse theory) заменяет векторное поле комбинаторным объектом.

Интуиция. Стрелка «из вершины в ребро» значит «отсюда течёт по этому ребру», стрелка «из ребра в треугольник» — «через это ребро течёт в треугольник». Каждый симплекс участвует не более чем в одной стрелке.

Определение (кратко). Дискретный градиент (discrete gradient) — набор пар $(\sigma^{(k)} < \tau^{(k+1)})$ «грань — ко-фасет», где каждый симплекс входит не более чем в одну пару и нет замкнутых V-путей $\sigma_0 \rightarrow \tau_0 \supset \sigma_1 \rightarrow \tau_1 \supset \dots$. Непарные симплексы называются критическими: вершина — минимум, ребро — седло, треугольник — максимум. Сепаратрисы — V-пути от критического ребра к критической вершине (вниз) и, в двойственном смысле, к критическому треугольнику (вверх). Пары строятся по нижним звёздам (lower star) вершин, с порядком симуляции простоты (simulation of simplicity, SoS) того же типа, что и в главе 5.

Ацикличность V-путей гарантирует, что дискретные сепаратрисы конечны, а комбинаторная природа построения — что результат не зависит от округлений и плато разбираются тем же правилом SoS. Именно так работает ТТК (Topology ToolKit, библиотека на C++): фильтр `ttkMorseSmaleComplex` сначала строит дискретный градиент, затем идёт по V-путям. Поэтому ТТК сообщает критические точки как клетки: в `ttk_pipeline.py` у точки есть `CellId` и `CellDimension` (0, 1 или 2), а сепаратриса хранит `SourceId/DestinationId` как номера клеток. Функция `_separatrices()` ищет концы по ключу (`CellDimension`, `CellId`), а вершина сетки `vertex_id` может быть `-1`.

6.6 Эвристический бэкенд `separatrices.py`

По умолчанию (`--msc-backend heuristic` в `cli.py`) сепаратрисы считает функция `trace_separatrices()`. Идея: у седла нижний линк распадается на компоненты (гл. 5), и каждая компонента — это «сектор», в который уходит одна нисходящая дуга. Из каждой компоненты нижнего линка запускаем монотонный спуск, из каждой компоненты верхнего — монотонный подъём.

```
def _trace(mesh, start, direction, neighbors):
    order = mesh.order
    path = [start]
    current = start
    while True:
        candidates = [item for item in neighbors[current]
                      if direction * (order[item] - order[current]) > 0]
        if not candidates:
            return tuple(path)
        current = min(candidates, key=lambda item: direction * order[item])
        path.append(current)
```

Здесь `mesh.order` — ранг вершины в порядке SoS: сначала по значению, при равенстве по индексу (`Tri-Mesh.order`, гл. 5). `direction = -1` означает спуск, `+1` — подъём. В `trace_separatrices()` стартовая вершина выбирается тем же ключом: `start = min(component, key=lambda item: direction * order[item])`, после чего к пути спереди добавляется само седло: `vertices = (point.vertex, *path)`.

Почему путь конечен и кончается в экстремуме. Ранги — перестановка чисел $0, \dots, n-1$. На каждом шаге `order` строго убывает (при спуске), значит, вершина не повторяется, цикла нет, и шагов не больше $n-1$. Цикл останавливается, когда у текущей вершины нет соседа ниже, то есть нижний линк пуст. По классификации `analyze_morse()` (`critical.py`) это и есть минимум. Для подъёма всё симметрично: путь

кончается в вершине с пустым верхним линком, то есть в максимуме. Проверка `RuntimeError(... ended at non-extremum ...)` в коде на замкнутой сетке сработать не может и служит страховкой.

Типичная ошибка. Docstring называет шаг «steepest», но ключ `direction * order[item]` под `min` выбирает при спуске соседа с *наибольшим* рангом среди тех, что ниже, а при подъёме — с *наименьшим* среди тех, что выше. Это самый пологий монотонный шаг, а не крутейший. Проверка на торе $\sin(2\pi x/16) + 0.4 \cos(2\pi y/12)$ (сетка 12×16): у максимума $f = 1.400$ соседи ниже имеют значения 1.270, 1.270, 1.324, 1.324, 1.346, 1.346, и `_trace` идёт в 1.346, хотя крутейший спуск пошёл бы в 1.270. Монотонность, конечность и правильный конец пути от этого не страдают, но дуга получается «лестницей», ближе к линии уровня, чем к линии градиента.

Сколько дуг. На каждую компоненту нижнего линка одна дуга `u`, на каждую компоненту верхнего — одна дуга `s`. У простого седла по 2 компоненты, то есть ровно 2 дуги `u` и 2 дуги `s`. У мульти-седла (multi-saddle) кратности k компонент по $k + 1$, дуг $2(k + 1)$. Тест `test_simple_saddles_emit_exactly_two_arcs_each_way` проверяет первое на том же торе: 2 седла, дуги `["s"]*4 + ["u"]*4`.

```
>>> d = build_descriptor(periodic_grid_mesh(field_12x16), 0)
>>> d.counts, [(a.kind, a.source, a.target) for a in d.separatrices][:4]
({'min': 1, 'regular': 188, 'saddle': 2, 'max': 1},
 [('u', 12, 108), ('u', 12, 108), ('s', 12, 4), ('s', 12, 4)])
```

Обе дуги `u` седла 12 приходят в один и тот же минимум 108 с разных сторон. Это нормально: на торе с одним минимумом иначе и быть не может. Пара (источник, цель) поэтому не определяет дугу, к этому вернёмся в гл. 7.

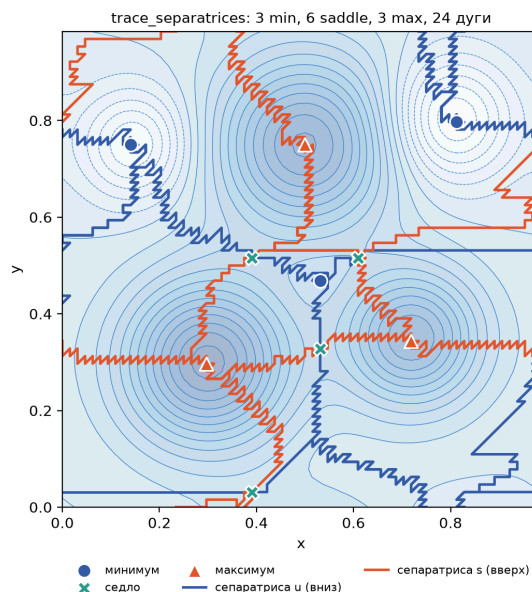


Рис. 6.2. Сепаратрисы, посчитанные `trace_separatrices` для суммы пяти периодических гауссиан на сетке 64×64 (тор). Синие — дуги `u`, красные — дуги `s`, поверх линий уровня. $3 - 6 + 3 = 0 = \chi(T^2)$.

На рис. 6.2 видны и сильные, и слабые стороны метода. Каждая дуга действительно соединяет седло с экстремумом нужного типа, а число дуг равно $4 \cdot 6 = 24$. Но дуги идут ступенями, а 19 пар дуг одного вида имеют общие внутренние вершины: дуги сливаются, не дойдя до экстремума (у линий настоящего градиента так не бывает, они могут сойтись только в критической точке).

Открытые поверхности. На сетке с краем (`grid_mesh`) граничные вершины исключены из классификации, и спуск может закончиться на крае, где нет критической точки. Тогда дуга не отбрасывается, а помечается `ends_on_boundary=True`, а в JSON её `destination_id` равен `null` (`cli.py`, `_descriptor_json()`). Путь при этом может пройти часть пути по краю. Подробно — в главе 10.

6.7 Чем эвристика отличается от настоящего комплекса

Docstring `trace_separatrices()` прямо говорит: «This is intentionally not presented as a substitute for TTK's MSC». Причины:

1. **Дуги могут сливаться и касаться.** В гладком случае линии градиента не сливаются вне критических точек. Монотонные пути по вершинам свободно проходят через одни и те же вершины, и никакой структуры, которая учитывала бы такие слияния, эвристика не строит.
2. **Нет ячеек.** Эвристика выдаёт только список дуг. Поля `quadrilaterals` и `colored_graph` у дескриптора из `build_descriptor()` пусты (`()` и `None`).
3. **Нет гарантии Морса–Смейла.** Ничто не проверяет, что дуги разных седел не совпадают и что четыре дуги седла действительно разделяют поверхность на четырёхугольники.
4. **Геометрия зависит от разбиения сетки.** Шаг выбирается по рангу, без учёта длины ребра, так что при анизотропной сетке форма дуг меняется.

Эвристика полезна как дешёвый, детерминированный эталон для тестов и для проверки контракта JSON. Комбинаторную структуру поверхности она не даёт.

6.8 ТТК-бэкенд

`ttk_backend.run_ttk_msc(mesh)` делает следующее.

1. Находит интерпретатор ParaView `pvpython` (`_find_pvpython()`: аргумент, переменная `MORSE_LIDAR_PVPYTHON`, `PATH`, затем локальная копия в `.tools/`). Если его нет, бросает `RuntimeError` с сообщением из `status()`.
2. Пишет сетку и значения в файл legacy VTK (`export_legacy_vtk()`, массив `ScalarField`).
3. Запускает `ttk_pipeline.py` в отдельном процессе. Тот применяет `ttkMorseSmaleComplex` (критические точки, восходящие и нисходящие 1-сепаратрисы; дуги седло–седло отключены: `SetComputeSaddleConnectors(False)`) и `ttkMorseSmaleQuadrangulation`, затем сериализует всё в JSON. Тип сепаратрисы `SeparatrixType = 0` становится `kind = "u"`, остальные — `"s"`.
4. Восстанавливает ячейки и раскрашенный граф (`_colored_dual()`, гл. 7).

CLI требует замкнутую сетку: при `--msc-backend ttk` на открытой сетке он завершается с сообщением «requires a closed mesh for the mandatory trivalent graph». Причина в том, что у каждой дуги должно быть ровно две соседние ячейки, а у дуги на крае только одна.

Каждая ячейка возвращается как `MorseSmaleQuadrilateral` (`descriptor.py`):

```
morse_smale_cells.append(
    MorseSmaleQuadrilateral(center, ids, (u_a, s_a, s_b, u_b))
)
```

где `ids = (min, saddle_a, max, saddle_b)` — идентификаторы критических точек, а `separatrices = (u_a, s_a, s_b, u_b)` — обход границы по кругу: u_a ($\min - \text{saddle}_a$), s_a ($\text{saddle}_a - \max$), s_b ($\max - \text{saddle}_b$), u_b ($\text{saddle}_b - \min$), как на рис. 6.1. Какое седло назвать `saddle_a`, решает сортировка путей по ключу (идентификатор дуги `u`, идентификатор седла). Поле `identifier` ячейки — номер вершины-центра в квадрангуляции ТТК, а не порядковый номер.

6.9 Упрощение по персистентности: ТТК и наш код

Шум даёт множество мелких пар «минимум–седло» и «седло–максимум» (гл. 8). В настоящем комплексе их убирают *сокращением* (cancellation): пара удаляется, а сепаратрисы соседей перестраиваются в обход. В экосистеме ТТК для этого перед построением комплекса применяют, например, фильтр `TopologicalSimplification` — его и рекомендует README. В `ttk_pipeline.py` упрощение не вызывается: комплекс строится по исходному полю.

В нашем коде есть только `persistence.persistent_counts()`: она считает, сколько критических точек участвует в парах с персистентностью не меньше порога. Docstring честно предупреждает: «does not reroute separatrices, i.e. it is not a full Morse-Smale cancellation». То же в `DATA_CONTRACT.md`: «`persistent_counts` filters pairs only; arcs and graph are not simplified». Значит, после `--persistence-threshold` числа в `persistent_counts` уменьшаются, а списки `separatrices`, `quadrilaterals` и `colored_graph` в отчёте остаются прежними.

В коде. Нельзя сравнивать `persistent_counts["saddle"]` с `len(separatrices) / 4`: первое отфильтровано, второе нет.

6.10 Вопросы для самопроверки

1. Почему интегральная линия не может начинаться и кончаться в одной и той же критической точке?
2. Сколько сепаратрис у мульти-седла кратности 2 в `trace_separatrices()` и почему?
3. Сколько ячеек у комплекса Морса–Смейла функции на сфере с 2 минимумами, 3 седлами и 3 максимумами?
4. Почему `_trace()` делает не больше $n - 1$ шагов. Что изменится, если заменить `min` на `max` в выборе соседа?
5. Почему ТТК-бэкенд отказывается работать с открытой сеткой?
6. После запуска с `--persistence-threshold 0.5` число седел в `persistent_counts` упало с 10 до 2. Сколько дуг будет в поле `separatrices`?

Ответы

1. Вдоль линии f строго растёт, поэтому $f(\text{org}) < f(\text{dest})$.
2. У мульти-седла кратности 2 по 3 компоненты нижнего и верхнего линка, значит, 3 дуги u и 3 дуги s , всего 6.
3. Проверка: $2 - 3 + 3 = 2 = \chi(S^2)$. Ячеек $F = 2n_1 = 6$, дуг $4n_1 = 12$.
4. Ранг строго монотонен и принимает n значений. С `max` путь станет крутейшим спуском в смысле ранга (самый низкий сосед); конечность и конец в экстремуме сохранятся, изменится форма дуг.
5. Дуга на краю граничит с одной ячейкой, и дуальное ребро построить нельзя; `_colored_dual()` и `validate_colored_graph()` требуют ровно двух треугольников на дугу.
6. Столько же, сколько без порога (для 10 простых седел — 40): `persistent_counts` дуги не перестраивает.

6.11 Что почитать

- H. Edelsbrunner, J. Harer, A. Zomorodian. Hierarchical Morse–Smale complexes for piecewise linear 2-manifolds. *Discrete & Computational Geometry*, 30 (2003), 87–107.
- R. Forman. A user’s guide to discrete Morse theory. *Séminaire Lotharingien de Combinatoire*, 48 (2002).
- J. Tierny, G. Favelier, J. A. Levine, C. Gueunet, M. Michaux. The Topology ToolKit. *IEEE Transactions on Visualization and Computer Graphics*, 24(1) (2018).

7. Графы и двойственные структуры

В главе 6 комплекс Морса–Смейла был набором ячеек-четырёхугольников. Чтобы хранить его в JSON, сравнивать и проверять, его переводят в граф: каждой ячейке и каждой дуге сопоставляется что-то дискретное. Эта глава даёт минимум теории графов и разбирает, как устроен и как проверяется `Colored-Graph` из `descriptor.py`.

7.1 Графы: основные понятия

Определение. Граф (graph) $G = (V, E)$ — множество вершин V и множество рёбер E , где ребро — неупорядоченная пара различных вершин $\{a, b\}$. *Степень* (degree) $\deg v$ — число рёбер, содержащих v . Граф *связен*, если между любыми двумя вершинами есть путь. Максимальные связные подграфы называются *компонентами связности*.

Каждое ребро добавляет единицу к степени двух вершин, поэтому

$$\sum_{v \in V} \deg v = 2|E| \quad (\text{лемма о рукопожатиях}).$$

Если все степени равны 3 (граф *трёхвалентный*, trivalent), то $3|V| = 2|E|$. В частности, $|V|$ чётно. Это соотношение проверяет тест `test_real_ttk_pipeline_builds_trivalent_graph_from_shared_arcs`: `len(edges) * 2 == node_count * 3`.

7.2 Обход графа: DFS и BFS на примере `link_components`

Компоненты связности находят обходом: берём непосещённую вершину, обходим всё, что из неё достижимо, получаем одну компоненту и повторяем. Если очередь кандидатов — *стек* (последним пришёл, первым ушёл), это обход в глубину (depth-first search, DFS); если *очередь* — обход в ширину (breadth-first search, BFS). Набор найденных компонент от выбора не зависит, меняется только порядок посещения.

В проекте так считаются компоненты нижнего и верхнего линка вершины (гл. 5), `critical.py`, функция `link_components()`:

```
graph = {vertex: set() for vertex in vertices}
for left, right in link_edges:
    if left in vertices and right in vertices:      # ребро линка внутри подмножества
        graph[left].add(right)
        graph[right].add(left)
components = []
unseen = set(vertices)
while unseen:
    start = unseen.pop()
    component = {start}
    stack = [start]
    while stack:
        current = stack.pop()                       # стек -> DFS
        for neighbor in graph[current] & unseen:
            unseen.remove(neighbor)                 # отмечаем при постановке в стек
            component.add(neighbor)
            stack.append(neighbor)
    components.append(component)
```

Заметим две детали. Во-первых, граф строится только на подмножестве `vertices` (например, на вершинах линка ниже центра): ребро линка, у которого хотя бы один конец выше, отбрасывается. Именно это превращает «линк — цикл» в «нижний линк — несколько дуг цикла». Во-вторых, вершина удаляется из `unseen` в момент постановки в стек, а не в момент снятия, поэтому каждая вершина попадает в стек ровно один раз и время работы $O(|V|+|E|)$. Замена `stack.pop()` на `stack.pop(0)` превратила бы обход в BFS с тем же результатом.

Пример. У простого седла линк — цикл из 6 соседей с рангами, скажем, $(+, -, -, +, +, -)$ относительно центра. Нижний граф содержит вершины 2, 3, 6 и рёбра $\{2, 3\}$ (ребро $\{6, 1\}$ отброшено, вершина 1 выше). Компоненты: $\{2, 3\}$ и $\{6\}$ — две, значит, седло.

7.3 Мультиграфы, петли, кратные рёбра

Если разрешить ребру соединять вершину с ней самой (*петля*, self-loop) и разрешить несколько рёбер между одной парой вершин (*кратные рёбра*), получится *мультиграф* (multigraph). Для графа комплекса Морса–Смейла кратные рёбра неизбежны: на торе из гл. 6 обе дуги и седла S_1 ведут в один минимум m . Поэтому 1-скелет комплекса — мультиграф, и представлять его парами (источник, цель) нельзя.

В раскрашенном двойственном графе (п. 7.5) ситуация обратная: там петли и одноцветные кратные рёбра на корректных данных невозможны, и `validate_colored_trivalent_graph()` их запрещает:

- петля означала бы, что треугольник граничит сам с собой по одной из своих сторон, а все три стороны треугольника разного цвета и разные;
- два ребра одного цвета между одной парой узлов означали бы, что у треугольника две стороны одного цвета.

Рёбра *разного* цвета между одной парой узлов допустимы: ключ дубликата — тройка `(min(l, r), max(l, r), color)`.

7.4 Двойственный граф триангуляции

Определение. Двойственный граф (dual graph) триангуляции: одна вершина (узел) на каждый треугольник и одно ребро на каждое ребро сетки, общее для двух треугольников.

На замкнутой поверхности у каждого треугольника три соседа, двойственный граф трёхвалентен, и $3F = 2E$ — то же равенство, что `TriMesh.closed` проверяет как «каждое ребро встречается дважды» (`mesh.py`, `edge_counts`). На открытой сетке граничное ребро принадлежит одному треугольнику, двойственного ребра у него нет, и степени граничных узлов меньше 3 (рис. 7.1).

grid_mesh 4×4: 18 треугольников; в кружке — степень узла

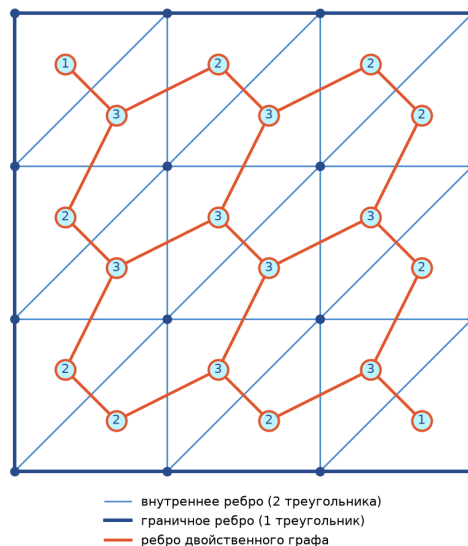


Рис. 7.1. Двойственный граф сетки `grid_mesh` 4×4 вершины (18 треугольников, 33 ребра, из них 12 граничных). Узлы — центры треугольников, число — степень узла.

Проверка по рисунку: внутренних рёбер $33 - 12 = 21$, сумма степеней узлов $2 \cdot 21 = 42$; узлов степени 1, 2, 3 соответственно 2, 8, 8, и $2 + 16 + 24 = 42$.

7.5 От ячеек Морса–Смейла к раскрашенному трёхвалентному графу

ТТК возвращает квадрангуляцию, в которой каждая ячейка Морса–Смейла дополнительно разрезана на 4 маленьких четырёхугольника. У их вершин массив `QuadVertType`: 0 — критическая точка, 1 — середина сепаратрисы, 2 — центр ячейки, а `QuadCellId` хранит номер критической точки или дуги. Функция `ttk_backend._colored_dual()`:

1. Поворачивает каждый маленький четырёхугольник так, чтобы типы шли как `[0, 1, 2, 1]`, иначе ошибка.
2. Группирует четвёрки по центру (тип 2): получается исходная ячейка, кусков должно быть ровно 4.
3. По серединам дуг восстанавливает смежность углов и проверяет, что в ячейке один минимум, один максимум и два пути $\min \rightarrow \text{saddle} \rightarrow \max$; виды дуг должны быть `u` ($\min$ – saddle) и `s` (saddle – $\max$).
4. Режет ячейку новой дугой `t` по диагонали $\min$ – $\max$ на два треугольника:

```
t_arc = ("t", center)
triangles.extend(((ids[0], ids[1], ids[2]), (ids[0], ids[2], ids[3])))
triangle_arcs.extend((
    ("separatrix", u_a), ("separatrix", s_a), t_arc),
    ("separatrix", s_b), ("separatrix", u_b), t_arc),
))
```

Здесь `ids = (min, saddle_a, max, saddle_b)`. Треугольник $(\min, \text{saddle}_a, \max)$ ограничен дугами u_a, s_a, t , треугольник $(\min, \max, \text{saddle}_b)$ — дугами s_b, u_b, t . Тройки дуг треугольника называются **primal arcs** (исходные, «первичные» дуги), а ключ дуги — пара: `("separatrix", SeparatrixId)` или `("t", центр ячейки)`.

5. Строит граф: узел — треугольник, ребро — дуга, общая ровно для двух треугольников (**shared primal arc**). Цвет ребра: `t` для диагонали, иначе вид сепаратрисы `u` или `s`. Если дуга лежит не в двух треугольниках, бросается «ТТК quadrangulation is not closed».

Получается граф, где у каждого узла ровно по одному ребру каждого цвета s, u, t : это просто три стороны треугольника. Заметим, что у каждого треугольника вершины трёх разных типов ($\min, \text{saddle}, \max$), а цвет стороны определяет противоположную вершину: сторона `u` ($\min$ – saddle) лежит напротив $\max$, `s` — напротив $\min$, `t` — напротив седла.

7.6 Проверки `validate_colored_trivalent_graph` и `validate_colored_graph`

Первая функция получает число узлов и список рёбер `(left, right, color)`:

Условие	Ошибка	Какую проблему данных ловит
<code>nodes <= 0</code>	«has no nodes»	нет ячеек: поверхность без сёдел (сфера с min и max) или пустой вывод ТТК
<code>color not in {"s","u","t"}</code>	«unknown edge color»	некорректный цвет при ручной сборке графа или после изменения <code>_colored_dual()</code> (из конвейера ТТК не возникает: любой <code>SeparatrixType</code> кроме 0 молча становится s)
<code>0 <= left, right < nodes</code>	«unknown node»	ребро ссылается на несуществующий треугольник
<code>left == right</code>	«self-loop»	треугольник «склеен» сам с собой
ключ уже в <code>seen</code>	«duplicate edge»	две одноцветные стороны у пары треугольников
все степени 3	«not trivalent»	лишняя или пропавшая дуга
цвета узла <code>["s","t","u"]</code>	«exactly one s, u, and t»	неверные виды дуг при правильной степени

Вторая функция работает со всем `ColoredGraph` и сначала проверяет, что граф *выводится* из треугольников:

1. `len(primal_arcs) == node_count` — у каждого узла сохранены его три дуги.
2. `len(set(arcs)) == 3` — три дуги треугольника различны.
3. По `incidence` каждая дуга лежит ровно в двух треугольниках. Если дуга в одном треугольнике — квадрангуляция не замкнута (край скана, потерянная ячейка); если в трёх и более — сетка не является многообразием.
4. Никакие два ребра не ссылаются на одну дугу («multiple dual edges reference the same primal arc»).
5. `actual == expected` — множество рёбер графа совпадает с множеством общих дуг: ни одно ребро не пропущено, не добавлено и не «перекинуто» на другой узел.
6. В конце вызывается `validate_colored_trivalent_graph()`.

Проверено запуском на игрушечном графе тора (п. 7.8): удаление последнего треугольника (узла 7) даёт «primal arc ('separatrix', 14) is shared by 1 triangles, not two»; удаление одного ребра и перенос ребра на другой узел дают «does not correspond exactly to shared primal arcs»; смена цвета одного ребра даёт «exactly one s, u, and t edge»; пустой граф — «has no nodes».

Важно. Валидатор не сверяет цвет ребра с видом его дуги. Если глобально поменять местами цвета `t` и `u`, у каждого узла по-прежнему будет по одному ребру каждого цвета, и проверка пройдёт, хотя ребро с дугой `("t", 12)` окрашено в `u`. Это проверено запуском; цвет здесь гарантирует только `_colored_dual()`.

7.7 Почему граф — полный комбинаторный инвариант

Комментарий к `ColoredEdge.primal_edge` в `descriptor.py`: «Keeping it makes it impossible to collapse the graph to critical-point pairs». Смысл виден на примере тора: все четыре ячейки имеют одинаковый кортеж углов $(0, 1, 2, 3) = (\min, S_1, \max, S_2)$, а обе дуги `u` седла S_1 соединяют одни и те же точки. Если описывать ребро парой критических точек, ячейки и дуги неразличимы, и структура теряется. Ключ дуги (`SeparatrixId` или центр ячейки) этого не допускает.

Сам граф с раскраской содержит всю комбинаторику. Трёхвалентный граф с правильной 3-раскраской рёбер задаёт двумерную поверхность: берём по треугольнику на узел и склеиваем треугольники по сторонам, соответствующим рёбрам (так устроены «раскрашенные графы» в теории кристаллизаций). Вершины поверхности восстанавливаются как двуцветные циклы:

Цвета цикла	Отсутствующий цвет	Вершина триангуляции
u, s	t	седло
u, t	s	минимум
s, t	u	максимум

Например, вокруг седла лежат треугольники, стороны которых при этом седле имеют цвета u и s . Отсюда эйлерова характеристика считается по одному графу:

$$\chi = \#\{\text{двуцветных циклов}\} - |E_{\text{dual}}| + |V_{\text{dual}}|.$$

7.8 Пример руками: сфера и тор

Сфера, 1 min, 1 max, 0 седел. Сепаратрис нет, ячеек-четырёхугольников нет, треугольников 0. Граф пуст, и `validate_colored_graph(ColoredGraph(), (), ())` падает с «colored dual graph has no nodes». Это ожидаемо: формула $F = 2n_1$ из главы 6 даёт 0.

Общий случай. При $n_1 \geq 1$ простых седлах:

$$F = 2n_1, \quad |V_{\text{dual}}| = 2F = 4n_1, \quad |E_{\text{dual}}| = \frac{3}{2}|V_{\text{dual}}| = 6n_1 = \underbrace{4n_1}_{s,u} + \underbrace{2n_1}_t,$$

и рёбер каждого цвета по $2n_1$. Проверка формулой из п. 7.7: циклов $n_0 + n_1 + n_2$, поэтому $\chi = (n_0 + n_1 + n_2) - 6n_1 + 4n_1 = n_0 - n_1 + n_2$ — снова соотношение Морса.

Тор, $f = \sin x + 0.4 \cos y$. Максимум $M = (\pi/2, 0)$, минимум $m = (3\pi/2, \pi)$, седла $S_1 = (\pi/2, \pi)$ и $S_2 = (3\pi/2, 0)$. Сепаратрисы лежат на прямых $x = \pi/2$ (s -дуги S_1), $y = \pi$ (u -дуги S_1), $y = 0$ (s -дуги S_2), $x = 3\pi/2$ (u -дуги S_2) и режут тор на 4 прямоугольника. По формулам: 4 ячейки, 8 узлов, 12 рёбер, по 4 каждого цвета; $3 \cdot 8 = 2 \cdot 12$.

Без ТТК это можно проверить, собрав вручную квадрангуляцию в формате ТТК (`TTKQuadrangulation` с типами вершин 0/1/2) и пропустив её через `_colored_dual()` и `validate_colored_graph()` — так сделано в `figures/ch07_figures.py`, функция `torus_example()`:

```
>>> cells, graph = torus_example()           # 4 ячейки по 4 куска, 8 дуг
>>> graph.node_count, len(graph.edges)
(8, 12)
>>> Counter(e.color for e in graph.edges)
Counter({'u': 4, 's': 4, 't': 4})
>>> validate_colored_graph(graph)           # без исключений
```

Двуцветных циклов: $\{u, s\}$ — два (узлы 0, 2, 4, 6 вокруг S_1 и 1, 3, 5, 7 вокруг S_2), $\{u, t\}$ и $\{s, t\}$ — по одному, длиной 8. Значит, $\chi = 4 - 12 + 8 = 0$, как и положено тору. Те же числа (4 ячейки, 8 узлов, 12 рёбер) утверждает тест с настоящим ТТК в `tests/test_ttk_backend.py`.

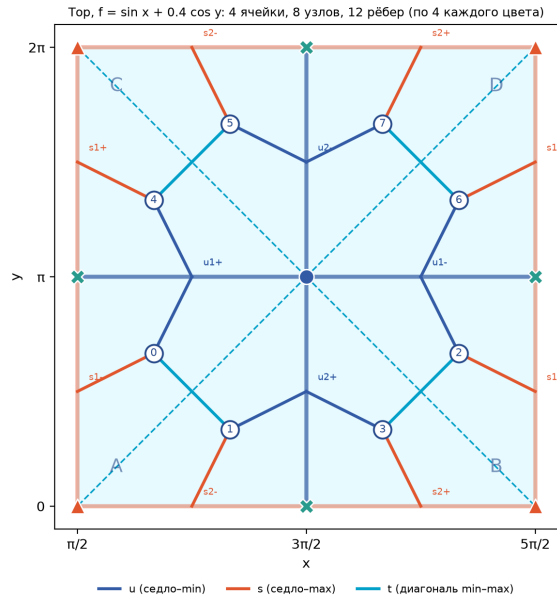


Рис. 7.2. Раскрашенный трёхвалентный граф тора из примера. Квадрат $[\pi/2, 5\pi/2] \times [0, 2\pi]$ — развёртка тора, противоположные стороны склеены, поэтому рёбра, уходящие за край, продолжаются с другой стороны (подписи дуг повторены). Штриховые линии — диагонали t .

7.9 Вопросы для самопроверки

1. Почему в `link_components()` вершину удаляют из `unseen` до того, как положить в стек? Что сломается, если удалять после `pop()` ?
2. Может ли трёхвалентный граф иметь 7 узлов?
3. Почему кратные рёбра в 1-скелете комплекса допустимы, а одноцветные кратные рёбра в раскрашенном графе — нет?
4. Какой проверкой поймается ситуация, когда у квадрангуляции пропала одна ячейка?
5. Сколько узлов и рёбер каждого цвета у графа для поверхности рода 3 с 1 минимумом, 6 седлами и 1 максимумом?
6. Какую ошибку данных не ловит `validate_colored_graph()` ?

Ответы

1. Иначе вершина может попасть в стек несколько раз (от разных соседей, пока её ещё не сняли). Компоненты останутся верными (`component` — множество), но оценка $O(|V| + |E|)$ на размер стека и число итераций перестанет быть очевидной.
2. Нет: $3|V| = 2|E|$, значит, $|V|$ чётно.
3. В 1-скелете две разные дуги могут соединять одну пару критических точек. В раскрашенном графе одноцветное кратное ребро значило бы, что у треугольника две стороны одного цвета, а у треугольника ровно одна сторона каждого цвета.
4. Дуги на её границе окажутся в одном треугольнике: «primal arc ... is shared by 1 triangles, not two» (в `_colored_dual()` раньше: «quadrangulation is not closed»).
5. $\chi = 1 - 6 + 1 = -4 = 2 - 2 \cdot 3$ — верно для рода 3. Ячеек 12, узлов 24, рёбер 36, по 12 каждого цвета.
6. Несоответствие цвета ребра виду дуги, если раскраска остаётся правильной (например, глобальная перестановка цветов); также она не проверяет поле `triangles`.

7.10 Что почитать

- R. Diestel. *Graph Theory*. Springer, Graduate Texts in Mathematics 173 (гл. 1: основные понятия).
- Т. Н. Cormen, С. Е. Leiserson, R. L. Rivest, С. Stein. *Introduction to Algorithms*, MIT Press (главы об обходе графов в ширину и глубину).

- M. Ferri, C. Gagliardi, L. Grasselli. A graph-theoretical representation of PL-manifolds — a survey on crystallizations. *Aequationes Mathematicae*, 31 (1986), 121–141.

8. Персистентные гомологии и сравнение сканов

Теория Морса (главы 4–6) говорит, какие критические точки есть у функции. Но на реальном скане их тысячи, и почти все порождены шумом. Персистентные гомологии (persistent homology) отвечают на другой вопрос: насколько каждая критическая точка важна. Ответ даётся одним числом, персистентностью, а множество таких чисел (диаграмма персистентности) оказывается устойчивым к шуму. На этом построено сравнение сканов в проекте: модуль `comrade.py` сравнивает именно диаграммы, а не списки критических точек.

8.1 Гомологии без алгебры

Интуиция. Число Бетти (Betti number) β_k считает «дыры размерности k »: β_0 — число связных компонент, β_1 — число независимых петель, которые нельзя стянуть в точку и которые не ограничивают кусок поверхности, β_2 — число замкнутых (без края) компонент ориентируемой поверхности, наглядно — число ограниченных ею полостей (как в гл. 4.7). Для сферы $(\beta_0, \beta_1, \beta_2) = (1, 0, 1)$, для тора $(1, 2, 1)$: одна компонента, две петли (вдоль и поперёк трубки), одна полость внутри. Эйлерова характеристика (глава 3) выражается через числа Бетти:

$$\chi = \beta_0 - \beta_1 + \beta_2,$$

для тора $\chi = 1 - 2 + 1 = 0$.

Строгое определение понадобится только для того, чтобы понимать слово «класс». Цепь (chain) размерности k — это формальная сумма k -симплексов с коэффициентами из $\mathbb{Z}/2 = \{0, 1\}$, то есть просто набор симплексов ($1 + 1 = 0$: симплекс, взятый дважды, исчезает). Граница ∂ треугольника — сумма трёх его рёбер, граница ребра — сумма двух вершин. Цикл — цепь с нулевой границей (замкнутая ломаная из рёбер: каждая вершина встречается чётное число раз). Граница чего-то — всегда цикл, так как $\partial\partial = 0$. Гомологии

$$H_k = Z_k / B_k, \quad Z_k = \ker \partial_k, \quad B_k = \operatorname{im} \partial_{k+1}, \quad \beta_k = \dim H_k,$$

то есть циклы, в которых два цикла считаются одинаковыми (одним классом), если отличаются на границу. Петля вокруг трубки тора и та же петля, сдвинутая вдоль трубки, — один класс: между ними лежит полоска треугольников, и их сумма равна границе полосы.

В коде. Коэффициенты $\mathbb{Z}/2$ задаются явно: `persistence.py`, функция `critical_pairs()`, вызов `tree.persistence(homology_coeff_field=2, ...)`. Над $\mathbb{Z}/2$ ориентации не нужны, а для поверхностей ответ тот же, что над полем рациональных чисел.

8.2 Фильтрация и нижняя звезда

Интуиция: будем «наливать воду» в рельеф f и смотреть, как меняется затопленная часть $M_t = f^{-1}(-\infty, t]$ (подуровневое множество, sublevel set, глава 4). Гомологии M_t меняются только при прохождении критических значений.

Определение. Фильтрация (filtration) — последовательность вложенных комплексов $K_0 \subseteq K_1 \subseteq \dots \subseteq K_m = K$. Её удобно задавать числом $f(\sigma)$ на каждом симплексе: $K_t = \{\sigma: f(\sigma) \leq t\}$. Чтобы K_t был комплексом, грань симплекса должна входить не позже самого симплекса: $\tau \subset \sigma \Rightarrow f(\tau) \leq f(\sigma)$. Для функции, заданной в вершинах, используется фильтрация нижней звезды (lower star):

$$f(\sigma) = \max_{v \in \sigma} f(v).$$

Симплекс появляется ровно тогда, когда появилась его последняя (самая высокая) вершина. Множество симплексов, появляющихся вместе с вершиной v , — это нижняя звезда v ; её строение задаётся нижним линком (глава 5), поэтому изменения гомологий происходят именно в PL-критических вершинах.

Пример. Треугольник с вершинами $f = (0, 1, 5)$: ребро $\{0, 1\}$ получает значение 1, рёбра $\{0, 2\}$ и $\{1, 2\}$ и сам треугольник получают 5.

Как это в коде: `persistence.py`, функция `_lower_star_tree()`:

```
tree = gudhi.SimplexTree()
for vertex, value in enumerate(filtration):
    tree.insert([int(vertex)], filtration=float(value))
for left, right in mesh.edge_counts:
    tree.insert([left, right], filtration=float(max(filtration[left], filtration[right])))
for face in mesh.faces:
    vertices = [int(vertex) for vertex in face]
    tree.insert(vertices, filtration=float(np.max(filtration[vertices])))
```

Порядок вставки здесь существенен. Метод `SimplexTree.insert` GUDHI при вставке треугольника создаёт недостающие грани с фильтрацией самого треугольника, а если грань уже есть с большим значением, понижает его. Если пропустить второй цикл, ребро получит минимум значений смежных треугольников, а не максимум своих двух вершин. Проверка: после вставки треугольника $\{0, 1, 2\}$ со значением 5 ребро $\{0, 1\}$ имеет фильтрацию 5 вместо 1; вторая грань $\{0, 1, 3\}$ со значением 1 понижает его до 1, но только потому, что такой треугольник нашёлся. Ребро появляется позже, чем должно, компоненты сливаются с опозданием, и в диаграмме возникают ложные классы. На сетке 12×12 с $f = \sin(x/2) + \cos(y/2.5)$ правильная фильтрация даёт 2 интервала размерности 0, а фильтрация без явных рёбер — 15. Это реальная ошибка, найденная при доработке проекта (пункт 12 в `docs/REVIEW.md`); там же пункт 13: без `persistence_dim_max=True` GUDHI не считал H_2 и терял глобальный максимум.

8.3 Рождение, смерть и правило старшего

При движении t вверх классы гомологий рождаются (birth) и умирают (death). Компонента рождается в минимуме; петля рождается в седле, которое замыкает кольцо; полость (на замкнутой поверхности) рождается в глобальном максимуме. Компонента умирает, когда сливается с другой; петля умирает, когда её заклеивает максимум.

Какая из двух сливающихся компонент умирает? Правило старшего (elder rule): выживает та, что родилась раньше (у неё ниже минимум), умирает младшая. Тогда каждой смерти соответствует однозначная пара (рождение, смерть). Для функции на поверхности получаются пары трёх типов:

размерность	рождение	смерть
0	минимум	седло
1	седло	максимум
$0, \infty$	глобальный минимум	никогда
$1, \infty$	сёдла петель рода ($2g$ штук)	никогда
$2, \infty$	глобальный максимум (только на замкнутой поверхности)	никогда

Классы с бесконечной смертью называются существенными (essential): они описывают топологию самой поверхности, а не функции. Их число равно $\beta_0, \beta_1, \beta_2$ поверхности.

Пример на отрезке. Пусть в вершинах $x_0, \dots, x_6$ значения $f = (3, 1, 4, 2, 6, 0, 5)$, рёбра соединяют соседей.

- $t = 0$: рождается компонента A в x_5 . $t = 1$: компонента B в x_1 . $t = 2$: компонента C в x_3 .
- $t = 3$: добавляется x_0 и ребро x_0x_1 ; x_0 просто присоединяется к B .
- $t = 4$: добавляется x_2 , рёбра x_1x_2 и x_2x_3 сливают B (рождена в 1) и C (рождена в 2). Умирает младшая C : пара $(2, 4)$.
- $t = 5$: x_6 присоединяется к A . $t = 6$: x_4 сливает B (1) и A (0); умирает B : пара $(1, 6)$.
- A не умирает никогда: пара $(0, \infty)$.

Персистентность пары — разность $6 - 1 = 5$ для B и $4 - 2 = 2$ для C : насколько «глубока» соответствующая яма. Тот же ответ дают GUDHI и функция проекта (рис. 8.1): `critical_pairs(grid_mesh(np.tile(f, (3, 1))))` на полосе из трёх одинаковых строк возвращает пары $(x_3, x_2) = (2, 4)$, $(x_1, x_4) = (1, 6)$ и (x_5, ∞) .

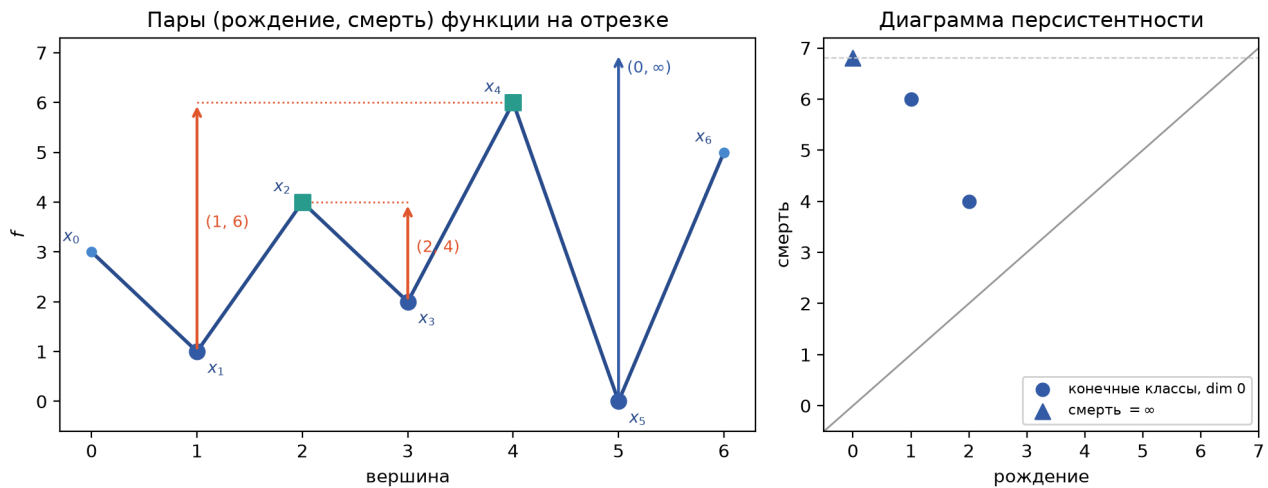


Рис. 8.1. Функция на отрезке: стрелки от минимума (рождение) до седла (смерть) и та же информация в виде диаграммы персистентности. Посчитано `critical_pairs()`.

Пример на маленькой сетке. Возьмём топ 4×4 (`periodic_grid_mesh`), вершина $v = 4 \cdot \text{строка} + \text{столбец}$:

$$f = \begin{pmatrix} 11 & 15 & 9 & 1 \\ 12 & 2 & 14 & 10 \\ 0 & 4 & 7 & 6 \\ 13 & 5 & 3 & 8 \end{pmatrix}.$$

Классификация `analyze_morse()`: минимумы v_8, v_3, v_5 (значения 0, 1, 2), седла v_9 (4, двукратное), v_{10} (7), v_7 (10), v_0 (11), максимумы v_4 (12), v_1 (15). С учётом кратности $3 - 5 + 2 = 0 = \chi$. Функция `critical_pairs()` возвращает:

dim	рождение	смерть	персистентность
0	$v_5, 2$	$v_9, 4$	2
0	$v_3, 1$	$v_9, 4$	3
0	$v_8, 0$	∞	∞
1	$v_0, 11$	$v_4, 12$	1
1	$v_{10}, 7$	∞	∞
1	$v_7, 10$	∞	∞
2	$v_1, 15$	∞	∞

Двукратное седло v_9 сливает сразу три компоненты $\{v_8\}, \{v_3, v_{14}\}$ (вершина v_{14} со значением 3 примыкает к v_3 через периодическую склейку) и $\{v_5\}$. По правилу старшего выживает v_8 , а v_3 и v_5 умирают в v_9 : мульти-седло участвует в стольких парах, какова его кратность. Существенные пары дают $\beta = (1, 2, 1)$ тора.

8.4 Диаграмма персистентности и порог

Диаграмма персистентности (persistence diagram) $\text{Dgm}_k(f)$ — мультимножество точек (b, d) на плоскости, по одной на пару размерности k ; существенные точки рисуют на линии « $d = \infty$ ». Все точки лежат выше диагонали $d = b$, а персистентность $d - b$ — это (с точностью до множителя $\sqrt{2}$) расстояние по вертикали до диагонали. Точки у диагонали — короткоживущие классы, то есть шум; далёкие точки — особенности формы. На рис. 8.2 слева гладкая функция $\sin(4\pi x/48) + 0.4\cos(2\pi y/40) + 0.2\sin(2\pi x/48)$ на торе 40×48 : две конечные точки (две ямы и два гребня) и четыре существенные. Справа та же функция с гаус-

совым шумом $\sigma = 0.05$: конечных точек стало 69 вместо двух, но все новые прижаты к диагонали, а «настоящие» точки сдвинулись мало.

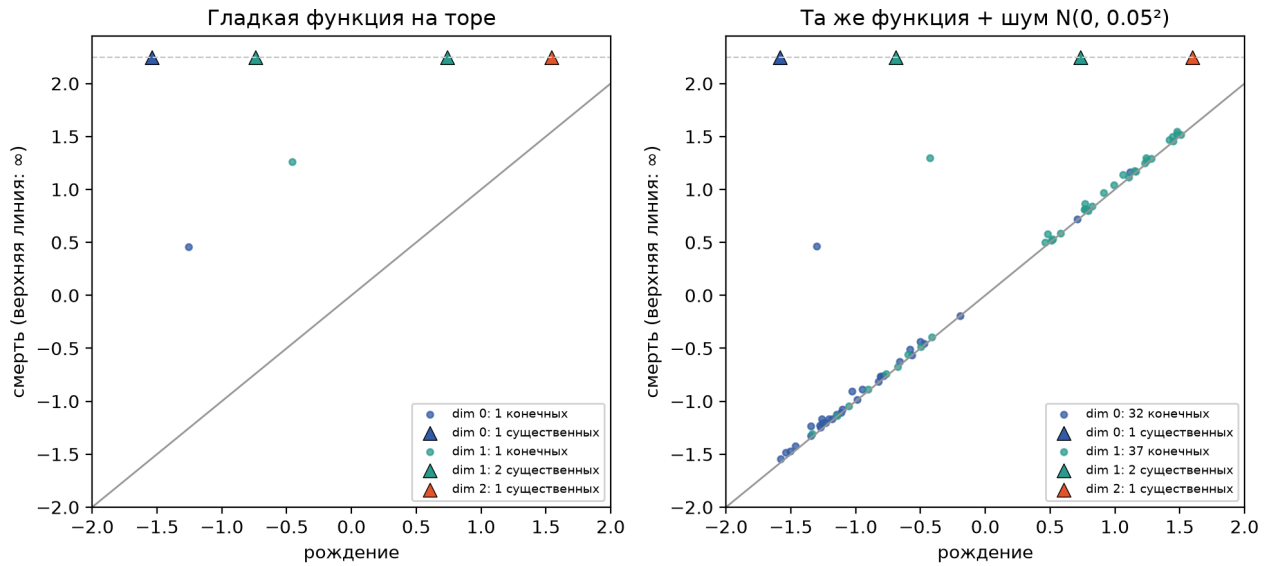


Рис. 8.2. Диаграммы персистентности гладкой функции на торе и той же функции с шумом. Треугольники на верхней линии — существенные классы.

Порог. Зададим $\tau \geq 0$ и будем считать значимыми пары с $d - b \geq \tau$ (существенные значимы всегда). Функция `persistent_counts()` считает, сколько критических точек каждого типа «выживает»:

```
for pair in pairs:
    if pair.persistence >= threshold:
        for vertex in (pair.birth_vertex, pair.death_vertex):
            if vertex is not None:
                significant[vertex] = significant.get(vertex, 0) + 1
...
hits = significant.get(point.vertex, 0)
counts[point.kind.value] += hits if point.kind == CriticalType.SADDLE else min(hits, 1)
```

Седло засчитывается столько раз, во скольких значимых парах оно участвует, экстремум — не больше одного раза. Почему сохраняется $\#min - \#saddle + \#max = \chi$ на замкнутой поверхности? При $\tau = 0$ учтены все пары, а каждая критическая вершина лежит ровно в стольких парах, какова её кратность (экстремум в одной, k -кратное седло в k). Это проверено численно на случайных полях на торе и следует из того, что нижняя звезда k -кратного седла меняет гомологии ровно на k единиц. Значит, при $\tau = 0$ это та самая сумма с кратностями из главы 5, равная χ . Каждая отброшенная конечная пара — это (минимум, седло) или (седло, максимум): она уменьшает на единицу ровно два слагаемых с противоположными знаками, и сумма не меняется. Существенные пары не отбрасываются никогда.

Пример (тор 4×4 выше): $\tau = 1 - 3/5/2$; $\tau = 3 - 2/3/1$ (отброшены пары с персистентностью 2 и 1); $\tau = 5 - 1/2/1$, как у идеального тора. Во всех случаях $\min - saddle + \max = 0$. На зашумлённом торе 128×128 (рис. 8.3) при $\sigma = 0.05$ от 1770/3544/1774 при $\tau = 0$ остаются 1/2/1 уже при $\tau \approx 0.2$.

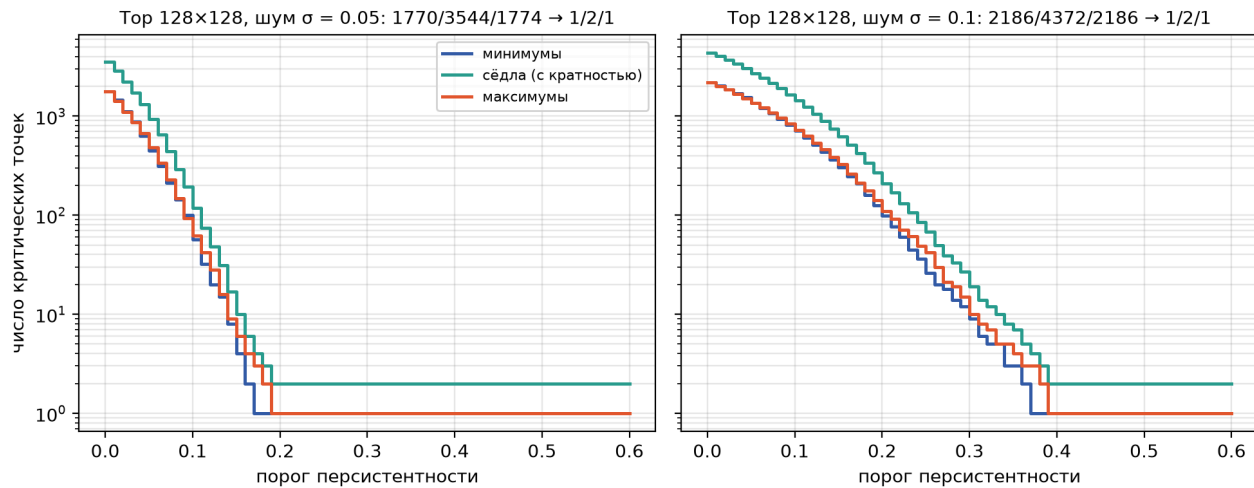


Рис. 8.3. Число критических точек в зависимости от порога персистентности для тора 128×128 с шумом. Счёт `persistent_counts()`, логарифмическая шкала.

Важно. `persistent_counts()` только фильтрует пары. Настоящее сокращение (cancellation) пары критических точек в комплексе Морса–Смейла удаляет обе точки, перестраивает градиентное поле и перенаправляет сепаратрисы: дуги, шедшие в удалённый минимум, продолжаются до выжившего. Кроме того, сократить за один шаг можно только пару, соединённую единственной сепаратрисой. В проекте ничего этого нет: сепаратрисы и граф в отчёте остаются построенными по всем точкам (см. `docs/DATA_CONTRACT.md`: «`persistent_counts` filters pairs only»).

Типичная ошибка. Сравнивать `counts` и `persistent_counts` при $\tau = 0$ и удивляться разнице в числе седел. `Descriptor.counts` считает мульти-седло один раз, а `persistent_counts` — с кратностью (как `saddle_index_sum`).

На открытой поверхности соотношение с χ не гарантировано: граничные вершины участвуют в фильтрации и могут входить в пары, но не классифицируются (`analyze_morse()` по умолчанию их маскирует), поэтому их пары никого не вычитают. Подробно — глава 10.

8.5 Пары в вершинах: `critical_pairs()`

GUDHI возвращает пары симплексов, а проекту нужны пары вершин, совпадающие с критическими точками из `critical.py`. Функция `critical_pairs()` решает это в три шага.

1. Фильтрация строится не по значениям, а по рангам SoS: `_lower_star_tree(mesh, order.astype(float))`, где `mesh.order` — ранг вершины при сортировке по (значение, индекс) (глава 5). Все вершины получают разные значения, ничьи на плато разрешаются так же, как в `analyze_morse()`, и пары согласованы с классификацией.
2. Симплекс отображается в вершину с максимальным рангом: при нижней звезде именно эта вершина «вносит» симплекс в фильтрацию. Рождающее ребро петли даёт седло, убывающий треугольник — максимум, убывающее ребро в размерности 0 — седло.
3. Пары нулевой длины отбрасываются (`death_vertex == birth_vertex`), а значения рождения и смерти берутся из `mesh.values`, то есть отчёт идёт в исходных единицах (метры, кривизна), а не в рангах.

Заметим про шаг 3: ранги различны, поэтому нулевая длина в рангах означает общую старшую вершину. GUDHI при `min_persistence=0` такие интервалы и так не возвращает (на полосе из 8.3: 3 пары против 45 при `-1`), так что проверка в коде — страховка. Пары нулевой длины в значениях (разные вершины с равным f) остаются, но они лежат на диагонали и на расстояние bottleneck не влияют.

Почему для максимумов не нужна отдельная фильтрация по $-f$? Внутренний максимум v появляется, когда весь его линк уже в M_t ; этот линк — петля, которую до появления v нельзя заклеить, и треугольники

звезды ν её убивают. Значит, каждый максимум уже есть в $Dgm_1(f)$ как точка смерти пары (седло, максимум), а глобальный максимум замкнутой поверхности — существенный класс размерности 2. Функция `mesh_persistence()` тем не менее считает два отчёта: `minima` по f и `maxima` по $-f$ (через `simplicial_persistence()`, без SoS и без вершин). На замкнутой поверхности это дублирование: конечная часть $Dgm_0(-f)$ совпадает с $Dgm_1(f)$ после замены $(b, d) \mapsto (-d, -b)$ (на зашумлённом торе 40×48 все 77 пар совпали). На открытой сетке 40×48 получилось 13 пар в $Dgm_1(f)$ и 18 в $Dgm_0(-f)$: лишние — граничные максимумы. Конвейер CLI `mesh_persistence()` не использует; это вспомогательная функция публичного API.

Функция `annotate_persistence()` записывает в каждую критическую точку наибольшую персистентность её пар (бесконечную для существенных; в JSON это `persistence: null, essential: true`). Критическая вершина без пары получает 0.

Для глубинной карты есть функция `cubical_persistence(): gudhi.CubicalComplex(top_dimensional_cells=...)`. Здесь пиксели — квадраты (2-клетки), а вершины и рёбра решётки получают минимум значений соседних пикселей. Поэтому пиксели, касающиеся углом, связаны сразу (для поля $\begin{pmatrix} 0 & 5 \\ 5 & 0 \end{pmatrix}$ — одна компонента), что не совпадает со связностью триангуляции из `grid_mesh()`; периодичность не учитывается. Функция `depth_persistence()` запускает её для f и $-f$. Пары не привязаны к критическим вершинам, и CLI их не использует.

8.6 Расстояние bottleneck и теорема устойчивости

Интуиция. Чтобы сравнить две диаграммы, сопоставим их точки друг другу; точку, которой нет пары, разрешим «утопить» в диагонали. Цена сопоставления — наибольшее смещение.

Определение. Дополним каждую диаграмму всеми точками диагонали (с бесконечной кратностью). Расстояние bottleneck:

$$d_B(X, Y) = \inf_{\gamma: X \rightarrow Y} \sup_{x \in X} \|x - \gamma(x)\|_\infty, \quad \|(b, d) - (b', d')\|_\infty = \max(|b - b'|, |d - d'|),$$

где инфимум берётся по биекциям. Ближайшая к (b, d) точка диагонали — $(\frac{b+d}{2}, \frac{b+d}{2})$, так что утопить точку стоит $(d - b)/2$. Пример: $X = \{(0, 1)\}$, $Y = \{(0, 1.2), (0.5, 0.6)\}$. Сопоставим $(0, 1)$ с $(0, 1.2)$ (цена 0.2), а $(0.5, 0.6)$ утопим (цена 0.05): $d_B = 0.2$, что и даёт `compare.bottleneck()`.

Теорема устойчивости (Cohen-Steiner, Edelsbrunner, Harer, 2007). Для функций f, g на одном и том же триангулируемом пространстве (в частности, для двух наборов значений на одной сетке с фильтрацией нижней звезды)

$$d_B(Dgm_k f, Dgm_k g) \leq \|f - g\|_\infty \quad \text{для каждого } k.$$

Смысл для сканов: если шум сдвигает каждую высоту не больше чем на ε , то каждая точка диаграммы сдвигается не больше чем на ε , а новые точки появляются только в полосе персистентности $\leq 2\varepsilon$ у диагонали. Рис. 8.4: к чистому полю тора добавлен равномерный шум амплитуды ε ; при любых ε точки d_B лежат ниже прямой $y = \|f - g\|_\infty$ (наибольшее отношение в расчёте 0.989).

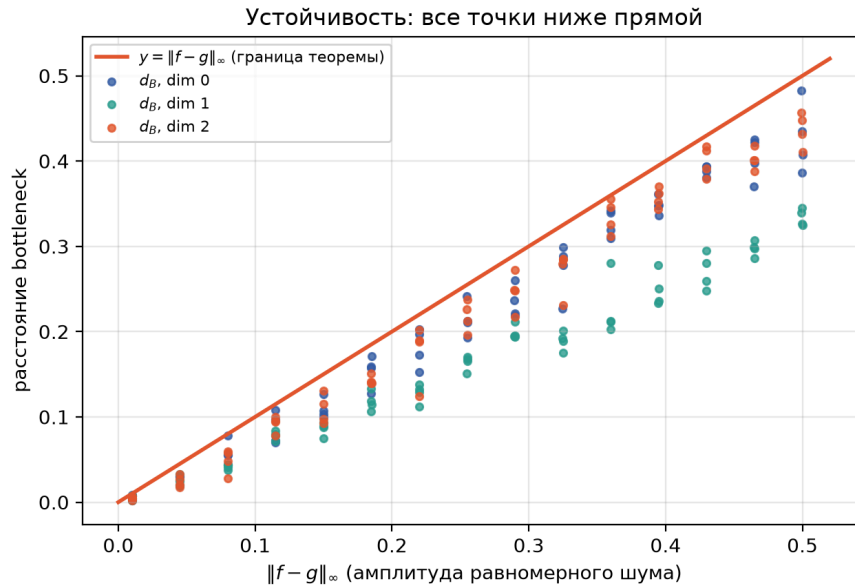


Рис. 8.4. Расстояние bottleneck между диаграммами чистого и зашумлённого поля на торе 40×48 в зависимости от амплитуды шума. Прямая — граница из теоремы устойчивости.

Чего теорема не обещает.

- Устойчивости счётчиков. Маленький бугорок добавляет пару максимум–седло: d_B меняется на половину её персистентности, а число критических точек — на 2. Поэтому `compare_reports()` сравнивает `counts` только «для наглядности», а `persistent_counts` сравнивает при фиксированном пороге. Но и они скажут, если у какой-то пары персистентность близка к τ .
- Ничего про несовмещённые сканы. В теореме f и g заданы на одном пространстве. Два скана дают разные сетки, и $\|f - g\|_\infty$ просто не определено; диаграммы сравнить можно, но гарантия пропадает. Хуже того, поле высоты зависит от позы: тот же объект, наклонённый на 10° , имеет другую функцию высоты, и разница может быть больше размеров деталей. Отсюда `--up-axis`, `--align pca` (глава 1) и инвариантные к движениям поля кривизны (глава 9).

Как это в коде: `compare.py`, функция `bottleneck()`:

```
finite = [[point for point in diagram if math.isfinite(point[1])] for diagram in (first, second)]
essential = [sorted(point[0] for point in diagram if not math.isfinite(point[1])) for diagram in (first, second)]
if len(essential[0]) != len(essential[1]):
    return math.inf
essential_distance = max((abs(left - right) for left, right in zip(*essential)), default=0.0)
return max(float(gudhi.bottleneck_distance(finite[0], finite[1])), essential_distance)
```

Существенную точку (b, ∞) нельзя утопить в диагонали (это стоило бы ∞), её можно сопоставить только другой существенной точке за $|b - b'|$. Если таких точек разное число, $d_B = \infty$ (в JSON `null`). Сопоставление отсортированных значений рождения оптимально для задачи «минимизировать наибольшее $|b - b'|$ » на прямой. Версия GUDHI 3.13 в окружении проекта и сама обрабатывает бесконечные точки так же (проверено), но явный код не зависит от версии.

Типичная ошибка. Получить `bottleneck: null` и решить, что сканы «бесконечно разные». Обычно это значит, что различается топология сетки: например, у одной из сеток есть лишняя связная компонента (лишняя существенная точка в размерности 0).

Функция `compare_reports()` отказывается сравнивать отчёты, если различаются `schema_version`, `parameters`, `scalar`, `geometry`, `surface` или `persistence_threshold`. Это правильно: диаграмма зависит от предобработки. Сглаживание `--sigma` срезает мелкие пары, `--voxel-size` меняет сетку, разные скалярные поля имеют разные единицы. Расстояние между отчётами с разными параметрами измеряло

бы разницу обработки, а не объектов. Порог на d_B не влияет (расстояние считается по полным диаграммам), но от него зависит `persistent_counts_equal`. Расстояния считаются по каждой размерности отдельно; `bottleneck_max` — максимум по размерностям.

8.7 Эксперимент повторных сканов

Цель — проверить, что дескриптор различает объекты, а не случайности съёмки. Модуль `experiment.py`, функция `run_experiment(manifest_path, output)`, читает JSON-манифест (пример `examples/repeat_scans.json`):

- `units` — `m`, `cm` или `mm`; пересчёта единиц нет, они только записываются;
- `parameters` — общие для всех сканов параметры CLI; обязательны `up_axis`, `geometry`, `scalar` и `persistence_threshold` (не `null`, нужен GUDHI); пути и устаревший `periodic` запрещены;
- `scans` — не меньше двух записей `{"path", "object"}`; хотя бы один объект должен встречаться дважды, пути разные.

Для каждого скана CLI запускается с одними и теми же параметрами (`scan_NNN.json` и лог; SHA-256 входа до и после обработки). Затем для всех пар $i < j$ вызывается `compare_reports()` с флагом `same_object` (совпадают ли метки; из имён файлов они не выводятся). В `results.json` попадают также версии пакетов и хеши исходников. Каталог вывода должен быть новым, старый прогон не перезаписывается.

Что хотим увидеть: расстояния внутри объекта (`same_object: true`) заметно меньше расстояний между объектами. Пример на синтетике: три скана поля A (тор, одна яма) и два скана поля B (две ямы), гауссов шум 0.03, порог 0.5:

пары	bottleneck_max	persistent_counts_equal
A–A (3 пары)	0.039–0.059	да, 1/2/1
B–B (1 пара)	0.038	да, 2/4/2
A–B (6 пар)	0.996–1.002	нет

Между объектами $d_B \approx 1$: у B есть пары с персистентностью около 2, которым в A нет партнёра, и их приходится утопить за половину персистентности. Демонстрация `examples/synthetic_demo.py` на облаках точек даёт в размерности 1: повторный скан 0.0011, тот же объект с добавленным выступом 0.031, другой объект 0.064.

Важно. `run_experiment()` только выписывает попарные расстояния. Статистику разделения (например, сравнение распределений внутри- и межобъектных расстояний) нужно считать отдельно по `results.json`. И помнить, что гарантии теоремы устойчивости для разных сканов нет (раздел 8.6): малые расстояния внутри объекта — экспериментальный факт, который надо проверять на реальных данных.

8.8 Вопросы для самопроверки

1. Почему у тора $\beta_1 = 2$ и как это видно в диаграмме, которую строит `critical_pairs()`?
2. Что пойдёт не так, если в `_lower_star_tree()` удалить цикл вставки рёбер?
3. Функция на отрезке $f = (2, 0, 3, 1, 4)$. Найдите пары персистентности по правилу старшего.
4. Почему отбрасывание пар в `persistent_counts()` не меняет `min – saddle + max` на замкнутой поверхности и почему седло считается с кратностью?
5. Почему `critical_pairs()` строит фильтрацию по рангам `mesh.order`, а не по `mesh.values`?
6. Два скана одного объекта сняты с разным наклоном телефона. Применима ли к ним теорема устойчивости?

Ответы

1. Тор имеет две независимые несжимаемые петли (вдоль и поперёк трубки). В диаграмме это две существенные точки размерности 1 (их рожают два седла), плюс по одной существенной в размерностях 0 и 2.
2. Рёбра будут созданы вместе с треугольниками и получают значение не меньше минимума значений смежных треугольников, а не максимум своих вершин. Фильтрация перестанет быть нижней звездой, появятся ложные пары (на сетке 12×12 — 15 интервалов вместо 2).
3. Компоненты: $x_1(0)$, $x_3(1)$. При $t = 3$ (x_2) они сливаются, умирает младшая: пара $(1, 3)$. Пара $(0, \infty)$ у x_1 . Вершины x_0 и x_4 ничего не меняют.
4. Каждая конечная пара — (минимум, седло) или (седло, максимум), её удаление уменьшает на 1 два слагаемых с разными знаками. k -кратное седло входит в k пар; если считать его один раз, сумма при $\tau = 0$ не совпала бы с χ .
5. Чтобы ничьи разрешались той же симуляцией простоты, что и в `analyze_morse()`: тогда вершины пар — те же критические вершины. Значения подставляются потом, для отчёта в исходных единицах.
6. Нет. Теорема требует двух функций на одном пространстве, а здесь разные сетки и, к тому же, функция высоты зависит от позы. Нужны совмещение (`--up-axis`, `--align pca`) или инвариантные поля (глава 9); и тогда малое расстояние — экспериментальный факт, а не следствие теоремы.

8.9 Что почитать

- H. Edelsbrunner, J. Harer. *Computational Topology: An Introduction*. AMS, 2010 — главы о гомологиях, персистентности и устойчивости.
- D. Cohen-Steiner, H. Edelsbrunner, J. Harer. *Stability of Persistence Diagrams*. *Discrete & Computational Geometry* 37 (2007), 103–120.
- Документация GUDHI: разделы Simplex tree, Cubical complex, Bottleneck distance (gudhi.inria.fr).

9. Геометрия поверхности, кривизна и скалярные поля

В главах 4–8 поверхность была «резиновой»: значение имели только вершины, рёбра, грани и порядок значений функции в вершинах. Но дескриптор Морса строится не для поверхности, а для пары «поверхность + скалярное поле», и само поле обычно берётся из геометрии: высота точки, кривизна в точке. Эта глава объясняет, какие геометрические величины считает `intrinsic.py`, что именно они измеряют и как ведут себя при повороте скана и шуме.

9.1 Метрика и топология

Интуиция. Возьмём лист бумаги. Его можно свернуть в цилиндр, не растягивая и не сминая. Длины кривых на листе при этом не меняются, а форма в пространстве меняется. Теперь возьмём резиновый лист и растянем его. Длины меняются, но лист остаётся диском: в нём не появилось дыр, и он не склеился сам с собой.

Строго. *Метрика* (metric) поверхности задаёт длины кривых и углы между ними, измеренные внутри поверхности. Отображение, сохраняющее длины всех кривых, называется *изометрией* (isometry); изгибание бумаги — изометрия. *Топология* (topology) сохраняется при любом гомеоморфизме, то есть взаимно однозначном и непрерывном в обе стороны отображении, включая растяжения. Получается три уровня:

Преобразование	Длины, углы	Гауссова кривизна K	Средняя кривизна H	Эйлерова характеристика χ
Поворот и сдвиг (движение)	сохраняются	сохраняется	сохраняется	сохраняется
Изгибание без растяжения (изометрия)	сохраняются	сохраняется	меняется	сохраняется
Растяжение (гомеоморфизм)	меняются	меняется	меняется	сохраняется

Для проекта это значит следующее. Число критических точек любой функции Морса на поверхности ограничено её топологией: $\# \text{min} - \# \text{saddle} + \# \text{max} = \chi$ (гл. 3–4). А сколько именно будет минимумов и седел и где они окажутся, зависит от выбранного поля. Это поле и несёт информацию о форме.

9.2 Скалярное поле: от его выбора зависит весь дескриптор

Скалярное поле (scalar field) на поверхности — функция $f: M \rightarrow \mathbb{R}$; в коде это массив `TriMesh.values`, одно число на вершину. Критические точки, сепаратрисы и пары персистентности считаются по `values`, а координаты `vertices` используются только для геометрии. Поэтому один и тот же скан с разными полями даёт совершенно разные дескрипторы. В CLI поле выбирается флагом `--scalar` из четырёх вариантов: `height`, `gaussian-curvature`, `mean-curvature`, `shape-index` (`intrinsic.py`, функция `intrinsic_scalar()`).

Поле высоты зависит от позы скана

Высота $f(\mathbf{p}) = z$ — это проекция на ось Z , то есть на направление, выбранное сканером, а не объектом. Если повернуть скан, функция f станет другой, и критические точки сдвинутся, появятся или исчезнут. Числовой пример (рис. 9.1): рельеф из двух бугров и одной ямы на сетке 61×61 с шагом 0,1. В исходной позе внутри 1 минимум, 1 седло и 2 максимума. Повернём ту же сетку на 20° вокруг оси y и возьмём новую координату z как поле. Получится 1 минимум, 3 седла и 2 максимума. После поворота на 45° внутри не остаётся ни одной критической точки: наклон «перевешивает» бугры, и все экстремумы уходят на край.

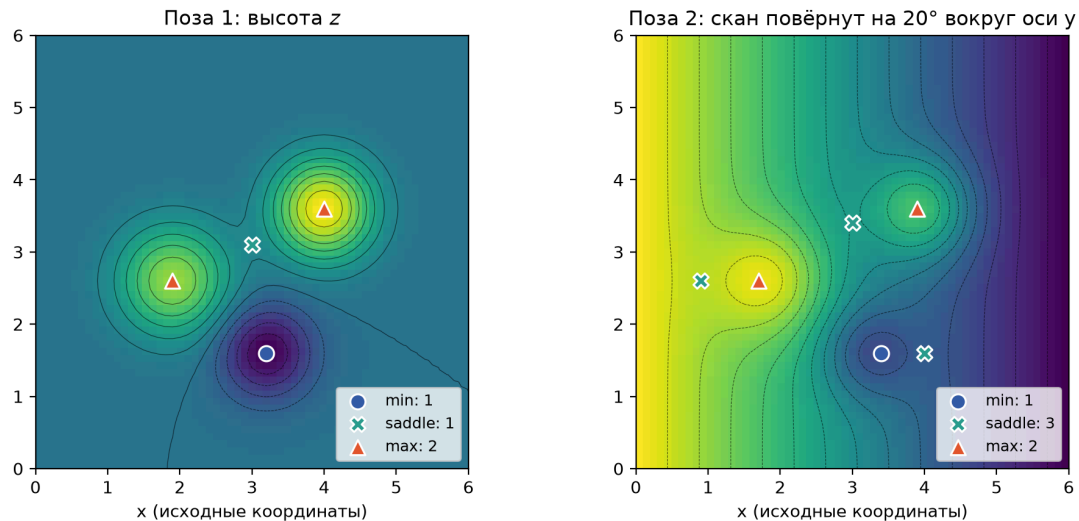


Рис. 9.1. Поле высоты до и после поворота скана на 20°. Сетка та же, изменилось только направление «вверх»; появились два новых седла (посчитано `critical.analyze_morse`).

В коде. Против зависимости от позы в `pointcloud.py` есть три средства, и `cli.py` (`_build_mesh()`) применяет их в таком порядке: 1. `set_up_axis(cloud, up)` переставляет оси так, чтобы указанная ось стала +Z. Это нужно для экспорта ARKit/iPhone, где «вверх» — это Y. 2. `remove_dominant_plane(cloud, distance)` методом RANSAC (500 итераций, `seed=0`) ищет плоскость с наибольшим числом точек в пределах `distance` и удаляет эти точки, например пол или стол. 3. `align_pca(cloud)` центрирует облако и поворачивает главные оси ковариации на X, Y, Z. Направление наименьшей дисперсии становится Z, его знак согласуется со старым +Z, а знак X выбирается по третьему моменту, чтобы результат был детерминирован.

После этого облако прореживается (`voxel_downsample`) и превращается в сетку.

Типичная ошибка. `align_pca` даёт устойчивую позу только тогда, когда собственные значения ковариации хорошо различаются (плоский или вытянутый объект). Для почти изотропного объекта оси PCA неустойчивы: небольшой шум поворачивает их, а вместе с ними меняется и поле высоты. Код такой случай не обнаруживает. Кроме того, если самый большой плоский участок — это сам объект (например, стена), `remove_dominant_plane` удалит именно его.

Второй путь — взять поле, которое от позы не зависит вообще. Это кривизна. Чтобы понять её, нужно несколько определений.

9.3 Нормаль, главные кривизны, H и K

Интуиция. Около гладкой точки поверхность почти совпадает со своей *касательной плоскостью* (tangent plane). Кривизна показывает, насколько быстро поверхность от этой плоскости отходит и в какую сторону.

Строго. Пусть $\mathbf{n}$ — единичная *нормаль* (normal) в точке $\mathbf{p}$, перпендикулярная касательной плоскости. Плоскость, которая содержит $\mathbf{n}$ и касательное направление $\mathbf{t}$, высекает из поверхности кривую — *нормальное сечение* (normal section). Её кривизна со знаком обозначается $k(\mathbf{t})$. В проекте принято соглашение `intrinsic.py`: $k > 0$, когда сечение загибается от нормали (центр кривизны лежит со стороны $-\mathbf{n}$), как у купола с нормалью наружу. Максимум и минимум $k(\mathbf{t})$ по направлениям называются *главными кривизнами* (principal curvatures) $k_1 \geq k_2$; соответствующие направления ортогональны. Из главных кривизн составляют две величины:

$$H = \frac{k_1 + k_2}{2}, \quad K = k_1 k_2.$$

H — средняя кривизна (mean curvature), измеряется в $1/\text{длина}$. K — гауссова кривизна (Gaussian curvature), измеряется в $1/\text{длина}^2$. Знак K не зависит от выбора нормали: $K > 0$ означает, что поверхность изогнута в одну сторону по всем направлениям (купол или чаша), $K < 0$ — что в разные стороны (седло). Знак H при смене нормали $\mathbf{n} \rightarrow -\mathbf{n}$ меняется на противоположный.

Поверхность	k_1	k_2	H	K
Сфера радиуса R (нормаль наружу)	$1/R$	$1/R$	$1/R$	$1/R^2$
Цилиндр радиуса R	$1/R$	0	$1/(2R)$	0
Седло $z = a(x^2 - y^2)$ в начале координат	$2a$	$-2a$	0	$-4a^2$
Плоскость	0	0	0	0

Пример с числами: на сфере радиуса $R = 10$ см получаем $H = 0,1 \text{ см}^{-1}$ и $K = 0,01 \text{ см}^{-2}$. Если перейти к метрам, получится $H = 10 \text{ м}^{-1}$ и $K = 100 \text{ м}^{-2}$. Поэтому `DATA_CONTRACT.md` требует одинаковых единиц у сравниваемых сканов: код единицы не пересчитывает.

9.4 Theorema Egregium и теорема Гаусса–Бонне

Theorema Egregium («замечательная теорема») Гаусса: K выражается только через метрику, то есть через длины и углы, измеренные внутри поверхности. Поэтому изгибание без растяжения K не меняет. Лист бумаги ($K = 0$) можно свернуть в цилиндр ($K = 0$, хотя $H = 1/(2R) \neq 0$), но нельзя без складок обернуть вокруг сферы ($K > 0$). H внутренней величиной не является: она зависит от того, как поверхность расположена в пространстве.

Теорема Гаусса–Бонне (Gauss–Bonnet) связывает геометрию с топологией. Для компактной поверхности M с краем ∂M

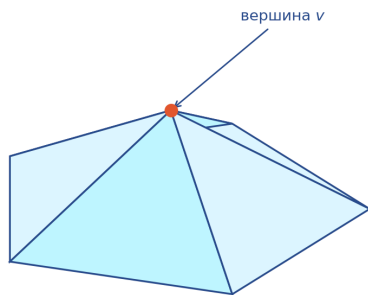
$$\int_M K dA + \int_{\partial M} k_g ds = 2\pi\chi(M),$$

где k_g — геодезическая кривизна (geodesic curvature) края, то есть его кривизна внутри поверхности. Для замкнутой поверхности второго слагаемого нет. Например, для сферы $\frac{1}{R^2} \cdot 4\pi R^2 = 4\pi = 2\pi \cdot 2$. Смысл теоремы: кривизну можно перераспределять по поверхности, но её полный запас фиксирован топологией. Из этого видно, что поле кривизны не может быть «произвольным»: у тора $\int K dA = 0$, поэтому на любом торе есть области и с $K > 0$, и с $K < 0$.

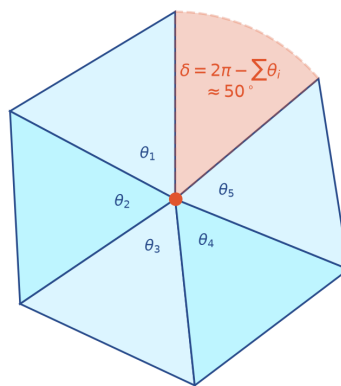
9.5 Дискретная гауссова кривизна: угловой дефект

Интуиция. Сложим углы всех треугольников, сходящихся в вершине v . Если звезда вершины плоская, сумма равна 2π . Если вершина — «кончик» конуса, сумма меньше 2π : развернув звезду на плоскость, получим зазор (рис. 9.2). Если звезда «гофрированная», как седло, сумма больше 2π .

Звезда вершины в пространстве



Развёртка на плоскость: зазор = угловой дефект

**Рис. 9.2.** Угловой дефект. Пять треугольников вокруг поднятой вершины разворачиваются на плоскость с зазором δ .

Строго. Угловой дефект (angle defect) вершины с углами треугольников θ_j при ней:

$$\delta(v) = 2\pi - \sum_j \theta_j \quad (\text{внутренняя вершина}), \quad \delta(v) = \pi - \sum_j \theta_j \quad (\text{границная вершина}).$$

Дефект — это «полная кривизна» маленького куска поверхности около вершины. Чтобы получить кривизну в точке, его делят на площадь, приписанную вершине. В проекте это *барицентрическая площадь* (barycentric area): каждой вершине достаётся треть площади каждого прилегающего треугольника, так что $\sum_v A_v$ равна площади всей сетки.

$$K(v) = \frac{\delta(v)}{A_v}, \quad A_v = \sum_{t \ni v} \frac{\text{area}(t)}{3}.$$

Дискретная теорема Гаусса–Бонне выполняется *точно*: $\sum_v \delta(v) = 2\pi\chi$. Граничный дефект $\pi - \sum \theta_j$ играет роль дискретной $k_g ds$. Проверка на открытом рельефе 41×41 из гл. 10: внутренние дефекты в сумме дают $-0,0013$, граничные $6,2845$, итого $\sum \delta/2\pi = 1,0000000$, что совпадает с χ диска.

В коде. `mesh.py`, `TriMesh.vertex_areas` — барицентрическая площадь. `intrinsic.py`, `_corner_geometry()` считает угол при каждом углу треугольника через `arctan2(|a×b|, a·b)` (это точнее, чем `arccos`) и котангенс `cos/sin`. `_raw_gaussian_curvature()` суммирует углы по вершинам через `pr.add.at`, вычитает их из цели 2π (для граничных вершин π) и делит на площадь. Наружу, через `gaussian_curvature()`, отдаётся уже результат `_extend_to_boundary()`: значения на краю заменяются продолжением изнутри (гл. 10), поэтому цель π в итоговом поле не участвует.

9.6 Дискретная средняя кривизна: котангенсный оператор Лапласа–Бельтрами

Интуиция. Если каждую точку поверхности сместить к среднему её соседей, купол «просядет» внутрь. Вектор этого сдвига направлен по нормали, и его длина пропорциональна H .

Строго. Оператор Лапласа–Бельтрами (Laplace–Beltrami operator), применённый к координатной функции $\mathbf{x}$ поверхности, даёт $\Delta \mathbf{x} = -2H\mathbf{n}$. Дискретная версия — *котангенсная формула* (cotangent formula):

$$\Delta \mathbf{x}_i \approx \frac{1}{2A_i} \sum_{j \sim i} (\cot \alpha_{ij} + \cot \beta_{ij})(\mathbf{x}_j - \mathbf{x}_i),$$

где α_{ij}, β_{ij} — углы, противолежащие ребру ij в двух прилегающих треугольниках. Отсюда $H_i = -\langle \Delta \mathbf{x}_i, \mathbf{n}_i \rangle / 2$.

```
def _raw_mean_curvature(mesh, corners, cotangents):
    laplacian = np.zeros_like(mesh.vertices)
    for corner in range(3):
        following, previous = (corner + 1) % 3, (corner + 2) % 3
        # Edge (corner, following) is opposite ``previous`` and vice versa.
        contribution = cotangents[:, previous, None] * (corners[:, following] - corners[:, corner]) \
            + cotangents[:, following, None] * (corners[:, previous] - corners[:, corner])
        np.add.at(laplacian, mesh.faces[:, corner], contribution)
    # Laplace-Beltrami of the embedding: laplacian / (2A) = -2 H n.
    return -np.einsum("ij,ij->i", laplacian, mesh.vertex_normals) / (4.0 * _safe_area(mesh))
```

Разбор. Каждый треугольник добавляет своей вершине `corner` два слагаемых: ребро к `following` с котангенсом противолежащего угла `previous` и ребро к `previous` с котангенсом угла `following`. После суммирования по всем треугольникам каждое ребро ij получает ровно $\cot\alpha + \cot\beta$, и `laplacian` равен сумме в формуле без множителя $1/(2A)$. Значит, $\langle \text{laplacian}, \mathbf{n} \rangle = 2A \cdot (-2H)$, и $H = -\langle \cdot \rangle / (4A)$, что и написано в последней строке. Нормаль `vertex_normals` — сумма ненормированных векторных произведений граней (то есть взвешенная по площади), затем нормированная.

Важно. Знак H задаётся ориентацией граней. При обходе вершин против часовой стрелки нормаль грани $(\mathbf{b} - \mathbf{a}) \times (\mathbf{c} - \mathbf{a})$ направлена «наружу купола», и тогда $H > 0$. Проверка: на икосфере с наружной ориентацией среднее $H = +0,5$ при $R = 2$; если перевернуть все грани (`faces[:, [0, 2, 1]]`), получится $-0,5$, а K не меняется. Для полей высоты нормали направлены к $+Z$: `grid_mesh` строит грани (a, b, d) и (a, d, c) с нормалью вверх, а `reconstruct_delaunay` явно переворачивает треугольники с $n_z < 0$, потому что SciPy ориентацию не гарантирует. Для Poisson-сетки ориентация берётся из Open3D, и код её не проверяет.

9.7 Shape index Кендеринка

H и K зависят от масштаба. *Shape index* (индекс формы) Кендеринка от масштаба не зависит и описывает только «тип» формы:

$$S = \frac{2}{\pi} \arctan \frac{k_1 + k_2}{k_1 - k_2} \in [-1, 1].$$

$S = +1$ — купол (сар, $k_1 = k_2 > 0$), $S = 0,5$ — цилиндр-«хребет», $S = 0$ — симметричное седло, $S = -0,5$ — желоб, $S = -1$ — чаша (сир). При растяжении поверхности в λ раз H делится на λ , K — на λ^2 , а S не меняется.

Главные кривизны выражаются через H и K : $k_{1,2} = H \pm \sqrt{H^2 - K}$. Значит, $k_1 + k_2 = 2H$ и $k_1 - k_2 = 2\sqrt{H^2 - K}$, откуда

$$S = \frac{2}{\pi} \arctan 2 \left(H, \sqrt{\max(H^2 - K, 0)} \right).$$

Именно так это написано в `intrinsic.py`, функция `shape_index()`. Обе детали в этой формуле важны. `arctan2` корректно обрабатывает омбилическую точку $k_1 = k_2$, где знаменатель равен нулю: при $H > 0$ получается $S = 1$. `max(·, 0)` нужен потому, что дискретные H и K считаются разными формулами и согласованы не точно, так что $H^2 - K$ может оказаться немного отрицательным. Рис. 9.3 показывает H и S на синтетическом рельефе: вершины бугров — купола (красное), дно ямы — чаша (синее), между ними седла и желоба.

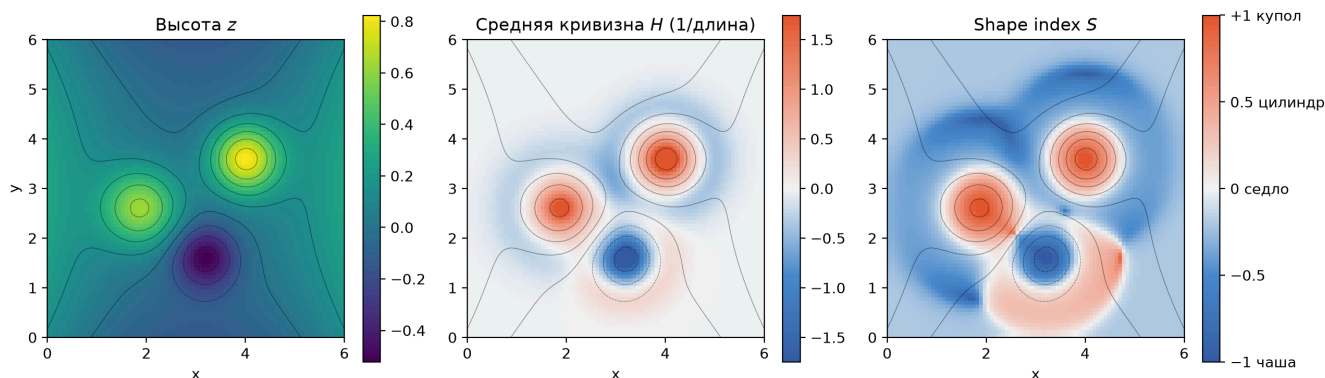


Рис. 9.3. Высота, средняя кривизна и shape index на синтетическом рельефе (сетка 81×81 , шаг 0,075; поля посчитаны `intrinsic.mean_curvature` и `intrinsic.shape_index`).

Типичная ошибка. На почти плоском участке H и K малы, и S определяется ошибками округления. При повороте рельефа из рис. 9.1 на 20° поля K и H совпали до $4 \cdot 10^{-13}$, а S в 92 почти плоских вершинах (там $|H| < 1,3 \cdot 10^{-6}$) изменился до 0,98. Строка документации «плоские вершины получают 0» верна только для *точного* нуля, когда $\arctan2(0, 0) = 0$.

9.8 Инвариантность к повороту и чувствительность к шуму

K , H и S строятся из длин, углов и площадей треугольников, а эти величины поворот не меняет. Поэтому для повторных сканов одного объекта, снятых с разных сторон, поле кривизны даёт одинаковые критические точки, если совпадает триангуляция. В опыте из п. 9.7 поле H после поворота дало те же 24 минимума, 31 седло и 10 максимумов. Оговорка: и растр, и `reconstruct_delaunay` триангулируют проекцию на плоскость XY , поэтому при повороте облака меняется и сама сетка. Тогда поля кривизны совпадают лишь приближённо, и тем точнее, чем мельче сетка и меньше шум (п. 9.9 показывает, что поточечная оценка зависит от формы треугольников).

Цена инвариантности — вторые производные. Шум глубины амплитуды ε при шаге сетки h даёт ошибку кривизны порядка ε/h^2 . Опыт на сферической шапке $R = 10$ с белым шумом $\varepsilon = 10^{-3}$ (средняя квадратичная ошибка по внутренним вершинам):

Шаг h	Ошибка H , без сглаживания	Ошибка H , sigma=1	Ошибка K , без сглаживания	Ошибка K , sigma=1
0,05	0,837	0,144	0,820	0,046
0,1	0,213	0,093	0,068	0,022
0,2	0,051	0,066	0,011	0,013
0,4	0,014	0,047	0,003	0,008

Без сглаживания ошибка H уменьшается в 4 раза при каждом удвоении шага, как и предсказывает оценка ε/h^2 . При крупном шаге гауссово сглаживание уже вредит: оно размывает саму сферу, и смещение (bias) становится больше шума. На рельефе 61×61 с шумом 0,01 поле H без сглаживания даёт 571 минимум, 1005 сёдел и 600 максимумов, при `sigma=3` — 33, 40 и 25. У высоты в тех же условиях было 306, 587 и 309 точек: шум бьёт по кривизне сильнее.

В коде. Есть два средства. `preprocess.py`, `denoise_depth()` применяет медианный фильтр (`--median-size`, нечётный) и затем гауссов (`--sigma`, в пикселях растра); режим края `nearest` для открытой поверхности и `wrap` для периодической. Работает он только для `--geometry raster`: для Delaunay/Poisson `cli._validate()` отклоняет ненулевые `--sigma` и `--median-size`, потому что сглаживания сетки в коде нет. `pointcloud.py`, `voxel_downsample()` оставляет в каждом вокселе *первую по индексу* точку (`np.unique(..., return_index=True)`), а не среднее. Шум отдельной точки это не уменьшает, но увеличивает шаг h , а значит, уменьшает ε/h^2 . Ни один из этих способов не заменяет фильтрацию по персистентности (гл. 8).

9.9 Проверка на сфере

Сравним дискретные K и H с точными $1/R^2 = 0,25$ и $1/R = 0,5$ при $R = 2$. Икосфера строится делением граней икосаэдра на четыре части с проекцией новых вершин на сферу; UV-сфера — сетка по широте и долготе с двумя полюсами. Сетку обернули в `TriMesh(V, F, V[:, 2])`.

Сетка	Вершин	K : медиана	K : худшая вершина	H : медиана	H : худшая вершина	$\sum KA_v/2\pi$
Икосфера, 1 деление	42	0,2613	0,2949 (валентность 5)	0,4839	0,5512	2,000000
Икосфера, 2 деления	162	0,2522	0,2885	0,4950	0,5673	2,000000
Икосфера, 4 деления	2562	0,2501	0,2866	0,4997	0,5726	2,000000
UV-сфера 64×128	8066	0,2502	0,1877 (полюс)	0,5001	0,3752	2,000000
Шапка $z = \sqrt{100 - x^2 - y^2}$, <code>grid_mesh</code> , $h = 0,1$	3721	центр 0,0100000 (точно 0,01)	—	центр 0,0999997 (точно 0,1)	—	—

Выводы. Интеграл кривизны точен всегда, это дискретная теорема Гаусса–Бонне ($\chi = 2$). Типичная вершина сходится к точному значению. Но в 12 вершинах валентности 5 икосферы ошибка около +15%, а на полюсе UV-сферы около −25%, и при измельчении сетки эти ошибки *не уменьшаются*. Причина в барицентрической площади: она точна для регулярных сеток вроде `grid_mesh` (тест `tests/test_curvature_accuracy.py`), но не равна «правильной» площади Вороного у нерегулярных вершин. По той же причине в вершинах валентности 5 shape index равен 0,78 вместо 1: ошибки H и K не согласованы, и сфера «выглядит» местами не как купол. Для Delaunay-сеток реальных сканов это значит, что поточечная кривизна содержит шум триангуляции.

9.10 Вопросы для самопроверки

1. Какие из величин K , H , S , χ сохраняются при сворачивании листа в цилиндр, а какие при растяжении резинового листа?
2. Почему при повороте скана меняются критические точки поля высоты, но не поля K ? Почему для растра и Delaunay это верно лишь приближённо?
3. Чему равна сумма угловых дефектов по всем вершинам открытой сетки-диска и почему это не зависит от формы рельефа?
4. Что произойдёт со знаком `mean_curvature` и `shape_index`, если перевернуть ориентацию всех граней? А с `gaussian_curvature`?
5. Зачем в `shape_index()` стоят `arctan2` и `np.maximum(..., 0)`?

6. Почему уменьшение шага сетки вдвое без сглаживания примерно учетверяет шум кривизны?

Ответы

1. При изгибании сохраняются K и χ , а H и S меняются: у плоскости $H = 0$ (код даёт $S = 0$), у цилиндра $H = 1/(2R)$ и $S = 0$, 5. При растяжении сохраняется только χ .
2. Высота — это проекция на внешнюю ось, а K строится из длин и углов, которые поворот не меняет. Растр и Delaunay строятся в проекции на XY , поэтому поворот облака меняет саму сетку, а с ней и дискретную оценку кривизны.
3. $2\pi\chi = 2\pi$ (если граничные вершины считать с целью π). Это дискретная теорема Гаусса–Бонне: сумма углов по всем треугольникам равна πF , а остальное — комбинаторика $V - E + F$.
4. H и S сменяют знак (купол станет чашей), K не изменится.
5. `arctan2` корректно обрабатывает омбилическую точку $k_1 = k_2$, где знаменатель равен нулю. `maximum` нужен, потому что дискретные H и K не согласованы точно и $H^2 - K$ может оказаться немного отрицательным.
6. Кривизна — это вторая производная; разностная оценка второй производной от шума амплитуды ε имеет порядок ε/h^2 .

9.11 Что почитать

- М. Р. do Carmo. *Differential Geometry of Curves and Surfaces*. Prentice-Hall, 1976 — главные кривизны, Theorema Egregium, Гаусс–Бонне.
- М. Meyer, М. Desbrun, Р. Schröder, А. Н. Barr. *Discrete Differential-Geometry Operators for Triangulated 2-Manifolds*. In: Visualization and Mathematics III, Springer, 2003 — котангенсная формула, угловой дефект, площади Вороного и барицентрические.
- J. J. Koenderink, A. J. van Doorn. *Surface shape and curvature scales*. Image and Vision Computing 10(8), 1992 — shape index и curvedness.

10. Граничные эффекты и открытые поверхности

Теория из глав 3–6 сформулирована для замкнутых поверхностей: сферы, тора. Реальный скан почти никогда не бывает замкнутым. В этой главе разбирается, что на открытой поверхности перестаёт работать, как код это маскирует и какие расхождения остаются.

10.1 Почему скан — открытая поверхность

LiDAR видит объект с одной стороны, поэтому обратная сторона в скан не попадает. Поле зрения обрезает сцену по краю. Блестящие, тёмные и прозрачные участки не возвращают сигнал, и в скане появляются дыры. Наконец, `remove_dominant_plane` удаляет пол, и там, где объект стоял на полу, появляется новый край. Итог — *поверхность с краем* (surface with boundary).

То, какой край увидит код, зависит от способа построения сетки:

Геометрия	Край сетки	Дыры в данных
<code>raster (grid_mesh)</code>	всегда прямоугольник растра	закрашиваются ближайшим значением (<code>_fill_nearest</code>); долю реально покрытых ячеек показывает <code>raster_coverage</code>
<code>delaunay</code>	выпуклая оболочка проекции точек на XY	затягиваются длинными треугольниками Delaunay; отдельной границей не становятся
<code>poisson, density_quantile > 0</code>	там, где обрезаны вершины с низкой плотностью	становятся настоящими краями
<code>poisson, density_quantile = 0</code>	края нет, сетка замкнута	«дорисованы» реконструкцией

10.2 Граничные вершины и рёбра

Напомним гл. 2. Ребро называется *граничным* (boundary edge), если оно принадлежит ровно одному треугольнику; во внутренней части многообразия ребро принадлежит двум. *Граничная вершина* — конец граничного ребра. Линк внутренней вершины — цикл, а линк граничной вершины — путь (path).

В коде. `mesh.py`: `edge_counts` считает, сколько граней содержит каждое ребро; `boundary_edges` — рёбра с числом 1; `boundary_vertices` — их концы; `closed` истинно, когда у всех рёбер число 2. Все эти свойства кэшируются (`cached_property`). Для сетки 41×41 граничных вершин $4 \cdot 41 - 4 = 160$, $\chi = 1$.

10.3 Что ломается на границе

Классификатор `critical.analyze_morse()` считает компоненты нижнего и верхнего линка (гл. 5). Для пути это правило даёт другой смысл:

- если нижний линк пуст, вершина — минимум, как и раньше;
- если верхний линк пуст, классификатор пишет «максимум». Но нижний линк при этом — весь путь, а путь стягиваем. Добавление такой вершины к подуровню топологию не меняет, то есть это *не* критическая точка;
- если нижний линк — одна средняя часть пути, а верхний — два конца, классификатор пишет «седло», хотя топологически вершина снова регулярна.

Аккуратный подсчёт выглядит так. При добавлении вершины v с нижним линком L_v эйлера характеристика подуровня меняется на $1 - \chi(L_v)$. Сумма этих вкладов по всем вершинам равна $\chi(M)$ для любой сетки, открытой или замкнутой:

$$\sum_v (1 - \chi(L_v)) = \chi(M).$$

Для внутренней вершины пустой линк ($\chi = 0$) и полный цикл ($\chi = 0$) дают вклад $+1$, а k дуг дают $1 - k$. Для граничной вершины пустой линк даёт $+1$, но полный путь ($\chi = 1$) даёт 0 , а k дуг дают $1 - k$. Проверка на четырёх рельефах (наклонный рельеф ниже и три случайных поля 40×40): сумма каждый раз равна 1 . На краю 20 случайных полей классификатор нашёл 152 «максимума» и 48 «сёдел» с одной нижней компонентой; все они дают вклад 0 и ни в одну пару персистентности не входят.

Второй эффект — *ложные экстремумы* (spurious extrema). У краевой вершины просто нет соседей с внешней стороны. На наклонном рельефе любая вершина нижнего края, которая ниже своих двух соседей по краю и внутреннего соседа, становится минимумом, хотя склон на самом деле продолжается дальше.

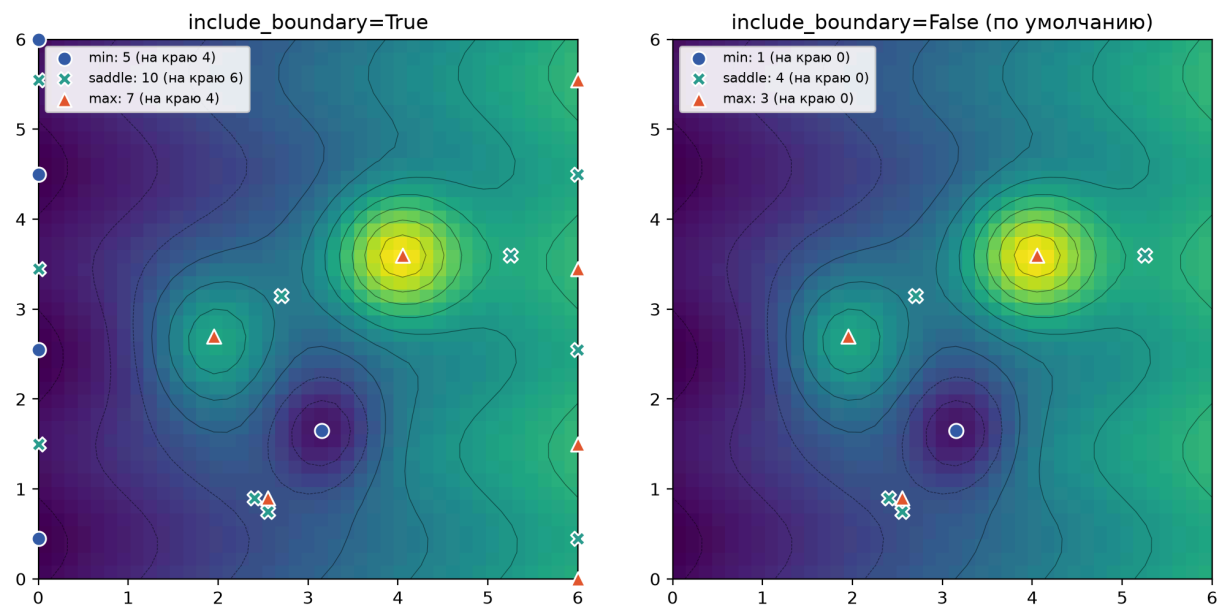


Рис. 10.1. Наклонный рельеф 41×41 ($z = \text{бугры} + 0,15x + 0,05\sin 3y$). Слева классифицированы все вершины: 14 из 22 критических точек лежат на краю. Справа — режим по умолчанию (посчитано `analyze_morse`).

Числа для рис. 10.1:

Режим	min	saddle	max	min – saddle + max	χ сетки
<code>include_boundary=True</code>	5	10	7	2	1
<code>include_boundary=False</code>	1	4	3	0	1

Ни один режим не даёт χ , и это не ошибка кода: для поверхности с краем формула $\#\text{min} - \#\text{saddle} + \#\text{max} = \chi$ по внутренним точкам не обязана выполняться. Недостающую часть вносит край, и правильный учёт — это формула с $1 - \chi(L_v)$ выше.

10.4 Маскирование в коде

В коде. `analyze_morse(mesh, include_boundary=False)` по умолчанию пропускает граничные вершины, и они вообще не попадают в список критических точек. `descriptor.build_descriptor()` вызывает его именно так. В `cli.main()` ожидаемая эйлерова характеристика задаётся только для замкнутой сетки:

```
expected_euler = mesh.euler_characteristic if mesh.closed else None
```

При `None` функция `validate_euler()` не вызывается, и в JSON попадает `"euler_characteristic": null`. Для замкнутой сетки проверка обязательна и при нарушении бросает `ValueError`.

Больше никакого маскирования края в коде нет. Поиск по `rim`, `mask`, `crop` находит только `include_boundary`, комментарии и маски RANSAC/Poisson. «Ширина обода» тоже не настраивается: маскируется ровно одно кольцо, то есть вершины граничных рёбер. Обрезки по области (`crop`) нет. Вершина рядом с краем, у которой есть хотя бы один внутренний сосед, классифицируется как обычно. Если нужно отрезать полосу у края, это придётся дописать (например, удалить грани у края до построения `Tri-Mesh`); сейчас такой функции нет.

Типичная ошибка. Поле `CriticalPoint.is_on_boundary` у эвристического бэкенда всегда `False`: `analyze_morse` его не заполняет даже при `include_boundary=True`. Заполняет его только ТТК (`ttk_backend.py`). Определять краевые точки нужно по `vertex in mesh.boundary_vertices`.

10.5 Сепаратрисы, уходящие в край

`separatrices._trace()` при спуске идёт от седла к соседу с *наибольшим* рангом SoS среди тех, что ниже текущей вершины, пока такой сосед есть. Это самый пологий монотонный шаг (ближайший по рангу сосед), а не спуск к самому низкому или самому крутому по метрике соседу (гл. 6.6). Дойдя до края, трасса не останавливается: она продолжает идти по граничным вершинам, пока не упрётся в локальный экстремум всей сетки. Если конечная вершина граничная и её нет в списке критических точек (она замаскирована), дуга получает `ends_on_boundary=True`. Если конечная вершина внутренняя, но не экстремум нужного типа, бросается `RuntimeError`.

В JSON (`DATA_CONTRACT.md`) у такой дуги `destination_id: null`, потому что ссылаться не на что, а `destination_vertex_id` хранит индекс граничной вершины. В сводке отчёта `boundary_separatrix_count` — число таких дуг.

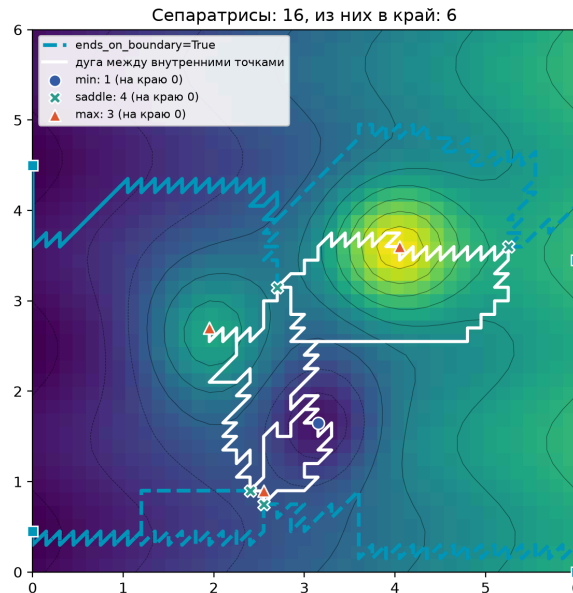


Рис. 10.2. Сепаратрисы эвристического бэкенда на наклонном рельефе. Сплошные белые дуги соединяют внутренние точки, пунктирные голубые — `ends_on_boundary` (квадрат — граничный конец).

На рис. 10.2 из 16 дуг 6 уходят в край. У этих 6 дуг последние 2–7 вершин лежат на краю, то есть дуга действительно «ползёт» вдоль обода. Отбрасывать такие дуги нельзя: у простого седла должно быть ровно две нисходящие и две восходящие дуги (тест `test_open_surface_arcs_to_the_rim_are_flagged_not_dropped`).

10.6 Кривизна на краю

Угловой дефект и котангенсная формула (гл. 9) требуют полной звезды вершины, а на краю звезда неполная. Что получается с сырыми формулами на сетке 41×41 (рис. 10.3):

- у углов раstra сумма углов равна $\pi/2$ или $\pi/4$, и сырая K достигает 203 и 416 при максимуме внутри 9,7;
- сырая H на краю почти нулевая ($|H| \leq 0,22$, в углах $\sim 10^{-6}$): вектор Лапласа там лежит почти в плоскости и перпендикулярен нормали. Получается ложное «плоское» кольцо.

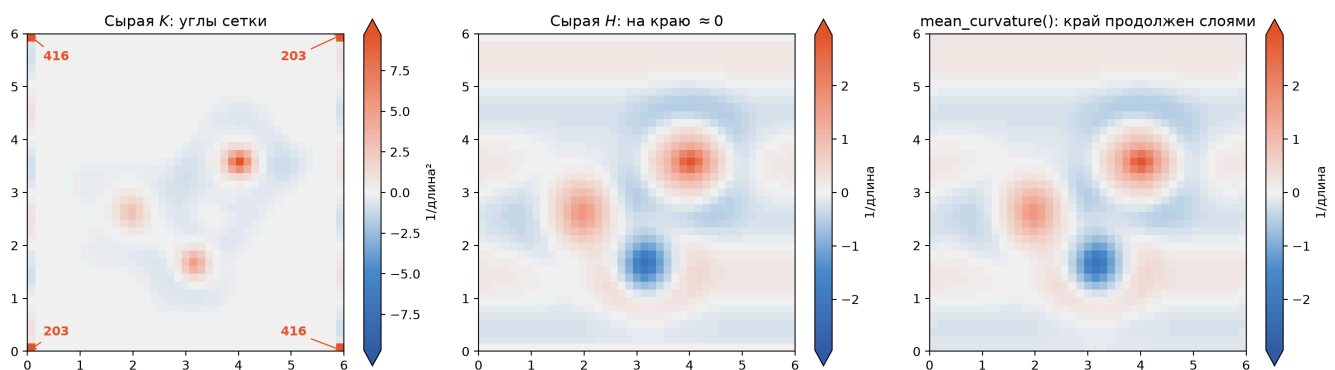


Рис. 10.3. Сырая K (выбросы в углах сетки), сырая H (ложный ноль на краю) и итоговая `mean_curvature()`.

Поэтому все три поля кривизны проходят через `intrinsic._extend_to_boundary()`:

```

unresolved = set(boundary)
while unresolved:
    updates = {}
    for vertex in sorted(unresolved):
        known = sorted(neighbors[vertex] - unresolved)
        if known:
            updates[vertex] = float(np.mean(result[known]))
    if not updates:
        raise ValueError("curvature boundary component has no interior vertices")
    for vertex, value in updates.items():
        result[vertex] = value
    unresolved.difference_update(updates)

```

Разбор. Граничные вершины заполняются *слоями по графовому расстоянию* от внутренних. В первом слое — граничные вершины, у которых есть внутренний сосед; каждая получает среднее по внутренним соседям. Обновления сначала собираются в `updates` и только потом записываются, поэтому слой синхронный и результат не зависит от порядка обхода. Сырая кривизна края в среднее не входит никогда: в `known` попадают только вершины вне `unresolved`. Это важно для углов. В триангуляции `grid_mesh` два угла, (0, последний столбец) и (последняя строка, 0), принадлежат одному треугольнику, и оба их соседа граничные. Они получают значение во втором слое, от уже заполненных соседей. Если у компоненты края внутренних вершин нет совсем (одиночный треугольник, полоса шириной в одну грань), цикл не может продвинуться, и бросается `ValueError` (тест `test_boundary_only_component_rejected`).

Важно. `DATA_CONTRACT.md` прямо говорит: кривизна на ободке — экстраполяция, а не измерение. Для `shape_index` продолжается уже готовое значение S , а не H и K по отдельности.

10.7 Персистентность на открытой поверхности

`persistence.critical_pairs()` строит фильтрацию нижней звезды по *всей* сетке, включая край, с тем же порядком SoS. Маска `include_boundary` на неё не действует. Значит, пары могут содержать граничные вершины, которых нет в списке критических точек; README об этом предупреждает. Проверка на наклонном рельефе: из 8 пар 4 содержат вершину края. Существенный класс H_0 (глобальный минимум) рождается на краю, поэтому ни одна внутренняя точка не получает `essential: true`, а единственный внутренний минимум имеет конечную персистентность 0,304. По 20 случайным полям 40×40 :

Тип пары (рождение → смерть)	Число
внутр. min → внутр. седло	114
краевой min → внутр. седло	103
внутр. min → краевое седло	6
краевой min → краевое седло	45
краевой min, существенная	20 из 20
внутр. седло → внутр. max	120
краевое седло → внутр. max	18

Следствия. `annotate_persistence()` даёт внутреннему седлу персистентность пары с замаскированным краевым минимумом. `persistent_counts()` засчитывает такое седло, но не его партнёра. Поэтому на открытой поверхности величина «min – saddle + max» в `persistent_counts` ничему не обязана равняться: в примере при пороге 0,05 получается $1 - 3 + 2 = 0$ при $\chi = 1$. Краевые «максимумы» и «сёдла» с вкладом 0 из п. 10.3 в пары не входят, что согласуется с формулой $1 - \chi(L_v)$. Расхождение между замаскированным списком и парами в коде не исправлено, оно только задокументировано.

10.8 Как закрыть поверхность

- `--surface periodic (periodic_grid_mesh)` склеивает противоположные стороны растра в тор, $\chi = 0$. Работает только с `raster`. Для облака точек CLI печатает предупреждение о шве, а кривизну на периодической сетке отклоняет, потому что треугольники шва «перекидывают» координаты через весь растр. На наклонном рельефе периодическая сетка даёт $4 - 10 + 6 = 0 = \chi$, но 12 из 20 критических точек лежат на строках и столбцах шва. Режим годится для синтетики, а не для сканов.
- `--geometry poisson --poisson-density-quantile 0` даёт водонепроницаемую сетку Open3D. Цена — поверхность, «дорисованная» там, где данных не было: топология честная, но часть геометрии выдумана. По умолчанию `density_quantile = 0,02`, и сетка снова открывается.
- ТТК-бэкенду нужна замкнутая сетка. `cli.main()` проверяет `mesh.closed` и при открытой сетке завершает работу с подсказкой о двух вариантах выше. Причина в цветном трёхвалентном графе (гл. 7): каждая дуга квадрангуляции Морса–Смейла должна разделяться ровно двумя треугольниками, иначе `ttk_backend` бросает «TTK quadrangulation is not closed».

10.9 Типичные ошибки и симптомы

Симптом	Вероятная причина	Что проверить
Много экстремумов и седел вдоль края	<code>include_boundary=True</code> или наклон скана	режим маски; <code>--align pca</code> , <code>--remove-plane</code>
Сепаратрисы «обрываются» на краю	<code>ends_on_boundary</code> , <code>destination_id: null</code>	<code>boundary_separatrix_count</code> ; это не ошибка
<code>euler_characteristic: null</code>	открытая сетка	<code>boundary_vertices > 0</code> ; так и задумано
«Morse Euler invariant failed»	замкнутая, но не-многообразная сетка (линк вершины — не один цикл) или ошибка после правок классификатора	звёзды вершин, <code>saddle_index_sum</code>
Выбросы кривизны в углах	используются сырые <code>_raw_*</code>	брать <code>gaussian_curvature()</code> и т. п.
«no interior vertices»	компонента сетки шириной в одну грань	мусорные фрагменты после Poisson/RANSAC
Нет точек с <code>essential: true</code>	глобальный минимум на краю	нормально для открытых сканов
<code>--msc-backend ttk requires a closed mesh</code>	открытая сетка	<code>--surface periodic</code> (синтетика) или Poisson с квантилем 0
Полоса точек вдоль строки/столбца	шов <code>--surface periodic</code>	предупреждение CLI о шве

10.10 Вопросы для самопроверки

1. Почему у граничной вершины с пустым верхним линком вклад в эйлерову характеристику равен 0, а у внутренней — 1?
2. Почему в CLI для открытой сетки не вызывается `validate_euler()`?
3. Что означают `destination_id: null` и `destination_vertex_id` у одной дуги?
4. Почему `_extend_to_boundary` не усредняет сырые значения краевых соседей и почему обновление сделано синхронным?
5. Как может получиться, что внутреннее седло имеет большую персистентность, а его партнёра в списке критических точек нет?
6. Чем Poisson с `density_quantile=0` хуже открытой сетки, хотя он замкнут?

Ответы

1. Нижний линк граничной вершины — весь путь, он стягиваем ($\chi = 1$), и вклад равен $1 - 1 = 0$. У внутренней вершины нижний линк — весь цикл ($\chi = 0$), вклад 1: замыкается полость, рождается класс H_2 или гибнет класс H_1 .
2. Формула $\min - \text{saddle} + \max = \chi$ по внутренним точкам для поверхности с краем не выполняется, поэтому `expected_euler = None`.
3. Дуга дошла до замаскированной граничной вершины: ссылаться на критическую точку нельзя, но индекс вершины сетки сохранён.
4. Сырые значения на краю — артефакты (выбросы K в углах, ложный ноль H). Синхронность делает результат независимым от порядка обхода вершин.
5. Пары строятся по всей сетке, включая край. Седло может «убивать» компоненту, рождённую в краевом минимуме, а краевой минимум замаскирован.
6. Часть поверхности дорисована без данных, и критические точки на ней описывают реконструкцию, а не объект.

10.11 Что почитать

- Н. Edelsbrunner, J. Harer. *Computational Topology: An Introduction*. AMS, 2010 — нижняя звезда, персистентность, PL-функции на комплексах.
- Документация проекта: `docs/DATA_CONTRACT.md` (разделы о краевой кривизне и сепаратрисах) и `README.md` (раздел о персистентности на открытых поверхностях).

11. Практикум, учебный маршрут, глоссарий, литература

11.1 Учебный маршрут на 2–4 недели

Маршрут рассчитан на 1–2 часа в день. Каждая неделя заканчивается практическим заданием на коде репозитория. Если заданием недели получилось воспользоваться без подсказок, тему можно считать освоенной.

Неделя	Теория	Практика на коде
1	Главы 1–3: векторы, нормали, PCA; симплексы, линк, граница; $\chi = V - E + F$	Построить <code>grid_mesh</code> и <code>periodic_grid_mesh 5x5</code> , руками посчитать V, E, F и сверить с <code>euler_characteristic</code> . Найти граничные вершины через <code>boundary_vertices</code> .
2	Главы 4–5: критические точки, индекс, подуровневые множества; нижний линк, SoS	Для 3–4 вершин маленькой сетки руками определить тип по линку и сверить с <code>analyze_morse</code> . Сделать плато (равные значения) и посмотреть, как SoS разбивает нищи.
3	Главы 8 и 6: фильтрация нижней звезды, пары, диаграмма, bottleneck; сепаратрисы	Зашумить рельеф, построить график «число критических точек от порога» и диаграмму персистентности. Сравнить два отчёта через <code>morse-lidar-compare</code> .
4	Главы 7, 9, 10: двойственный граф, кривизна, граница	Посчитать shape index на синтетике, сравнить критические точки высоты и кривизны. Прогнать открытый скан с <code>include_boundary=True</code> и без, найти ложные экстремумы на краю.

Если на всё есть только 2 недели, объедините первые две недели в одну, а из главы 4 оставьте разделы про индекс и подуровневые множества. Главы 7 и 10 читайте по мере надобности: при работе с ТТК или реальными открытыми сканами.

11.2 Сквозные упражнения

Упражнения идут по нарастающей. Ответы и подсказки приведены в 11.3.

- Эйлер на сетке.** Регулярная сетка $n \times m$ вершин, каждая клетка разбита диагональю. Выведите формулы для V, E, F и докажите, что $\chi = 1$. Для периодической сетки покажите, что $\chi = 0$.
- Тип вершины.** Вершина со значением 5 имеет шесть соседей по кругу со значениями (3, 7, 4, 8, 2, 9). Определите компоненты нижнего и верхнего линка, тип вершины и кратность седла.
- Тот же линк, другой порядок.** Соседи те же, но по кругу (3, 4, 2, 7, 8, 9). Какой теперь тип?
- Ничья.** Три вершины подряд имеют одинаковое значение 1, а вокруг значения больше. Сколько минимумов найдёт `analyze_morse` и почему ровно столько? Какова персистентность пар, которые

возникают внутри плато?

5. **Баланс.** На замкнутой поверхности нашли 12 минимумов, 30 седел (одно из них обезьянье, остальные простые) и 17 максимумов. Чему равна χ ? Какой это может быть род?
6. **Порог.** Функция на отрезке принимает значения (0, 3, 1, 4, 2.5, 5, 0.5) в точках 0, ..., 6. Найдите все пары (минимум, максимум) по правилу старшего в подуровневой фильтрации. Какие пары останутся при пороге 2?
7. **Устойчивость.** Два скана отличаются шумом амплитуды не больше 0,3 мм в каждой вершине и совпадают по сетке. Какое наибольшее расстояние bottleneck между их диаграммами допускает теорема устойчивости? Почему для двух сканов с *разной* сеткой эта оценка напрямую не применима?
8. **Граница.** Объясните, почему на наклонной плоскости $f(x, y) = x$, заданной на квадрате, при `include_boundary=True` получается минимум и максимум, а при маскировании границы — ни одной критической точки. Почему это не противоречит формуле для χ ?
9. **Кривизна сферы.** Для сферы радиуса 10 мм найдите H , K и shape index. В каких единицах `intrinsic.gaussian_curvature` вернёт K , если координаты в миллиметрах?
10. **Граф.** Тор разбит на ячейки Морса–Смейла функцией с 1 минимумом, 2 седлами и 1 максимумом. Сколько сепаратрис, четырёхугольников, узлов двойственного графа и рёбер каждого цвета? Проверьте, что граф трёхвалентный.

11.3 Ответы и подсказки

1. $V = nm$. Рёбер $n(m - 1)$ горизонтальных, $m(n - 1)$ вертикальных и $(n - 1)(m - 1)$ диагоналей. Треугольников $F = 2(n - 1)(m - 1)$. Подстановка даёт $V - E + F = 1$. На торе $V = E/3 = F/2 = nm$: $V = nm$, $E = 3nm$, $F = 2nm$, поэтому $\chi = 0$.
2. Нижние соседи (значения меньше 5): 3, 4, 2. По кругу они стоят на позициях 1, 3, 5 и разделены верхними, поэтому дают 3 компоненты. Верхние 7, 8, 9 тоже дают 3 компоненты. Это обезьянье седло кратности 2.
3. Нижние 3, 4, 2 стоят подряд — одна компонента. Верхние 7, 8, 9 тоже подряд — одна. Вершина регулярная.
4. Зависит от индексов вершин. По SoS ничья разрешается по индексу, поэтому вершина плато с меньшим индексом считается «ниже». Если индексы идут монотонно вдоль плато, минимум один. Если наименьший индекс у средней вершины — тоже один. Если же у средней вершины наибольший индекс, крайние вершины не соседствуют друг с другом, и обе оказываются минимумами, а средняя — седлом. Такая лишняя пара (минимум, седло) имеет персистентность $1 - 1 = 0$ и отсекается любым положительным порогом.
5. $\chi = 12 - (29 + 2) + 17 = -2$: обезьянье седло считается дважды. По формуле $\chi = 2 - 2g$ род $g = 2$ (крендель).
6. Локальные минимумы: 0 (в точке 0), 1 (в точке 2), 2,5 (в точке 4), 0,5 (в точке 6). Локальные максимумы между ними: 3, 4, 5. Подсказка: идите по уровню вверх и при каждом слиянии убивайте младшую компоненту. Ответ: (1, 3), (2,5, 4), (0,5, 5) и существенный класс 0. При пороге 2 остаются пары (0,5, 5) с персистентностью 4,5 и (1, 3) с персистентностью 2. Пара (2,5, 4) с персистентностью 1,5 отсекается.
7. $d_B \leq 0,3$ мм. Теорема сравнивает две функции на одном пространстве. Разные сетки — это разные пространства, поэтому нужна дополнительная оценка: регистрация сканов и ошибка интерполяции. Отсюда требование эксперимента — фиксированные параметры и согласованная поза.
8. Без маски минимум и максимум лежат на краю: у угловых и граничных вершин просто нет соседей «ниже» или «выше». Формула $\sum (-1)^{\text{ind}} = \chi$ в простом виде верна для замкнутых поверхностей. Для поверхностей с краем нужна версия с граничными слагаемыми, поэтому в коде `expected_euler = None` для открытых сеток.
9. $k_1 = k_2 = 1/R = 0,1 \text{ мм}^{-1}$, $H = 0,1 \text{ мм}^{-1}$, $K = 0,01 \text{ мм}^{-2}$, shape index = +1 (купол) при нормалях наружу. Код единицы не переводит: K будет в мм^{-2} .
10. Сепаратрис $E = 4s$, где s — число простых седел. Ячеек F находим из $V - E + F = \chi$ при $V = \# \text{min} + s + \# \text{max}$: получаем $F = 2s$. Для тора: $V = 4$, $E = 8$, $F = 4$ четырёхугольника и $4 - 8 + 4 = 0$.

Двойственный граф: каждый четырёхугольник делится диагональю min–max на 2 треугольника, всего 8 узлов. Рёбер 12: 4 цвета t (диагонали), 4 цвета u и 4 цвета s (сепаратрисы). Проверка трёхвалентности: $3 \cdot 8 = 2 \cdot 12$.

11.4 Почему Python, а не C++

Короткая справка — этот вопрос обязательно зададут на защите.

- **Основная работа исследовательская:** перебор порогов, полей, способов реконструкции и масок. Здесь важнее скорость проверки гипотезы, чем скорость кода. Python с NumPy и SciPy подходит для этого лучше всего.
- **Тяжёлая топология уже написана на C++.** GUDHI (персистентность) и ТТК (комплекс Морса–Смейла) — библиотеки на C++. Python в проекте только оркеструет их: готовит сетку, вызывает движок, собирает JSON. Так устроено большинство научных пакетов.
- **Собственные циклы на Python** (классификация по линку, трассировка сепаратрис) работают за $O(V \cdot \deg)$. Для сканов в сотни тысяч вершин это секунды. Если понадобится обработка миллионов вершин в реальном времени, переписывать стоит именно эти функции (Numba, Cython или C++), а не весь проект.

11.5 Глоссарий

Термин	English	Где
Барицентрическая площадь	barycentric area	9
Бутылочное горлышко, расстояние	bottleneck distance	8
Верхний / нижний линк	upper / lower link	5
Граница поверхности	boundary	2, 10
Двойственный граф	dual graph	7
Диаграмма персистентности	persistence diagram	8
Звезда вершины	star	2
Индекс критической точки	Morse index	4
Комплекс Морса–Смейла	Morse–Smale complex	6
Критическая точка	critical point	4, 5
Кусочно-линейная функция	piecewise linear (PL) function	5
Линк вершины	link	2, 5
Мульти-седло (частный случай — обезьянье седло)	multi-saddle (monkey saddle)	4, 5
Нижняя звезда, фильтрация	lower-star filtration	8
Персистентность	persistence	8
Подуровневое множество	sublevel set	4, 8
Правило старшего	elder rule	8
Сепаратриса	separatrix	6
Симуляция простоты	simulation of simplicity (SoS)	5
Существенный класс	essential class	8
Трёхвалентный граф	trivalent (cubic) graph	7
Угловой дефект	angle defect	9
Числа Бетти	Betti numbers	8
Эйлерова характеристика	Euler characteristic	3
Shape index (индекс формы)	shape index	9

11.6 Литература

Базовые книги (в порядке полезности для проекта):

1. Edelsbrunner H., Harer J. *Computational Topology: An Introduction*. AMS, 2010. Главное пособие: симплициальные комплексы, теория Морса, PL-функции, персистентность, устойчивость.
2. Edelsbrunner H., Harer J., Zomorodian A. Hierarchical Morse–Smale complexes for piecewise linear 2-manifolds. *Discrete & Computational Geometry*, 2003, 30(1). Ровно наш случай: комплекс Морса–Смейла на триангулированной поверхности.
3. Cohen-Steiner D., Edelsbrunner H., Harer J. Stability of persistence diagrams. *Discrete & Computational Geometry*, 2007, 37(1). Теорема устойчивости, на которой держится сравнение сканов.
4. Edelsbrunner H., Mücke E. Simulation of simplicity. *ACM Transactions on Graphics*, 1990, 9(1). Источник SoS.

5. Banchoff T. Critical points and curvature for embedded polyhedra. *Journal of Differential Geometry*, 1967, 1. Классификация критических точек PL-функции.
6. Милнор Дж. *Теория Морса*. М.: Мир, 1965. Классика гладкой теории, часть I.
7. Meyer M., Desbrun M., Schröder P., Barr A. Discrete differential-geometry operators for triangulated 2-manifolds. *Visualization and Mathematics III*, 2003. Котангенсная формула и угловой дефект.
8. Tierny J. et al. The Topology ToolKit. *IEEE TVCG*, 2018. Описание ТТК.
9. Документация GUDHI: gudhi.inria.fr (SimplexTree, CubicalComplex, bottleneck_distance).